

C# và ASP.NET Core — Tập 3

Web nâng cao: kiến trúc sạch, mẫu thiết kế, vận hành, bảo mật và triển khai — giả định đã nắm Tập 1 và Tập 2

https://github.com/phamtuanchip/csharp_book

Mục lục

Phần 0 — Kiến trúc

1. Từ khối đơn nhất đến kiến trúc phân lớp
2. Clean Architecture thực hành
3. Mô hình hoá miền (DDD cơ bản)
4. Repository, Unit of Work và Specification

Phần 1 — Mẫu thiết kế nâng cao

5. CQRS và Mediator
6. Pipeline behavior: validation, logging, transaction
7. Result pattern và validation
8. Sự kiện miền, xử lý nền và Outbox

Phần 2 — Chất lượng vận hành

9. Chiến lược kiểm thử
10. Caching
11. Quan sát và hiệu năng

Phần 3 — Bảo mật nâng cao

12. OpenID Connect và OAuth 2.0
13. Idempotency, audit log và bảo vệ dữ liệu

Phần 4 — Đóng gói và triển khai

14. Docker và docker-compose
15. CI/CD với GitHub Actions
16. Triển khai lên máy chủ và đám mây
17. Microservices và giao tiếp giữa dịch vụ

Phần 5 — Dự án tổng hợp

18. Dự án tổng hợp Tập 3: Hệ thống đặt hàng

Phụ lục

19. Phụ lục A — Lỗi và bẫy kiến trúc thường gặp
20. Phụ lục B — Tài liệu tham khảo
21. Phụ lục C — Cheat sheet kiến trúc và triển khai

Chương 1 — Từ khối đơn nhất đến kiến trúc phân lớp

Mục tiêu học

Sau chương này, bạn sẽ:

- Nhận ra các dấu hiệu của "**quả cầu bùn**" (**big ball of mud**) và hiểu vì sao kiến trúc quan trọng khi hệ thống lớn dần.
- Nắm **Layered**, **Onion**, **Hexagonal (Ports & Adapters)**, **Clean Architecture** và điểm chung: **quy tắc phụ thuộc**.
- Biết cách chia solution thành **Domain / Application / Infrastructure / Web**.
- Biến quy tắc kiến trúc thành **kiểm thử tự động**.

Code mẫu: [code/ch01-kien-truc/](#) — bộ khung 4 tầng cực nhỏ kèm 6 **architecture test** (đều đạt).

Tập 3 giả định bạn đã đọc Tập 1–2. Dự án Chương 22 (Tập 2) chính là "điểm xuất phát" ta sẽ tái cấu trúc.

Vấn đề: hệ thống lớn lên

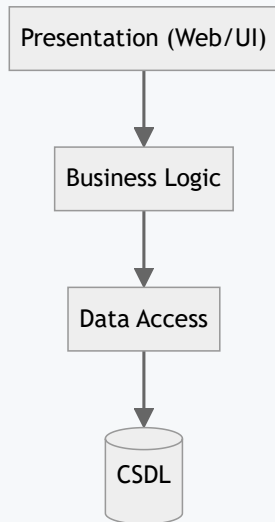
Dự án Chương 22 chạy tốt: endpoint gọi `KhoService`, `KhoService` dùng thẳng `DbContext`. Với 4 màn hình điều đó ổn. Nhưng sau hai năm, 40 đội tính năng, 300 endpoint:

- Logic nghiệp vụ **rải khắp**: một phần trong endpoint, một phần trong service, một phần trong câu SQL, một phần trong JavaScript.
- Thay đổi cách lưu trữ (đổi CSDL, thêm cache) làm **vỡ** nghiệp vụ; sửa nghiệp vụ lại dựa vào code truy cập dữ liệu.
- **Không test được** nghiệp vụ nếu không dựng cả CSDL và HTTP.
- Ai cũng gọi mọi thứ: `Controller` → `DbContext` → `Controller` khác ... phụ thuộc chằng chịt (vòng tròn).
- Người mới mất vài tuần để hiểu "cái gì ở đâu".

Đó là **big ball of mud**: mọi thứ phụ thuộc mọi thứ. Kiến trúc là tập quy tắc **giới hạn ai được phụ thuộc ai** để thay đổi ở một nơi không lan ra khắp nơi.

Đừng vội "kiến trúc hoá". Với ứng dụng CRUD nhỏ, kiến trúc một khối như Tập 2 là **lựa chọn đúng** (đơn giản hơn = ít lỗi hơn). Kiến trúc phân lớp trả giá bằng nhiều project, nhiều ánh xạ, nhiều "nghịch thức" — chỉ đáng khi **độ phức tạp nghiệp vụ** hoặc **quy mô đội/vòng đời** đủ lớn. Hãy tiến hoá dần: bắt đầu đơn giản, tách khi cơn đau xuất hiện.

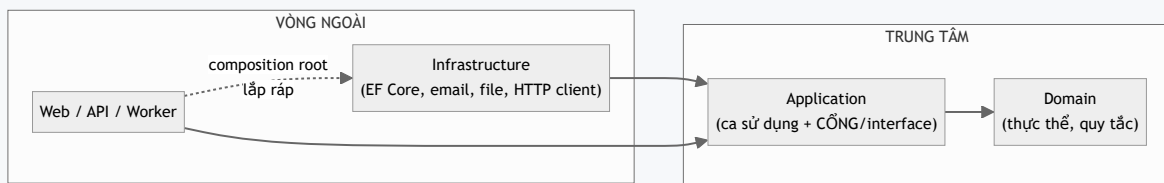
Kiến trúc phân lớp truyền thống (N-tier)



Phụ thuộc đi **một chiều từ trên xuống**: giao diện → nghiệp vụ → truy cập dữ liệu. Đơn giản, quen thuộc. Vấn đề: **nghiệp vụ phụ thuộc vào tầng dữ liệu** (nó gọi DbContext /repository cụ thể) — nên kiểu CSDL, cấu trúc bảng, ORM "rò rỉ" vào nghiệp vụ; nghiệp vụ khó test và bị ràng buộc với công nghệ.

Đảo ngược phụ thuộc: Onion / Hexagonal / Clean

Ba tên gọi (Jeffrey Palermo — *Onion*, Alistair Cockburn — *Hexagonal/Ports & Adapters*, Robert Martin — *Clean Architecture*) cùng một ý tưởng: **nghiệp vụ ở trung tâm, không phụ thuộc gì cả; mọi thứ kỹ thuật ở ngoài phụ thuộc vào nghiệp vụ**.



Quy tắc phụ thuộc (Dependency Rule): mã nguồn chỉ được phụ thuộc vào vòng trong, không bao giờ vào vòng ngoài. Mũi tên tham chiếu (`ProjectReference`) luôn chỉ **vào trung tâm**.

Làm sao nghiệp vụ dùng CSDL mà không phụ thuộc vào nó? Nhờ **Dependency Inversion** (Tập 1, Chương 42): tầng Application **định nghĩa interface** (cổng — *port*) `ISanPhamRepository` ; tầng Infrastructure **cài đặt** (bộ điều hợp — *adapter*). Nghiệp vụ chỉ biết cổng.

```
// Application/SanPhamService.cs – biết Domain, KHÔNG biết EF hay Web
public interface ISanPhamRepository { Task<SanPham?> LayAsync(int id, CancellationToken ct); ... }
public class SanPhamService(ISanPhamRepository repo) { ... }

// Infrastructure/... – cài đặt cổng bằng EF Core / in-memory / API ngoài
public class InMemorySanPhamRepository : ISanPhamRepository { ... }

// Web/Program.cs – COMPOSITION ROOT: nơi duy nhất biết tất cả và nối chúng lại
builder.Services.AddSingleton<ISanPhamRepository, InMemorySanPhamRepository>();
```

Lợi ích:

- **Test nghiệp vụ không cần hạ tầng:** thay repository bằng bản giả trong bộ nhớ.
- **Thay công nghệ dễ:** đổi SQL Server → PostgreSQL, thêm cache, gọi API thay vì CSDL — chỉ viết adapter mới.
- **Nghệp vụ tập trung** một chỗ, đọc như "ngôn ngữ của doanh nghiệp".
- Framework ([ASP.NET](#), EF) là **chi tiết**, không phải trung tâm ứng dụng — ứng dụng "sống" lâu hơn framework.

Bốn tầng và trách nhiệm

Tầng (project)	Chứa gì	Được tham chiếu tới
Domain	thực thể, value object, quy tắc nghiệp vụ, sự kiện miền, interface thuần miền	không gì cả
Application	ca sử dụng (command/query handler, service), cổng (IRepository, IEmailSender, IClock), DTO	Domain
Infrastructure	EF Core (DbContext, migration, repository), gửi email, gọi API ngoài, file, cache	Application (+ Domain)
Web/API/Worker	endpoint/controller, DI (composition root), middleware, cấu hình	Application, Infrastructure

Cấu trúc solution mẫu (ch01-kien-truc):

```
KienTruc.slnx
├─ KienTruc.Domain/           # SanPham – không tham chiếu gì
├─ KienTruc.Application/      # ISanPhamRepository + SanPhamService – tham chiếu Domain
├─ KienTruc.Infrastructure/   # InMemorySanPhamRepository – tham chiếu Application
├─ KienTruc.Web/              # Program.cs (composition root) – tham chiếu Application + Infrastructure
└─ KienTruc.Tests/           # kiểm thử kiến trúc
```

Lưu ý: Infrastructure **tham chiếu** Application (để cài đặt interface của nó) chứ **không** ngược lại. Đây là chỗ "đảo ngược" so với N-tier (ở đó nghiệp vụ tham chiếu dữ liệu).

Biến quy tắc thành kiểm thử

Quy tắc kiến trúc chỉ tồn tại trên giấy nếu không có gì ngăn ai đó thêm `using Microsoft.EntityFrameworkCore;` vào Domain "cho tiện". **Architecture test** giải quyết:

```
[Fact]
public void Domain_KhongPhuThuocBatKyThuGiGi()
{
    Assert.Empty(ThamChieuDuAn("KienTruc.Domain")); // đọc <ProjectReference> trong .csproj
    Assert.Empty(GoiNuGet("KienTruc.Domain"));       // và <PackageReference>
}

[Fact] public void Application_ChiPhuThuocDomain() => Assert.Equal(["KienTruc.Domain"], ThamChieuDuAn

[Fact]
public void Domain_KhongThamChieuAssemblyHaTang()
{
    var cam = new[] { "Microsoft.EntityFrameworkCore", "Microsoft.AspNetCore", "System.Data", "Newton:

```

```
var duocGoi = typeof(SanPham).Assembly.GetReferencedAssemblies().Select(a => a.Name!);
Assert.DoesNotContain(duocGoi, ten => cam.Any(c => ten.StartsWith(c))); // kiểm tra cả ở mức
}
```

Trong CI, test hỏng $\Rightarrow$ pull request bị chặn. Thư viện **NetArchTest.Rules** hoặc **ArchUnitNET** viết được quy tắc phong phú hơn (kiểu/namespace/thuộc tính): "mọi lớp `*Handler` nằm trong Application", "lớp trong Domain không `public set` "... Mẫu tự viết ở trên đủ cho bốn quy tắc nền tảng.

Cách tổ chức trong mỗi tầng: theo chiều dọc

Trong Application, đừng gom mọi thứ theo *loại kỹ thuật* (`Services/` , `Dtos/` , `Validators/` — thư mục nào cũng phình to). Nhóm theo **tính năng** (vertical slice):

```
Application/
├─ SanPham/
│   ├─ ThemSanPham/      (Command, Handler, Validator, Result)
│   └─ TimSanPham/       (Query, Handler, Dto)
└─ DonHang/ ...
```

Mọi thứ cho một ca sử dụng nằm cạnh nhau: dễ tìm, dễ xoá, ít xung đột merge (Chương 5).

Khi nào chọn gì?

Tình huống	Gợi ý
CRUD nhỏ, đội 1–3 người, vòng đời ngắn	một project, vertical slice đơn giản (như Tập 2)
Nghiệp vụ phức tạp, nhiều quy tắc, sống nhiều năm	Clean Architecture / DDD
Nhiều đội độc lập, cần deploy riêng	Modular monolith (mỗi module một bộ Domain/Application/Infra, giao tiếp qua hợp đồng) $\rightarrow$ cân nhắc microservices khi thật cần (Chương 17)
Cần thay CSDL/nhà cung cấp linh hoạt, test nghiệp vụ dày	Ports & Adapters

Modular monolith (một ứng dụng triển khai một khối, nhưng chia module có ranh giới rõ, không truy cập bằng của nhau) là điểm khởi đầu thực dụng nhất cho đa số hệ thống lớn: hưởng lợi kiến trúc mà chưa phải trả chi phí phân tán.

Chi phí và cạm bẫy

- **Nhiều "nghỉ thức"**: DTO $\leftrightarrow$ entity $\leftrightarrow$ view model; interface cho mọi thứ. Mỗi lớp thêm ma sát — chỉ đáng khi nó bảo vệ một ranh giới thật.
- **Anemic domain**: Domain chỉ là túi dữ liệu (`get/set`), mọi logic dồn vào service — mất hết lợi ích (Chương 3).
- **Đảo ngược quá mức**: interface cho thứ không bao giờ đổi (`IDateTimeWrapper` cho mọi hàm) — chỉ đảo ngược ở **ranh giới I/O** (CSDL, mạng, thời gian, file).
- **"Tầng Shared/Common" phình to**: thùng rác kéo mọi phụ thuộc lại với nhau.
- Kiến trúc đẹp trên giấy nhưng đội không tuân thủ $\rightarrow$ dùng architecture test và review.

Lỗi thường gặp

- Domain tham chiếu `Microsoft.EntityFrameworkCore` (đặt attribute EF trong entity) — dùng Fluent API bên Infrastructure.
- Application gọi thẳng `DbContext` (chưa qua cổng) rồi vẫn gọi là "Clean".
- Infrastructure tham chiếu Web hoặc ngược lại tạo vòng.
- Đặt logic nghiệp vụ trong controller/endpoint hoặc trong Infrastructure (SQL/stored procedure).
- Chia project nhưng không có test kiến trúc → sau 6 tháng quy tắc bị phá.
- Áp Clean Architecture cho ứng dụng CRUD 5 bảng.

Bài tập

1. Thêm `KienTruc.Infrastructure` một cài đặt thứ hai của `ISanPhamRepository` (đọc từ file JSON) và đổi bằng một dòng ở composition root.
2. Viết architecture test: mọi lớp trong `KienTruc.Domain` **không** có property `public set` (dùng reflection).
3. Cố tình thêm `using Microsoft.AspNetCore...;` và `<PackageReference>` vào Domain rồi xem test nào đổ.
4. Vẽ sơ đồ các tầng cho dự án Chương 22 sau tái cấu trúc: `KhoService` đi về tầng nào? `KhoDb` ? `JwtOptions` ?
5. Liệt kê ba ràng buộc nghiệp vụ trong Chương 22 và quyết định chúng nằm ở Domain, Application hay Infrastructure.

Chương 2 — Clean Architecture thực hành

Mục tiêu học

Sau chương này, bạn sẽ:

- Đọc hiểu một solution Clean Architecture hoàn chỉnh: **Domain / Application / Infrastructure / Web / Tests**.
- Theo dõi **một request** đi qua các tầng, từ HTTP đến CSDL và ngược lại.
- Hiểu vai trò của **composition root**, DI theo tầng, và **migration** trong kiến trúc nhiều project.
- Biết cấu trúc thư mục theo **tính năng (vertical slice)**.

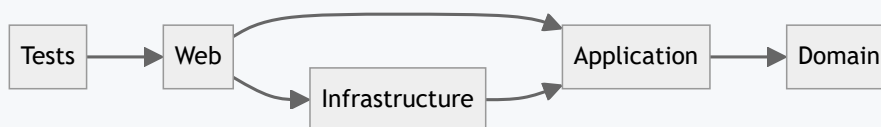
Code: [code/kho-clean/](https://code.kho-clean/) — hệ thống kho của Tập 2 được **tái cấu trúc** (45 test đều đạt). Chương 2–8 và 13 lần lượt giải thích từng phần của cùng một solution.

Bản đồ solution

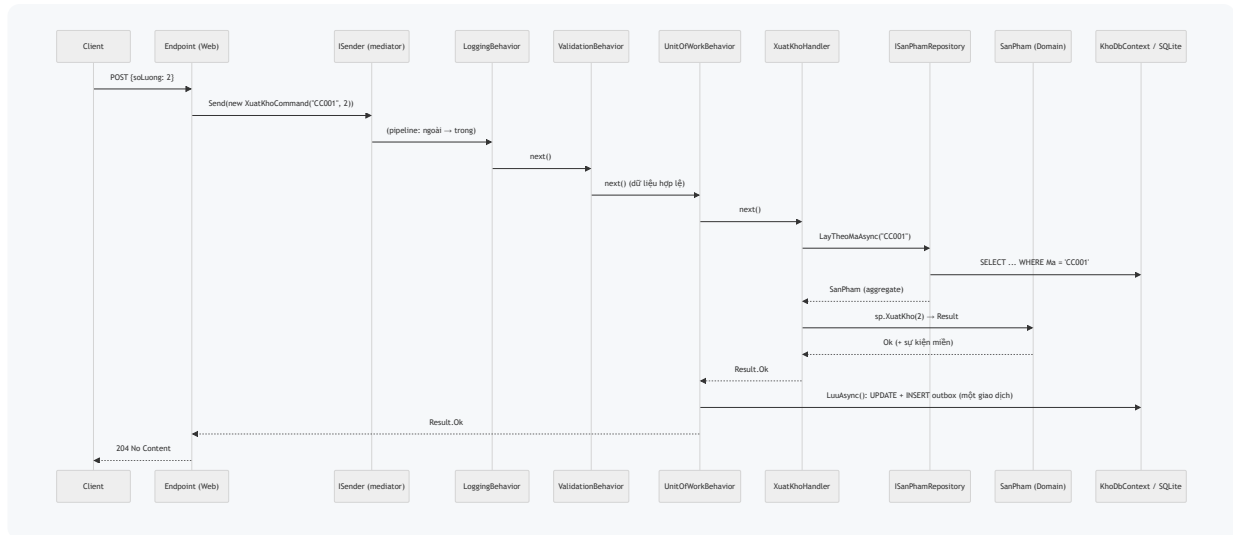
```
kho-clean/
├─ Kho.Domain/
│   └─ Common/
│       └─ SanPhams/
├─ Kho.Application/
│   └─ Abstractions/
│   └─ Messaging/
│   └─ Behaviors/
│   └─ SanPhams/
│   └─ DependencyInjection.cs
├─ Kho.Infrastructure/
│   └─ Persistence/
│   └─ Outbox/
│   └─ DependencyInjection.cs
├─ Kho.Web/
│   └─ Endpoints/
│   └─ Program.cs
└─ Kho.Tests/
```

KHÔNG tham chiếu gì
Result, Loi, Entity, IDomainEvent
SanPham (aggregate), MaSanPham, Tien (value object), sự kiện
→ Domain
CỔNG: ISanPhamRepository, IUnitOfWork, IKhoDocDuLieu, IPhatHanhSi
ICommand/IQuery/IRequestHandler + mediator tự viết
Logging, Validation, UnitOfWork
ca sử dụng: Thêm, Nhập, Xuất, Tìm, Lấy
services.AddApplication()
→ Application
KhoDbContext, Repositories, Migrations
OutboxProcessor (BackgroundService)
services.AddInfrastructure(cfg)
→ Application + Infrastructure (composition root)
Minimal API mỏng, dịch Result → HTTP
Domain / Application / Architecture / Integration

Mũi tên → là ProjectReference ; test kiến trúc (Chương 1) ép đúng chiều đó.



Đi theo một request: POST /api/san-pham/CC001/xuat



Mỗi tầng chỉ làm đúng việc của nó:

Tầng	Việc làm trong request này	Không làm
Endpoint (Web)	bind JSON → tạo <code>XuatKhoCommand</code> , gọi <code>sender.Send</code> , dịch <code>Result</code> → 204/404/422/409	ng nghiệp vụ, truy cập CSDL
Behavior	log, validate đầu vào, lưu sau khi thành công	ng nghiệp vụ cụ thể
Handler (Application)	điều phối: tải aggregate, gọi phương thức nghiệp vụ	quy tắc "đủ hàng" (thuộc Domain), SQL
Domain	XuatKho kiểm tra đủ hàng, giảm tồn, phát sự kiện	biết CSDL, HTTP
Infrastructure	SQL, giao dịch, outbox	quyết định nghiệp vụ

Endpoint mởng

```
g.MapPost("/{ma}/xuat", async (string ma, SoLuongRequest r, ISender sender, CancellationToken ct)
=> (await sender.Send(new XuatKhoCommand(ma, r.SoLuong), ct)).ToHttp(noContent: true));
```

Toàn bộ endpoint là **một dòng**. Điểm dịch duy nhất từ ngôn ngữ nghiệp vụ sang HTTP:

```
private static IResult LoiSangHttp(Loi loi) => Results.Problem(title: loi.Ma, detail: loi.MoTa, statu:
{
    LoiLoi.DuLieuKhongHopLe => 400,
    LoiLoi.KhongTimThay => 404,
    LoiLoi.XungDot => 409,
    LoiLoi.NghiepVu => 422,
    _ => 500,
});
```

Thay HTTP bằng gRPC, hàng đợi hay giao diện console: chỉ viết lại lớp dịch này; **Application và Domain không đổi**.

Composition root và DI theo tầng

Mỗi tầng tự khai báo những gì nó cung cấp bằng một extension method; `Program.cs` chỉ ghép:

```
builder.Services.AddApplication(); // mediator + behaviors + validators
builder.Services.AddInfrastructure(builder.Configuration); // EF, repository, UoW, outbox
```

```
// Infrastructure/DependencyInjection.cs
services.AddDbContext<KhoDbContext>(o => o.UseSqlite(cfg.GetConnectionString("Kho")) ?? "Data Source=kl
services.AddScoped<IUnitOfWork>(sp => sp.GetRequiredService<KhoDbContext>()); // cùng MỘT DbContext
services.AddScoped<ISanPhamRepository, SanPhamRepository>();
services.AddScoped<IKhoDocDuLieu, KhoDocDuLieu>();
```

`IUnitOfWork` được ánh xạ về chính `KhoDbContext` để repository và UoW **chia sẻ một context** trong cùng request. Ứng dụng đối công nghệ chỉ cần thay `AddInfrastructure`.

Migration khi EF nằm ở tầng Infrastructure

`DbContext` và migration thuộc `Kho.Infrastructure`, nhưng công cụ `dotnet ef` cần một **startup project** chạy được (`Kho.Web`, chứa cấu hình):

```
cd Kho.Web
dotnet ef migrations add KhoiTao -p ../Kho.Infrastructure -s . -o Persistence/Migrations -- --Migrate
```

`-p` (project chứa `DbContext`), `-s` (startup project), `-o` (thư mục sinh), `--` truyền tham số cho ứng dụng (tắt migrate khi khởi động để công cụ không mở CSDL). Package `Microsoft.EntityFrameworkCore.Design` đặt ở `Kho.Web`.

Tổ chức Application theo tính năng

`Application/SanPhams/CaSuDung.cs` gom **mọi thứ của một ca sử dụng cạnh nhau**: command, validator, handler:

```
public sealed record XuatKhoCommand(string Ma, int SoLuong) : ICommand<Result>;
public sealed class XuatKhoValidator : AbstractValidator<XuatKhoCommand> { ... }
public sealed class XuatKhoHandler(ISanPhamRepository repo, TimeProvider dongHo) : IRequestHandler<Xu
```

Khi dự án lớn, mỗi ca sử dụng một thư mục/file (`SanPhams/XuatKho/...`). Thêm tính năng = thêm file mới, hầu như **không sửa file cũ** (Open/Closed).

Kiểm thử theo tầng

Loại test	Ví dụ trong <code>Kho.Tests</code>	Phụ thuộc	Tốc độ
Domain	<code>XuatKho_KhongDuHang_ThatBaiVaTonKhongDoi</code> , <code>MaSanPham_ChiChapNhanDungDinhDang</code>	không gì cả	µs
Application	<code>ThemSanPham_SaiDinhDang_ValidationBehaviorChanTruocHandler</code> — repository, UoW giả	DI + fake	ms
Architecture	Domain không phụ thuộc gì; aggregate không có setter công khai	file csproj, reflection	ms

Loại test	Ví dụ trong Kho.Tests	Phụ thuộc	Tốc độ
Integration	XuatDongThoi_KhongBaoGioAm_..., outbox — HTTP + SQLite thật	toàn bộ	chục ms

Chiến lược đầy đủ ở Chương 9. Điểm mấu chốt: **phần lớn test rơi vào Domain/Application** (nhanh, ổn định); ít test tích hợp hơn nhưng phủ luồng chính và ghép nối.

So sánh với Tập 2

	Tập 2 (khối đơn giản)	Tập 3 (Clean)
Nghệ vụ nằm ở	KhoService dùng thẳng DbContext	SanPham (Domain) + handler (Application)
Test nghiệp vụ cần	CSDL + HTTP	không cần gì (Domain) / bản giả (Application)
Đổi CSDL	sửa nhiều chỗ	thay AddInfrastructure
Số file	ít	nhiều hơn ~2–3 lần
Hợp với	CRUD, đội nhỏ, ít nghiệp vụ	nghệ vụ phức tạp, nhiều người, sống lâu

Chi phí có thật (nhiều file, nhiều ảnh xạ). Hãy dùng kiến trúc này khi độ phức tạp nghiệp vụ **xứng đáng** với nó.

Lỗi thường gặp

- Đặt DbContext / DbSet trong Application vì "tiện" — vi phạm quy tắc phụ thuộc.
- Quên services.AddScoped<IUnitOfWork>(sp => sp.GetRequiredService<KhoDbContext>()) → repository và UoW dùng hai context khác nhau, SaveChanges không thấy thay đổi.
- Chạy dotnet ef ở thư mục sai (-p , -s thiếu) hoặc quên package Design ở startup project.
- Để endpoint chứa if (sp.TonKho < n) — nghiệp vụ rò ra Web.
- Application trả entity/aggregate ra ngoài thay vì DTO/ Result .
- Quên test kiến trúc; sau vài tháng tầng Domain tham chiếu EF.

Bài tập

1. Thêm ca sử dụng DoiGiaCommand(string Ma, decimal GiaMoi) (command + validator + handler) mà **không sửa** file nào hiện có ngoài đăng ký (assembly scan tự nhận).
2. Thêm endpoint PUT /api/san-pham/{ma}/gia gọi ca sử dụng ở trên; viết test tích hợp.
3. Vẽ sơ đồ tuần tự cho POST /api/san-pham (thêm sản phẩm) tương tự sơ đồ ở trên.
4. Thay SanPhamRepository bằng bản lưu file JSON (chỉ tạo class mới + đổi AddInfrastructure). Test Application có cần đổi không?
5. Liệt kê những chỗ trong solution mà đổi SQLite → PostgreSQL phải sửa (gợi ý: ConfigureConventions , Like , kiểu ngày giờ).

Chương 3 — Mô hình hoá miền (DDD cơ bản)

Mục tiêu học

Sau chương này, bạn sẽ:

- Phân biệt **Entity**, **Value Object**, **Aggregate**, **Domain Event** và biết khi nào dùng cái nào.
- Viết domain **giàu hành vi** (rich domain model) thay vì túi dữ liệu (anemic).
- Bảo vệ **bất biến (invariant)** ở chính lớp miền; dùng `Result` cho lỗi nghiệp vụ.
- Ánh xạ domain sang EF Core **mà không làm bẩn** domain.

Code: [code/kho-clean/Kho.Domain/](#) và [DomainTests.cs](#).

Domain-Driven Design (DDD) là gì?

DDD (Eric Evans) là cách tiếp cận thiết kế cho **nghiệp vụ phức tạp**: mô hình phần mềm phản ánh **ngôn ngữ và quy tắc của doanh nghiệp**, do lập trình viên và chuyên gia nghiệp vụ cùng xây. Hai nhóm ý tưởng:

- **Chiến lược**: *ubiquitous language* (ngôn ngữ chung: "xuất kho", "sắp hết hàng" dùng thống nhất trong code, tài liệu, họp), *bounded context* (mỗi phần hệ thống có mô hình riêng; "Sản phẩm" trong Kho khác "Sản phẩm" trong Marketing).
- **Chiến thuật** (bộ khối xây dựng, chương này): entity, value object, aggregate, domain event, repository, domain service.

Sách này dùng **DDD chiến thuật ở mức thực dụng**. DDD đầy đủ (event storming, context mapping) là một chủ đề lớn — xem Phụ lục B. Áp dụng khi nghiệp vụ *thật sự phức tạp*; CRUD thuần không cần.

Anemic vs Rich domain model

```
// ✗ ANEMIC: chỉ là túi dữ liệu; luật nằm ở service → ai cũng bỏ qua được luật
public class SanPham { public int TonKho { get; set; } }
class KhoService { void Xuat(SanPham sp, int n) { if (sp.TonKho >= n) sp.TonKho -= n; } } // quên kiểm tra

// ✓ RICH: dữ liệu + hành vi + luật ở CÙNG MỘT nơi
public Result XuatKho(int soLuong, DateTimeOffset bayGio)
{
    if (soLuong <= 0) return Result.Fail(...);
    if (soLuong > TonKho) return Result.Fail(Loi.NghiepVu("SanPham.KhongDuHang", ...));
    TonKho -= soLuong;
    ...
}
```

Trong domain giàu, **không có cách nào** làm `TonKho` âm ngoài phương thức này (setter `private`). Test kiến trúc khoá điều đó: `Aggregate_KhongCoSetterCongKhai`. Đây là tinh thần của *encapsulation* (Tập 1, Chương 14) nâng lên cấp kiến trúc: **object luôn ở trạng thái hợp lệ**.

Value Object

Value object đại diện một **giá trị** — *không có danh tính, bất biến*, so sánh **theo giá trị**, tự bảo đảm hợp lệ. Ví dụ: mã sản phẩm, số tiền, địa chỉ, khoảng ngày.

```
public sealed partial record MaSanPham
{
    public string GiaTri { get; }
    private MaSanPham(string giaTri) => GiaTri = giaTri;

    public static Result<MaSanPham> Tao(string? ma)
    {
        if (string.IsNullOrEmpty(ma)) return Result<MaSanPham>.Fail(Loi.DuLieuKhongHopLe("SanPham"));
        ma = ma.Trim().ToUpperInvariant();
        return DinhDang().IsMatch(ma) ? new MaSanPham(ma) : Result<MaSanPham>.Fail(Loi.DuLieuKhongHopLe("SanPham"));
    }
    [GeneratedRegex("[A-Z]{2}[0-9]{3}$")] private static partial Regex DinhDang();
}
```

Lợi ích:

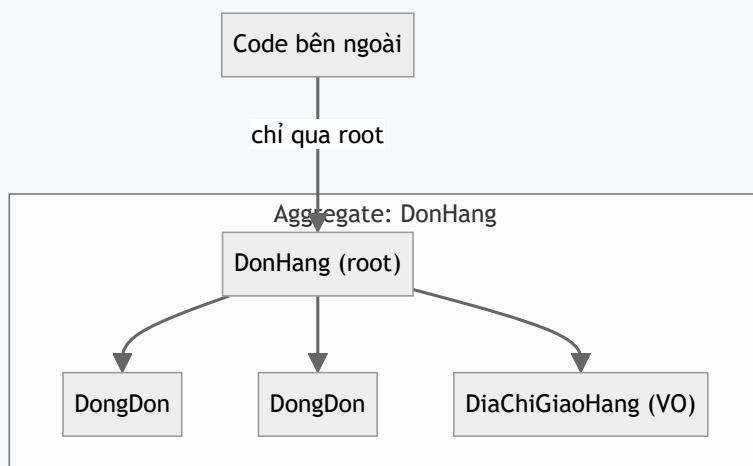
- **Không thể tồn tại "mã sản phẩm sai"** trong hệ thống: constructor `private`, chỉ có factory `Tao` trả `Result`. Kiểm tra đúng **một chỗ**, không rải rác.
- **Chống "primitive obsession"** (dùng `string` / `decimal` tràn khắp nơi): `XuatKho(string ma, decimal gia)` dễ đảo nhầm tham số; `XuatKho(MaSanPham ma, Tien gia)` thì không.
- Hành vi gắn với giá trị: `Tien` có `+`, `*`, làm tròn VND.
- `record` cho sẵn so sánh theo giá trị (`MaSanPham.Tao("LT001") == MaSanPham.Tao("lt001")`) — test `MaSanPham_SoSanhTheoGiaTri`).

Dùng `readonly record struct` cho VO nhỏ, thường xuyên tạo (`Tien`, `Toado`) nếu cần hiệu năng; `record class` khi có nhiều trường/logic.

Entity và Aggregate

Entity: có **danh tính** (`Id`) tồn tại suốt vòng đời, hai entity bằng nhau nếu cùng `Id` dù thuộc tính khác. `SanPham` là entity.

Aggregate: cụm entity/value object được đối xử như **một đơn vị nhất quán**, có một **aggregate root** là cổng duy nhất để thay đổi:



Quy tắc aggregate (giúp giữ nhất quán và hiệu năng):

1. **Chỉ truy cập/sửa qua root**; không giữ tham chiếu tới entity con bên trong từ bên ngoài.
2. **Một giao dịch = một aggregate**: mỗi lần `SaveChanges` chỉ nên đổi một aggregate; nhất quán *giữa các aggregate* dùng **sự kiện** và nhất quán cuối cùng (Chương 8).
3. Aggregate **nhỏ**: chỉ chứa những gì cần nhất quán tức thời. `SanPham` không chứa danh sách hàng nghìn giao dịch — giao dịch là aggregate/bảng khác, sinh ra từ sự kiện.
4. Tham chiếu aggregate khác **bằng Id**, không bằng object.

Trong solution, `SanPham` là aggregate root độc lập (tồn kho là bất biến cần bảo vệ). Trong hệ thống có đơn hàng, `DonHang` tham chiếu `MaSanPham` (Id), không nhúng `SanPham`.

Domain Event

Sự kiện miền: điều gì **đã xảy ra** trong nghiệp vụ, đặt tên thì **quá khứ**, bất biến.

```
public sealed record TonKhoThayDoi(string Ma, int TonCu, int TonMoi, string LyDo, DateTimeOffset XayRa, IDomainEvent? Event = null) : IDomainEvent
{
    public sealed record SapHetHang(string Ma, int TonKho, int MucCanhBao, DateTimeOffset XayRaLuc) : IDomainEvent
    {
        public Result XuatKho(int soLuong, DateTimeOffset bayGio)
        {
            ...
            TonKho -= soLuong;
            PhatSuKien(new TonKhoThayDoi(Ma.GiaTri, cu, TonKho, "Xuat kho", bayGio));

            // Quy tắc nghiệp vụ: vừa CHẠM ngưỡng (từ trên xuống) thì phát sự kiện – chỉ một lần
            if (cu > MucCanhBao && TonKho <= MucCanhBao)
                PhatSuKien(new SapHetHang(Ma.GiaTri, TonKho, MucCanhBao, bayGio));
            return Result.Ok();
        }
    }
}
```

Aggregate **không biết** ai quan tâm (gửi email? báo mua hàng?) — chỉ ghi nhận sự kiện. Nhờ vậy domain giữ thuần và các hệ quả (tác dụng phụ) tách khỏi luật (Chương 8 xử lý bằng outbox). Test kiểm chứng thời điểm phát:

```
sp.XuatKho(3, Gio); // 7 -> 4: chạm ngưỡng 5
sp.XuatKho(1, Gio); // 4 -> 3: đã dưới ngưỡng, KHÔNG phát lại
Assert.Single(sp.SuKienMien.OfType<SapHetHang>());
```

Lưu ý: sự kiện chỉ nên **phát sau khi thay đổi thành công** — `XuatKho` thất bại không phát gì (test `XuatKho_KhongDuHang_...`).

Lỗi nghiệp vụ là *giá trị*

Vi phạm luật (hết hàng, dữ liệu sai) là **kết quả có thể đoán trước**, không phải "ngoại lệ". Domain trả `Result` (Chương 7) với `Loi` mang **mã ổn định** (`"SanPham.KhongDuHang"`) và **loại** (`LoiLoi.NghiepVu`) — tăng ngoài dịch sang HTTP/log/UI tùy ý. Exception dành cho lỗi **lập trình** (truy cập `GiaTri` của `Result` thất bại, tham số null) và lỗi hạ tầng.

Domain service

Logic nghiệp vụ **không thuộc riêng aggregate nào** (ví dụ tính phí ship dựa trên nhiều aggregate, chuyển kho giữa hai `Kho`) đặt trong **domain service** — lớp không trạng thái trong tầng Domain, nhận aggregate làm tham số. Đừng vội tạo `xxxService`: hầu hết logic nên nằm trong entity/VO; chỉ tách khi thật sự "không có nhà".

Ánh xạ sang EF Core mà không làm bẩn domain

Domain **không** có attribute EF, không cần `virtual`. Mọi cấu hình nằm ở Infrastructure (Fluent API):

```
mb.Entity<SanPham>(e =>
{
    e.Ignore(s => s.SuKienMien); // sự kiện không |
    e.Property(s => s.Ma).HasConversion(m => m.GiaTri, v => MaSanPham.TuCsd1(v)) // value object <
        .HasMaxLength(20).IsRequired();
    e.Property(s => s.DonGia).HasConversion(t => (double)t.SoTien, v => Tien.TuCsd1((decimal)v));
    e.Property(s => s.TonKho).IsConcurrencyToken();
    e.HasIndex(s => s.Ma).IsUnique();
});
```

- **Constructor** `private SanPham() { }` cho EF tạo đối tượng rồi gán qua *backing field/private setter* — EF làm được với setter `private`.
- **Tái tạo từ CSDL**: `MaSanPham.TuCsd1(v)` bỏ qua kiểm tra (dữ liệu đã hợp lệ lúc ghi); factory `Tao` dành cho **đầu vào bên ngoài**. Đặt tên rõ để không dùng nhầm.
- `HasConversion` cho value object đơn giá trị. Với VO nhiều thuộc tính dùng **owned types/complex types** (Tập 2, Chương 13).
- **Bài học thực tế**: truy vấn LINQ *lọc trên value object đã chuyển đổi* (`s.Ma.GiaTri == ...`) không luôn dịch được sang SQL. Giải pháp của solution: phía **đọc** dùng *read model* riêng (`SanPhamDoc`, cột đơn giản — Chương 5) thay vì ép EF lọc trên aggregate; phía **ghi** chỉ so sánh nguyên value object (`s.Ma == MaSanPham.TuCsd1(ma)`) — EF áp bộ chuyển đổi cho tham số.

Thiết kế tốt hơn nhờ câu hỏi

Khi mô hình hoá, hỏi:

1. **Bất biến nào** phải luôn đúng? (tồn ≥ 0 , mã đúng định dạng) → phương thức nghiệp vụ + kiểm tra ở domain.
2. Khái niệm này có **danh tính** không? (có → entity; không → value object)
3. Những thứ nào phải **nhất quán trong cùng một giao dịch**? → cùng aggregate.
4. Chuyện gì **đã xảy ra** mà phần khác của hệ thống quan tâm? → domain event.
5. Chuyên gia nghiệp vụ gọi nó là gì? → dùng đúng tên đó (tiếng Việt cũng được).

Lỗi thường gặp

- Domain "anemic": mọi property `public set`, logic ở service.
- Aggregate quá lớn (nhúng cả lịch sử giao dịch) → tải chậm, xung đột đồng thời liên tục.
- Value object có setter, hoặc dùng `string / decimal` trần thay vì tạo kiểu.
- Tham chiếu aggregate khác bằng object (kéo theo cả đồ thị) thay vì Id.
- Phát sự kiện **trước khi** thay đổi thành công, hoặc phát khi thao tác thất bại.
- Domain phụ thuộc EF/JSON (`[JsonIgnore]`, `[Key]`) vì "tiện".
- Ném exception cho lỗi nghiệp vụ dự đoán được rồi bắt ở khắp nơi.

- Đặt tên kỹ thuật (`SanPhamManager` , `Helper`) thay vì ngôn ngữ nghiệp vụ.

Bài tập

1. Thêm value object `SoLuong` (`int > 0`) và dùng trong `NhapKho/XuatKho` ; cập nhật test.
2. Thêm phương thức `DieuChinhTon(int tonMoi, string lyDo)` (kiểm kê) phát `TonKhoThayDoi` với lý do; viết test cả nhánh lỗi.
3. Thiết kế (giấy) aggregate `DonHang` gồm nhiều `DongDon` và giới hạn: tổng giá trị đơn ≤ 100 triệu. Nó tham chiếu sản phẩm thế nào?
4. Thêm sự kiện `GiaThayDoi` khi `DoiGia` thành công; đăng ký phát vào outbox (đọc Chương 8).
5. Tìm trong Tập 2 (Chương 22) ba chỗ đang là "anemic" và mô tả cách chuyển vào domain.

Chương 4 — Repository, Unit of Work và Specification

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **Repository** và **Unit of Work** giải quyết vấn đề gì, và vì sao `DbContext` *đã là* cả hai.
- Quyết định **khi nào** cần tự viết repository (và khi nào là "lớp bọc thừa").
- Tách **phía ghi (aggregate)** khỏi **phía đọc (read model)**.
- Dùng **Specification** để đóng gói điều kiện truy vấn tái sử dụng.

Code: [Repositories.cs](#), [Cong.cs](#), [KhoDbContext.cs](#).

Repository — "bộ sưu tập aggregate"

Repository cho code nghiệp vụ cảm giác **thao tác trên bộ sưu tập đối tượng trong bộ nhớ**, che giấu cách lưu trữ:

```
// Application/Abstractions – CÔNG, ngôn ngữ của nghiệp vụ
public interface ISanPhamRepository
{
    Task<SanPham?> LayTheoMaAsync(string ma, CancellationToken ct);
    Task<bool> TonTaiMaAsync(string ma, CancellationToken ct);
    void Them(SanPham sanPham);
}

// Infrastructure – cài đặt bằng EF Core
public class SanPhamRepository(KhoDbContext db) : ISanPhamRepository
{
    public Task<SanPham?> LayTheoMaAsync(string ma, CancellationToken ct)
        => db.SanPhams.FirstOrDefaultAsync(s => s.Ma == MaSanPham.TuCSDL(ma.Trim()).ToUpperInvariant(), ct);
    public void Them(SanPham sanPham) => db.SanPhams.Add(sanPham);
}
```

Nguyên tắc:

- **Một repository cho mỗi aggregate root** (không cho mỗi bảng): `ISanPhamRepository`, không có repository riêng cho "dòng đơn hàng".
- Phương thức mang **ngữ nghĩa nghiệp vụ** (`LayTheoMaAsync`, `TonTaiMaAsync`), không lộ `IQueryable` — để tầng ngoài không viết truy vấn tùy ý, và để Application không phụ thuộc EF.
- `Them` là **đồng bộ và chưa ghi CSDL**: chỉ đăng ký thay đổi; việc ghi thuộc Unit of Work.
- Chỉ thao tác trên **aggregate đầy đủ** (tải cả aggregate, lưu cả aggregate).

Unit of Work

Unit of Work theo dõi các thay đổi trong một "cuộc trao đổi nghiệp vụ" và **ghi chúng một lần trong một giao dịch**.

```
public interface IUnitOfWork { Task<int> LuuAsync(CancellationToken ct); }
```

Trong solution, UoW **do behavior gọi sau khi lệnh thành công** (`UnitOfWorkBehavior` , Chương 6) — handler không tự `Save` . Vì vậy handler đơn giản, và "một lệnh = một lần lưu = một giao dịch" được bảo đảm ở một chỗ.

DbContext đã là Repository + Unit of Work

Đây là điểm tranh luận lâu năm. `DbContext` **là** một Unit of Work (change tracker + `SaveChanges` trong giao dịch) và mỗi `DbSet<T>` **là** một repository. Bọc thêm `IRepository<T>` generic lên trên thường **không thêm giá trị**:

```
// X Generic repository: lớp bọc mỏng, che mất sức mạnh EF (Include, projection, ExecuteUpdate...), 1
public interface IRepository<T> { Task<T?> GetByIdAsync(int id); Task<IEnumerable<T>> GetAllAsync(); }
```

Vấn đề của repository generic: (1) không có chỗ cho truy vấn thực tế (lọc, phân trang, projection) → hoặc lộ `IQueryable` (rò rỉ) hoặc thêm hàng chục phương thức; (2) ép mọi entity có cùng bộ thao tác dù aggregate khác nhau; (3) `GetAll` kéo cả bảng; (4) khó nhận biết cái gì xảy ra ở CSDL.

Vậy khi nào tự viết repository?

Tình huống	Khuyến nghị
CRUD đơn giản, ít nghiệp vụ (Tập 2)	dùng thẳng <code>DbContext</code> — đừng bọc
Clean Architecture / DDD với Application không được biết EF	repository theo aggregate như solution: cổng ở Application, cài đặt ở Infrastructure
Cần thay nguồn dữ liệu (CSDL ↔ API ↔ cache) hoặc test nghiệp vụ không cần CSDL	repository/cổng
Chỉ để "sau này có thể đổi ORM"	thường không đáng (YAGNI); đổi ORM hiếm khi xảy ra

Quy tắc thực dụng: repository đáng giá khi nó **diễn đạt câu hỏi nghiệp vụ** và **bảo vệ ranh giới tầng**, không phải khi nó chỉ đổi tên `db.SanPhams` .

Phía đọc khác phía ghi: Read Model

Phía **ghi** cần *aggregate đầy đủ* để bảo vệ bất biến. Phía **đọc** (danh sách, tìm kiếm, báo cáo) chỉ cần *dữ liệu phẳng để hiển thị* — nạp aggregate cho việc này là lãng phí và dính vào value object/quan hệ. Solution dùng cổng riêng:

```
public interface IKhoDocDuLieu
{
    Task<TrangKetQua<SanPhamDto>> TimAsync(string? tuKhoa, string? nhom, int trang, int kíchThuoc, CancellationTok
    Task<SanPhamDto?> LayTheoMaAsync(string ma, CancellationToken ct);
}
```

Cài đặt đọc trực tiếp từ một **thực thể không khoá** ánh xạ vào cùng bảng bằng truy vấn SQL:

```
mb.Entity<SanPhamDoc>(e =>
{
    e.HasNoKey();
    e.ToSqlQuery("SELECT Id, Ma, Ten, Nhom, DonGia, TonKho, MucCanhBao FROM san_pham");
    e.Property(x => x.DonGia).HasConversion<double>();
}
```

```
});

// Lọc/sắp xếp trên read model (cột đơn giản), chiếu sang DTO ở CUỐI
IQueryable<SanPhamDoc> q = db.SanPhamDocs.AsNoTracking();
if (!string.IsNullOrEmpty(tuKhoa)) q = q.Where(s => EF.Functions.Like(s.Ten, $"%{tuKhoa}%")) || E
int tong = await q.CountAsync(ct);
var dong = await q.OrderBy(s => s.Id).Skip((trang - 1) * kichThuoc).Take(kichThuoc).ToListAsync(ct);
```

Ưu điểm: truy vấn đọc **nhANH** (không tracking, không hydrate aggregate), **không bị ràng buộc** bởi value object, và có thể tối ưu độc lập (chỉ mục riêng, view, thậm chí CSDL/cache riêng — đó là CQRS ở Chương 5). Kinh nghiệm thật khi viết solution: lọc trên DTO đã `select` từ thực thể có value object gây lỗi dịch LINQ; chuyển sang read model phẳng và chiếu DTO **sau cùng** giải quyết gọn.

Specification — đóng gói điều kiện truy vấn

Khi cùng một điều kiện ("sản phẩm sắp hết", "khách VIP hoạt động") xuất hiện ở nhiều nơi, **Specification** gói nó thành đối tượng có tên, tái sử dụng và kết hợp được:

```
public abstract class Specification<T>
{
    public abstract Expression<Func<T, bool>> DieuKien { get; }

    public Specification<T> Va(Specification<T> khac) => new VaSpec<T>(this, khac);
}

public sealed class SapHetHangSpec : Specification<SanPhamDoc>
{
    public override Expression<Func<SanPhamDoc, bool>> DieuKien => s => s.TonKho <= s.MucCanhBao;
}

public sealed class TrongNhomSpec(string nhom) : Specification<SanPhamDoc>
{
    public override Expression<Func<SanPhamDoc, bool>> DieuKien => s => s.Nhom == nhom;
}

// Dùng: q.Where(new SapHetHangSpec().Va(new TrongNhomSpec("Phu kien")).DieuKien)
```

(`VaSpec<T>` — nối hai cây biểu thức thành một — được để lại làm **bài tập 1**; đoạn trên là phác thảo ý tưởng, không nằm trong solution.)

Điểm quan trọng: điều kiện là `Expression<Func<T, bool>>` (cây biểu thức), không phải `Func<T, bool>` — EF dịch được sang SQL. Ghép `Va/Hoac` cần nối cây biểu thức (thư viện **Ardalis.Specification** làm sẵn, kèm `Include`, sắp xếp, phân trang). Đặt tên rõ ("SapHetHang") biến quy tắc kinh doanh thành khái niệm hạng nhất, thay vì rải `TonKho <= MucCanhBao` khắp nơi (và lệch nhau khi ai đó sửa một chỗ). Chỉ dùng khi thật sự có điều kiện lặp/phức tạp; với truy vấn đơn giản, LINQ trực tiếp trong read model là đủ.

Giao dịch ngoài "một lệnh, một lần lưu"

- **Nhiều SaveChanges cần chung giao dịch**: `await using var tx = await db.Database.BeginTransactionAsync();` (đưa vào cổng `IUnitOfWork.BatDauGiaoDichAsync` nếu Application cần).

- **Xung đột đồng thời:** `IsConcurrencyToken` trên `TonKho` → `DbUpdateConcurrencyException` → Infrastructure dịch thành `ConcurrencyException` (Application hiểu) → behavior trả `Loi.XungDot` → HTTP 409. Application **không bao giờ thấy** kiểu exception của EF.
- **Một lệnh sửa hai aggregate cùng lúc:** dấu hiệu ranh giới aggregate sai, hoặc cần **sự kiện** và nhất quán cuối cùng (Chương 8).

Lỗi thường gặp

- Repository generic `IRepository<T>` + `GetAll()` rồi lọc trong bộ nhớ.
- Trả `IQueryable` từ repository → tầng trên viết truy vấn, phụ thuộc EF ngầm.
- Repository gọi `SaveChanges` trong từng phương thức → mất khả năng gom giao dịch.
- Nhiều repository cho các bảng con của cùng một aggregate.
- Dùng aggregate cho truy vấn danh sách/báo cáo (chậm, kéo theo cả đồ thị).
- Specification dùng `Func` thay vì `Expression` → EF tải cả bảng rồi lọc trong bộ nhớ (chỉ thấy khi dữ liệu lớn).
- Để exception EF (`DbUpdateException`) lọt tới tầng Web.

Bài tập

1. Viết `Specification<SanPhamDoc>` thật (kể cả `Va`) dùng `Expression.Invoke` / thay tham số; kiểm tra bằng `ToQueryString()` rằng vẫn ra **một** mệnh đề `WHERE`.
2. Thêm truy vấn `SapHetAsync()` vào `IKhoDocDuLieu` dùng `SapHetHangSpec`.
3. Đo số câu SQL của `TimAsync` (dùng `LogTo`); so với việc nạp aggregate rồi map sang DTO.
4. Thêm `IUnitOfWork.BatDauGiaoDichAsync` và dùng trong một lệnh "chuyển kho" sửa hai aggregate; bàn về rủi ro.
5. Viết test: khi hai người cùng xuất và một bị `ConcurrencyException`, handler/behavior trả 409 chứ không phải 500 (đã có ở `ApplicationTests` — đọc và giải thích).

Chương 5 — CQRS và Mediator

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **CQRS** (Command Query Responsibility Segregation): tách **lệnh** (ghi) và **truy vấn** (đọc).
- Viết `ICommand / IQuery`, **handler**, và một **mediator** tự viết (~40 dòng) để kết nối chúng.
- Biết khi nào CQRS đáng dùng (và các mức độ: cùng CSDL → đọc/ghi tách biệt hoàn toàn).
- Hiểu ưu/nhược điểm mediator và các lựa chọn (MediatR, Wolverine, tự viết).

Code: [Messaging/Mediator.cs](#), [SanPhams/CaSuDung.cs](#).

Vấn đề: một SanPhamService phình to

```
class KhoService // Tập 2 - ban đầu vài phương thức, sau 1 năm...
{
    ThemAsync, NhapAsync, XuatAsync, DoiGiaAsync, XoaAsync, TimAsync, LayAsync,
    BaoCaoTheoNhomAsync, SapHetAsync, LichSuAsync, XuatExcelAsync, ... // 40 phương thức, 15 phụ th
}
```

Lớp làm quá nhiều, phụ thuộc nhiều thứ không liên quan (báo cáo cần `IExcelWriter`, lệnh xuất cần `IEmailSender`), khó test (mock 15 thứ), mọi thay đổi đụng cùng một file (xung đột merge).

CQRS

CQRS (Bertrand Meyer — *Command–Query Separation*; Greg Young — CQRS): phân biệt hai loại thao tác:

	Lệnh (Command)	Truy vấn (Query)
Mục đích	thay đổi trạng thái	đọc dữ liệu
Tác dụng phụ	có	không
Trả về	kết quả thành công/lỗi (+ Id mới)	dữ liệu (DTO)
Đặt tên	mệnh lệnh: <code>XuatKhoCommand</code>	câu hỏi: <code>TimSanPhamQuery</code>
Mô hình	aggregate (bảo vệ bất biến)	read model phẳng, tối ưu hiển thị
Giao dịch	có (Unit of Work)	không cần

Ba mức áp dụng:

1. **Tách trong code** (solution này): cùng một CSDL, nhưng lệnh đi qua aggregate, truy vấn đi qua read model riêng. Chi phí thấp, lợi ích lớn — điểm bắt đầu hợp lý.
2. **Tách lược đồ đọc**: bảng/view riêng cho đọc (denormalized), cập nhật bằng sự kiện.
3. **Tách hạ tầng**: CSDL ghi (quan hệ) + CSDL đọc (Elasticsearch, Redis, kho cột), đồng bộ bất đồng bộ — nhất quán cuối cùng. Chỉ khi quy mô đọc/ghi chênh lệch lớn.

CQRS **không** đồng nghĩa với Event Sourcing hay microservices, và không phải lúc nào cũng cần.

Mỗi ca sử dụng một "thông điệp" + một handler

```
// Lệnh: dữ liệu vào + kiểu kết quả
public sealed record XuatKhoCommand(string Ma, int SoLuong) : ICommand<Result>;

// Handler: MỘT lớp cho MỘT ca sử dụng
public sealed class XuatKhoHandler(ISanPhamRepository repo, TimeProvider dongHo) : IRequestHandler<XuatKhoCommand, Result>
{
    public async Task<Result> Handle(XuatKhoCommand cmd, CancellationToken ct)
    {
        var sp = await repo.LayTheoMaAsync(cmd.Ma, ct);
        return sp is null
            ? Result.Fail(Loi.KhongTimThay("SanPham.KhongTimThay", $"Khong tim thay san pham {cmd.Ma}"))
            : sp.XuatKho(cmd.SoLuong, dongHo.GetUtcNow());
    }
}

// Truy vấn: cũng một thông điệp + một handler, nhưng đi qua cổng ĐỌC
public sealed record TimSanPhamQuery(string? TuKhoa, string? Nhóm, int Trang = 1, int KichThuoc = 10)
```

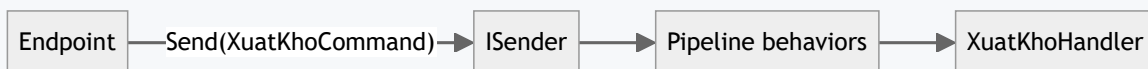
Mỗi handler chỉ có **những phụ thuộc nó thật sự cần** (xuất kho cần repository và đồng hồ; không cần Excel). Nhỏ, tập trung, dễ test (dùng handler với fake — hoặc gọi qua mediator như `ApplicationTests`), dễ tìm ("ca sử dụng nào? → file đó"), và **thêm tính năng = thêm file** (không sửa `KhoService`).

Mediator

Ai gọi handler? Endpoint có thể inject trực tiếp `XuatKhoHandler`, nhưng khi đó:

- Endpoint phụ thuộc lớp cụ thể của Application;
- **Không có chỗ chung** để chèn hành vi xuyên suốt (log, validate, giao dịch, cache) — Chương 6.

Mediator đứng giữa: endpoint chỉ biết *thông điệp* và một cổng `ISender`; mediator tìm đúng handler và chạy qua **pipeline**.



Hợp đồng

```
public interface IRequest<TResponse> where TResponse : IKetQua<TResponse>;
public interface ICommand<TResponse> : IRequest<TResponse> where TResponse : IKetQua<TResponse>;
public interface IQuery<TResponse> : IRequest<TResponse> where TResponse : IKetQua<TResponse>;

public interface IRequestHandler<TRequest, TResponse> where TRequest : IRequest<TResponse> where TResponse : IKetQua<TResponse>
{ Task<TResponse> Handle(TRequest request, CancellationToken ct); }

public interface ISender
{ Task<TResponse> Send<TResponse>(IRequest<TResponse> request, CancellationToken ct = default) where TResponse : IKetQua<TResponse>; }
```

`ICommand` và `IQuery` cùng kế thừa `IRequest`, khác nhau ở **ý nghĩa** — và cho phép behavior áp dụng **riêng cho lệnh** (giao dịch) hay **riêng cho truy vấn** (cache).

Cài đặt tối giản

```
internal sealed class Sender(IServiceProvider sp) : ISender
{
    private static readonly ConcurrentDictionary<Type, object> Wrappers = new();

    public Task<TResponse> Send<TResponse>(IRequest<TResponse> request, CancellationToken ct = default)
    {
        var wrapper = (Boc<TResponse>)Wrappers.GetOrAdd(request.GetType(),
            t => Activator.CreateInstance(typeof(Boc<, >).MakeGenericType(t, typeof(TResponse))))!);
        return wrapper.Chay(request, sp, ct);
    }
    ...
}
```

Vấn đề kỹ thuật: `Send` chỉ biết kiểu **đối tượng lúc chạy** (`request.GetType()`), nhưng DI cần kiểu **generic đã đóng** `IRequestHandler<XuấtKhoCommand, Result>`. Giải pháp: một lớp bọc generic `Boc<TRequest, TResponse>` được tạo **một lần** cho mỗi kiểu request bằng reflection và **cache** — về sau mỗi lần gọi là lời gọi có kiểu tĩnh, không reflection. Toàn bộ mediator ~40 dòng.

Đăng ký

```
public static IServiceCollection AddMediator(this IServiceCollection services, Assembly assembly, params Assembly[] otherAssemblies)
{
    services.AddScoped<ISender, Sender>();
    foreach (var type in assembly.GetTypes().Where(t => t is { IsClass: true, IsAbstract: false }))
        foreach (var i in type.GetInterfaces().Where(i => i.IsGenericType && i.GetGenericTypeDefinition() == typeof(IRequestHandler<,>)))
            services.AddScoped(i, type); // quét assembly: mỗi lần
    foreach (var b in behaviorMoRong) services.AddScoped(typeof(IPipelineBehavior<, >), b);
    return services;
}
```

Thêm ca sử dụng mới = thêm command + handler; **không sửa** file đăng ký hay endpoint khác.

Sử dụng ở endpoint

```
g.MapPost("/{ma}/xuat", async (string ma, SoLuongRequest r, ISender sender, CancellationToken ct)
=> (await sender.Send(new XuấtKhoCommand(ma, r.SoLuong), ct)).ToHttp(noContent: true));
```

static abstract — chi tiết C# hiện đại làm nên sự gọn

Behavior generic cần **tạo kết quả lỗi** cho *bất kỳ* `TResponse` (`Result` hay `Result<T>`):

```
public interface IKetQua<TSelf> where TSelf : IKetQua<TSelf>
{
    static abstract TSelf TuLoi(Loi loi);
    bool ThanhCong { get; }
```

```
    Loi? Loi { get; }
}

// Trong ValidationBehavior<TRequest, TResponse> where TResponse : IKetQua<TResponse>:
return TResponse.TuLoi(Loi.DuLieuKhongHopLe("Validation", moTa));    // gọi thành viên static trên tL
```

Static abstract interface members (C# 11) cho phép gọi `TResponse.TuLoi(...)` không cần reflection hay `dynamic`. (Nếu không có tính năng này, các thư viện phải dùng reflection hoặc ép kiểu.)

Ưu và nhược điểm của mediator

Ưu:

- Endpoint mỏng, tách rời khỏi handler; thêm/xoá tính năng cục bộ.
- Một chỗ chung để gắn hành vi xuyên suốt (pipeline).
- Handler nhỏ, test độc lập.

Nhược:

- **Gián tiếp**: từ endpoint khó "nhảy tới định nghĩa" handler (IDE hỗ trợ tìm theo kiểu); luồng bị "ẩn" sau DI.
- Thêm một lớp trừu tượng và chi phí nhỏ mỗi lần gọi.
- Dễ lạm dụng: handler gọi handler khác qua mediator tạo dây phụ thuộc khó thấy — dùng dịch vụ miền/domain thay vì handler-gọi-handler.
- Không cần thiết nếu ứng dụng nhỏ (endpoint gọi thẳng dịch vụ là đủ).

Thư viện hay tự viết?

- **MediatR** (Jimmy Bogard) là thư viện phổ biến nhất, API tương tự (`IRequest<T>` , `IRequestHandler` , `IPipelineBehavior`). Lưu ý: từ các bản gần đây MediatR chuyển sang **giấy phép thương mại** cho một số đối tượng dùng — hãy kiểm tra điều khoản hiện hành trước khi dùng trong dự án.
- **Wolverine, Mediator (martinohamar, source generator, nhanh), Immediate.Handlers**: lựa chọn thay thế.
- **Tự viết** (như chương này): ~40 dòng, đủ cho nhu cầu phổ biến, không phụ thuộc bên ngoài, hiểu rõ bên trong. Đủ tốt để đi sản phẩm với đội nhỏ; chuyển sang thư viện nếu cần tính năng nâng cao (notification, streaming, generic behavior phức tạp).

API giữa các lựa chọn giống nhau nên đổi sau này dễ.

Khi nào dùng CQRS + mediator?

Dấu hiệu	Gợi ý
CRUD đơn giản, ít quy tắc	Không cần (Tập 2)
Service phình to, mỗi tính năng đung cùng file	Tách use case + handler
Cần hành vi xuyên suốt (log, validate, giao dịch, cache) áp đồng loạt	Mediator + pipeline
Đọc/ghi có yêu cầu hiệu năng, mô hình khác hẳn nhau	CQRS mức 2–3
Nhiều đội cùng sửa một codebase	Vertical slice + CQRS giảm xung đột

Lỗi thường gặp

- Truy vấn có tác dụng phụ (ghi log nghiệp vụ, đổi trạng thái) — phá tính chất "không đổi trạng thái".
- Lệnh trả về cả đối tượng lớn để "tiện" — nên trả `Id/ Result`, muốn dữ liệu thì gửi truy vấn.
- Handler gọi `ISender.Send` của handler khác (spaghetti).
- Quên đăng ký handler/behavior (không quét đúng assembly) → `InvalidOperationException: No service for type IRequestHandler<...> .`
- Bọc mọi thứ trong mediator kể cả nơi đơn giản — "nghi thức" thừa.
- Đặt logic nghiệp vụ trong handler thay vì domain (handler thành "transaction script" phình to).

Bài tập

1. Thêm `LayLichSuQuery` (đọc từ bảng outbox) qua mediator; đọc bằng cổng đọc, không nạp aggregate.
2. Viết test kiểm chứng `ISender` ném lỗi rõ ràng khi thiếu handler; sửa thông báo lỗi cho dễ hiểu.
3. Thêm `INotification / IPublisher` (một sự kiện, nhiều handler) vào mediator tự viết (~20 dòng).
4. So sánh số dòng/độ phức tạp: viết lại `XuatKho` dưới dạng phương thức của `KhoService` (Tập 2) và dạng `command+handler`. Khi nào cách nào tốt hơn?
5. Đo chi phí mỗi lần `Send` với `BenchmarkDotNet` so với gọi handler trực tiếp.

Chương 6 — Pipeline behavior: validation, logging, transaction

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng **pipeline behavior** (mẫu Decorator/Chain of Responsibility) để xử lý các mối quan tâm **xuyên suốt** ở một nơi.
- Đọc hiểu ba behavior của solution: **Logging, Validation, UnitOfWork** và **thứ tự** của chúng.
- Thêm behavior mới (cache, authorization, retry, timing) và giới hạn behavior theo **kiểu request** bằng generic constraint.
- Chọn giữa behavior, middleware và filter.

Code: [Behaviors/Behaviors.cs](#) , [ApplicationTests.cs](#) .

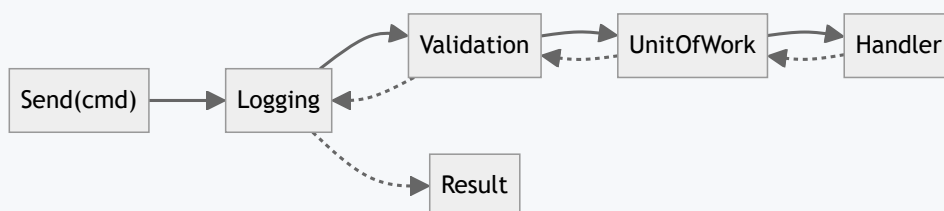
Cross-cutting concerns

Có việc **mọi** ca sử dụng đều cần: ghi log, kiểm tra dữ liệu đầu vào, mở/đóng giao dịch, kiểm tra quyền, đo thời gian, cache. Viết lặp trong từng handler:

```
public async Task<Result> Handle(XuatKhoCommand cmd, CancellationToken ct)
{
    log.LogInformation("Bat dau xuat kho ...");           // lặp ở 30 handler
    var loi = validator.Validate(cmd); if (!loi.IsValid) return ...; // lặp
    var sw = Stopwatch.StartNew();                        // lặp
    ...nghiệp vụ thật (3 dòng)...
    await uow.SaveChangesAsync(ct);                     // lặp, dễ quên
    log.LogInformation("Xong sau {Ms}", sw.ElapsedMilliseconds); // lặp
}
```

Hậu quả: nghiệp vụ chìm giữa "rác hạ tầng", dễ **quên** ở một handler (không validate → dữ liệu xấu; không lưu → mất dữ liệu). Pipeline behavior đưa chúng ra một chỗ, **áp dụng đồng loạt và không thể quên**.

Pipeline: vỏ hành xung quanh handler



Mỗi behavior nhận `next` (đoạn còn lại của chuỗi) — như middleware [ASP.NET Core](#) (Tập 2, Chương 3): làm việc **trước**, gọi `next()` , làm việc **sau**, hoặc **ngắn mạch** không gọi `next` .

```
public interface IPipelineBehavior<TRequest, TResponse> where TRequest : IRequest<TResponse> where TR
{
```

```
Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next, CancellationToken
}
```

1. Logging

```
public sealed class LoggingBehavior<TRequest, TResponse>(ILogger<LoggingBehavior<TRequest, TResponse>>
    where TRequest : IRequest<TResponse> where TResponse : IKetQua<TResponse>
{
    public async Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next, CancellationTok
    {
        var dong = Stopwatch.StartNew();
        var kq = await next();
        if (kq.ThanhCong) log.LogInformation("{Request} thanh cong sau {Ms} ms", typeof(TRequest).Name, kq.ThanhCong, dong.ElapsedMilliseconds);
        else log.LogWarning("{Request} that bai ({Ma}) sau {Ms} ms", typeof(TRequest).Name, kq.Loi!.Ma, dong.ElapsedMilliseconds);
        return kq;
    }
}
```

Vì kết quả là `Result` (có `ThanhCong`, `Loi.Ma`), behavior log được **thành công/thất bại và mã lỗi** mà không cần `try/catch` hay biết kiểu cụ thể. **Không log nội dung request** mặc định (có thể chứa dữ liệu nhạy cảm) — chỉ tên request và kết quả.

Đặt **ngoài cùng** để đo tổng thời gian của mọi behavior bên trong.

2. Validation

```
public sealed class ValidationBehavior<TRequest, TResponse>(IEnumerable<IValidator<TRequest>> validators
    where TRequest : IRequest<TResponse> where TResponse : IKetQua<TResponse>
{
    public async Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next, CancellationToken
    {
        if (!validators.Any()) return await next();

        var ctx = new ValidationContext<TRequest>(request);
        var loi = (await Task.WhenAll(validators.Select(v => v.ValidateAsync(ctx, ct)))).SelectMany(r => r.Loi);
        if (loi.Count == 0) return await next();

        string moTa = string.Join("; ", loi.Select(e => $"{{e.PropertyName}}: {{e.ErrorMessage}}"));
        return TResponse.TuLoi(Loi.DuLieuKhongHopLe("Validation", moTa)); // ngắn mạch: handle
    }
}
```

- Nhận `IEnumerable<IValidator<TRequest>>`: request nào **không có** validator thì danh sách rỗng, đi thẳng — không cần cấu hình gì.
- Validator viết bằng **FluentValidation** cạnh command (Chương 7):

```
public sealed class XuatKhoValidator : AbstractValidator<XuatKhoCommand>
{
    public XuatKhoValidator() => RuleFor(x => x.SoLuong).InclusiveBetween(1, 100_000);
}
```

- Lỗi trả về như **giá trị** (`Result`), không ném exception — nhanh hơn và không phải bắt ở khắp nơi. Test `ThemSanPham_SaiDinhDang_ValidationBehaviorChanTruocHandler` chứng minh handler và Unit of Work không được gọi.

Phân biệt: **validation đầu vào** (định dạng, khoảng — behavior này) khác **luật nghiệp vụ** (đủ hàng — trong Domain). Domain vẫn tự bảo vệ bất biến; validator chặn sớm dữ liệu rác với thông báo tốt.

3. Unit of Work

```
public sealed class UnitOfWorkBehavior<TRequest, TResponse>(IUnitOfWork uow) : IPipelineBehavior<TRequest, TResponse>
    where TRequest : ICommand<TResponse> where TResponse : IKetQua<TResponse> // CHỈ áp cho LỆNH
{
    public async Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next, CancellationToken ct)
    {
        var kq = await next();
        if (!kq.ThanhCong) return kq; // handler thất bại: không lưu gì

        try { await uow.LuuAsync(ct); }
        catch (ConcurrencyException) { return TResponse.TuLoi(Loi.XungDot("Kho.XungDotDongThoi", "Dữ liệu đang bị xử lý")); }
        catch (TrungLapException e) { return TResponse.TuLoi(Loi.XungDot("Kho.TrungLap", e.Message)); }
        return kq;
    }
}
```

Điểm hay:

- **Constraint** `where TRequest : ICommand<TResponse>` giới hạn behavior này **chỉ cho lệnh**. Đăng ký mở `typeof(IPipelineBehavior<,>) → UnitOfWorkBehavior<,>`; với query (không thỏa constraint) DI **tự bỏ qua** (test `TruyVan_KhongDiQuaUnitOfWorkBehavior`: truy vấn không bao giờ lưu).
- **Handler không bao giờ gọi Save** — không thể quên; luôn "một lệnh = một lần lưu = một giao dịch".
- **Dịch lỗi hạ tầng sang ngôn ngữ nghiệp vụ**: `ConcurrencyException` / `TrungLapException` do Infrastructure ném (từ `DbUpdateConcurrencyException` / `UNIQUE`) → `Loi.XungDot` → HTTP 409. Test `XungDotDongThoi_...` mô phỏng bằng UoW giả ném exception.
- Lưu ý: nếu `LuuAsync` thành công thì aggregate và **outbox** (sự kiện miền) đã được ghi cùng giao dịch (Chương 8).

Thứ tự quan trọng

```
services.AddMediator(assembly, typeof(LoggingBehavior<,>), typeof(ValidationBehavior<,>), typeof(UnitOfWorkBehavior<,>))
// ngoài cùng → trong
```

Thứ tự đăng ký = thứ tự **từ ngoài vào trong**. Lý do chọn:

1. **Logging** ngoài cùng: thấy cả những request bị validation chặn.
2. **Validation** trước UoW: dữ liệu sai thì khỏi mở giao dịch.
3. **UnitOfWork** trong cùng, sát handler: chỉ lưu khi handler đã xong.

Nếu thêm **Authorization** (kiểm tra quyền): đặt *trước* validation (từ chối sớm mà không tốn công); **Caching** (chỉ query): đặt trước handler, sau authorization; **Retry** (lỗi tạm thời): bọc quanh UoW.

Test `Pipeline_ChayTheoThuTuNgoaiVaoTrongRoiTrongRaNgoai` chứng minh bằng hai behavior ghi vết: `["A:vao","B:vao","B:ra","A:ra"]`.

Thêm behavior: ví dụ cache cho truy vấn

```
public interface ICachedQuery { string CacheKey { get; } TimeSpan ThoiGianSong { get; } }

public sealed class CachingBehavior<TRequest, TResponse>(IMemoryCache cache) : IPipelineBehavior<TRequest, TResponse>
    where TRequest : IQuery<TResponse>, ICachedQuery where TResponse : IKetQua<TResponse>
{
    public async Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next, CancellationToken cancellationToken)
    {
        if (cache.TryGetValue(request.CacheKey, out TResponse? trongCache) && trongCache is not null)
        {
            var kq = await next();
            if (kq.ThanhCong) cache.Set(request.CacheKey, kq, request.ThoiGianSong); // chỉ cache kết quả thành công
            return kq;
        }
        return await next();
    }
}
```

Truy vấn nào muốn cache chỉ cần `implements ICachedQuery` — không sửa handler (Chương 10 nói kỹ về chiến lược cache và vô hiệu hoá).

Behavior, middleware hay filter?

	Middleware (ASP.NET)	Endpoint filter	Pipeline behavior
Tầng	HTTP, mọi request	endpoint HTTP	Application (ca sử dụng)
Biết HTTP?	có	có	không
Dùng cho	lỗi toàn cục, xác thực, CORS, nén	validation HTTP, ánh xạ tham số	log nghiệp vụ, validation lệnh, giao dịch, cache
Tái sử dụng khi đổi giao diện (gRPC, worker)	không	không	có

Quy tắc: mỗi quan tâm **thuộc HTTP** → middleware/filter; **thuộc nghiệp vụ/ca sử dụng** → behavior (chạy được cả từ worker, message consumer, test).

Lỗi thường gặp

- Đặt thứ tự sai (UoW ngoài validation → mở giao dịch cho dữ liệu rác; logging trong cùng → không thấy lỗi validation).
- Handler **vẫn** tự gọi `Save` (lưu hai lần hoặc lưu ngay cả khi behavior sau đó thất bại).
- Behavior cho query có tác dụng phụ (ghi DB) hoặc behavior giao dịch chạy cả với query (mở giao dịch vô ích).
- Ném exception thay vì trả `Result` trong behavior → phá luồng xử lý lỗi thống nhất.
- Cache theo khóa thiếu tham số (kết quả người này lộ cho người khác) hoặc cache cả kết quả lỗi.
- Log nội dung request chứa mật khẩu/thẻ.
- Behavior quá nhiều, mỗi request đi qua 10 lớp — cân nhắc lại.

Bài tập

1. Viết `TimingBehavior` cảnh báo (`LogWarning`) khi ca sử dụng chậm hơn 500 ms; đặt ở vị trí hợp lý.
2. Viết `AuthorizationBehavior` với attribute `[YeuCauVaiTro("Admin")]` trên command và cổng `IUserContext` (Application) — Web cung cấp cài đặt từ `HttpContext` .
3. Viết `RetryBehavior` cho lỗi tạm thời (`TimeoutException`) tối đa 3 lần với backoff; chỉ áp dụng cho **query** hoặc lệnh idempotent.
4. Thêm `CachingBehavior` ở trên và test: lần gọi thứ hai không chạm handler.
5. Đổi thứ tự Validation và `UnitOfWork`; chạy bộ test và quan sát test nào đỏ. Giải thích.

Chương 7 — Result pattern và validation

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng **Result pattern** để biểu diễn lỗi nghiệp vụ như **giá trị**, thay vì exception.
- Thiết kế **Loi** (mã ổn định + loại + mô tả) và dịch sang **HTTP** ở một chỗ.
- Viết validation bằng **FluentValidation**, phân biệt **validation đầu vào** với **luật nghiệp vụ**.
- Biết khi nào **nên** và **không nên** ném exception.

Code: [Common/Result.cs](#) , [CaSuDung.cs](#) , [KhoEndpoints.cs](#) .

Exception hay Result?

Tập 1 (Chương 21) dạy exception. Tập 2 (Chương 9) dùng exception nghiệp vụ + `ExceptionHandler` . Cách đó ổn, nhưng có vấn đề khi lỗi là **kết quả bình thường, xảy ra thường xuyên**:

```
// Exception cho "không đủ hàng" (xảy ra hàng ngày, không phải bất thường)
try { sp.XuatKho(n); }
catch (KhongDuHangException e) { ... }
```

- **Chi phí**: ném/bắt exception chậm hơn nhiều so với trả giá trị (đáng kể khi lập/tải cao).
- **Luồng ẩn**: chữ ký `void XuatKho(int)` **không nói** nó có thể thất bại theo cách nào; người gọi dễ quên `try/catch` .
- **Goto vô hình**: exception nhảy qua nhiều tầng; khó đọc luồng.

Result pattern: hàm trả về **giá trị mô tả thành công hoặc lỗi**; chữ ký tự nói lên khả năng thất bại và trình biên dịch/IDE nhắc xử lý.

Dùng Result cho	Dùng exception cho
lỗi nghiệp vụ dự đoán được (hết hàng, dữ liệu sai, không tìm thấy, xung đột)	lỗi lập trình (null, tham số sai, trạng thái không thể)
luồng bình thường mà "thất bại" là một kết quả	lỗi hạ tầng bất ngờ (mất mạng, đĩa đầy, bug)
cần cho người gọi xử lý từng trường hợp	lỗi mà tầng gần không thể xử lý → nổi lên tới handler toàn cục (500)

Hai cách **bổ sung** nhau: lỗi *nghiệp vụ* qua `Result` , lỗi *bất thường* qua exception + `ExceptionHandler` .

Thiết kế Result và Loi

```
public enum LoiLoi { DuLieuKhongHopLe, KhongTimThay, XungDot, NghiepVu }

public sealed record Loi(string Ma, string MoTa, LoiLoi Loi)
{
    public static Loi DuLieuKhongHopLe(string ma, string moTa) => new(ma, moTa, LoiLoi.DuLieuKhongHopLe);
    public static Loi KhongTimThay(string ma, string moTa) => new(ma, moTa, LoiLoi.KhongTimThay);
}
```

```

    public static Loi XungDot(string ma, string moTa) => new(ma, moTa, LoaiLoi.XungDot);
    public static Loi NghiepVu(string ma, string moTa) => new(ma, moTa, LoaiLoi.NghiepVu);
}

public class Result : IKetQua<Result>
{
    public bool ThanhCong { get; }
    public Loi? Loi { get; }
    public static Result Ok() => new(true, null);
    public static Result Fail(Loi loi) => new(false, loi);
}

public sealed class Result<T> : Result, IKetQua<Result<T>>
{
    public T GiaTri => ThanhCong ? _giaTri! : throw new InvalidOperationException("Ket qua that bai ki
    public static Result<T> Ok(T giaTri) => new(giaTri, true, null);
    public static implicit operator Result<T>(T giaTri) => Ok(giaTri); // return giaTri; tự thàn
    public Result<TOut> Map<TOut>(Func<T, TOut> f) => ThanhCong ? Result<TOut>.Ok(f(GiaTri)) : Result
    public Result<TOut> Bind<TOut>(Func<T, Result<TOut>> f) => ThanhCong ? f(GiaTri) : Result<TOut>.Fa
}

```

Các quyết định:

- **Loi.Ma ổn định và có cấu trúc** ("SanPham.KhongDuHang"): client/log/i18n dựa vào **mã** (không đổi), còn MoTa là văn bản cho người đọc (có thể dịch/đổi).
- **Loi.Loai** là *phân loại nghiệp vụ*, không phải mã HTTP — domain không biết HTTP. Tầng Web ánh xạ một lần (LoisangHttp , Chương 2).
- **Truy cập GiaTri khi thất bại ném InvalidOperationException** : đọc giá trị mà không kiểm tra là **bug lập trình** (đúng loại để dùng exception).
- **Không có Result "rỗng" mơ hồ**: hoặc thành công (có giá trị) hoặc thất bại (có lỗi).
- Có thể mở rộng thành **danh sách lỗi** (validation nhiều trường) — solution gộp thành một chuỗi cho đơn giản; thư viện như **ErrorOr**, **FluentResults**, **OneOf** làm phong phú hơn.

Dùng Result

Trong domain

```

public static Result<SanPham> Tao(string? ma, string? ten, ...)
{
    var maKq = MaSanPham.Tao(ma);
    if (!maKq.ThanhCong) return Result<SanPham>.Fail(maKq.Loι!); // dừng ngay, chuyển tiếp lỗi
    ...
    return sp; // conversion ngầm: T → Resu
}

```

Trong handler: "đường ray" (railway-oriented)

```

var kq = MaSanPham.Tao("LT001").Bind(m => Tien.Tao(-5).Map(_ => m)); // Bind/Map chỉ chạy tiếp khi

```

Map biến đổi giá trị, Bind nối bước có thể lỗi khác; lỗi đầu tiên "trượt" thẳng ra cuối, các bước sau bị bỏ qua. Với chuỗi dài đọc gọn hơn if (!x.ThanhCong) return ... lồng nhau — nhưng với 2–3 bước, if tường minh

cũng rõ ràng, đừng ép dùng `Bind`.

Ở ranh giới HTTP

```
public static IActionResult ToHttp<T>(this Result<T> kq) => kq.ThanhCong ? Results.Ok(kq.GiaTri) : LoaiSangH  
  
private static IActionResult LoaiSangHttp(Loi loi) => Results.Problem(title: loi.Ma, detail: loi.MoTa, statu  
{  
    LoaiLoi.DuLieuKhongHopLe => 400, LoaiLoi.KhongTimThay => 404, LoaiLoi.XungDot => 409, LoaiLoi.Ngh  
});
```

Kết quả `ProblemDetails` (RFC 9457) với `title = mã lỗi` giúp client xử lý theo mã. Muốn phong phú hơn: thêm `extensions` (danh sách lỗi theo trường).

Validation với FluentValidation

Validator viết bằng code (tách khỏi DTO, dễ test, biểu đạt được điều kiện phức tạp):

```
public sealed class ThemSanPhamValidator : AbstractValidator<ThemSanPhamCommand>  
{  
    public ThemSanPhamValidator()  
    {  
        RuleFor(x => x.Ma).NotEmpty().Matches("^[A-Za-z]{2}[0-9]{3}$").WithMessage("Ma gom 2 chu cai  
        RuleFor(x => x.Ten).NotEmpty().Length(2, 200);  
        RuleFor(x => x.Nhom).NotEmpty().MaximumLength(100);  
        RuleFor(x => x.DonGia).GreaterThanOrEqualTo(0);  
        RuleFor(x => x.TonDau).InclusiveBetween(0, 1_000_000);  
    }  
}
```

Các quy tắc hữu ích: `NotEmpty`, `NotNull`, `Length/MinimumLength/MaximumLength`, `Matches`, `EmailAddress`, `InclusiveBetween`, `GreaterThan`, `Must(x => ...)` (điều kiện tùy ý), `When/Unless` (điều kiện áp dụng), `RuleForEach` (phần tử danh sách), `SetValidator` (validator lồng), `MustAsync` (gọi CSDL — cẩn thận chi phí), `CascadeMode`.

Đăng ký tự động bằng quét assembly (trong `AddApplication`); `ValidationBehavior` (Chương 6) chạy mọi `IValidator<TRequest>` và trả `Result` lỗi.

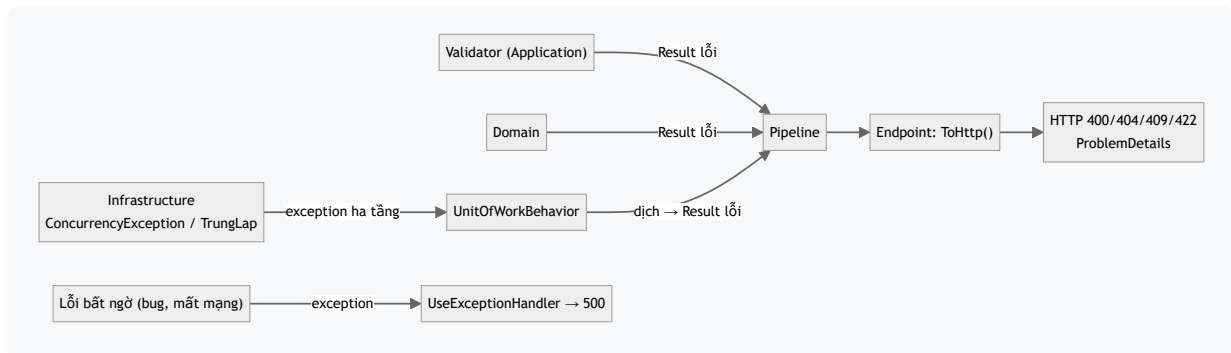
Ba tầng kiểm tra — mỗi tầng một việc

Tầng	Kiểm tra gì	Công cụ	Lỗi
Web/DTO	JSON đúng cú pháp/kiểu	model binding	400
Application (validator)	định dạng, phạm vi của đầu vào lệnh	FluentValidation	400
Domain	bất biến nghiệp vụ (đủ hàng, mã hợp lệ, số tiền không âm)	factory/phương thức trả <code>Result</code>	400/404/409/422
CSDL	ràng buộc cuối cùng (UNIQUE, CHECK, FK)	schema	dịch thành <code>XungDot...</code>

Đừng chỉ tin một tầng: **Domain tự bảo vệ** (đối tượng không bao giờ ở trạng thái sai dù ai gọi), validator cho **phản hồi sớm, đầy đủ, thân thiện**, CSDL là **lưới cuối**. Chúng có thể *trùng lặp* một phần (vd `TonDau >= 0`) — chấp nhận, vì mục đích khác nhau. Để tránh lệch, nhiều đội để validator gọi lại hàm của Domain (`MaSanPham.Tao(x).ThanhCong`) cho quy tắc quan trọng.

Ví dụ thực tế về trùng khoá: `ThemSanPhamHandler` kiểm tra `TonTaiMaAsync` trước (phản hồi tốt) **và** ràng buộc `UNIQUE` + `TrungLapException` xử lý trường hợp hai request đồng thời cùng qua bước kiểm tra — "kiểm tra rồi hành động" (check-then-act) luôn có khe race; CSDL là người phân xử.

Tổng hợp luồng lỗi trong hệ thống



Một bảng duy nhất quyết định "lỗi nào thành mã nào" (`LoiSangHttp`), và mọi lỗi nghiệp vụ đều cùng hình dạng `ProblemDetails`.

Lỗi thường gặp

- Trả `Result` nhưng vẫn ném exception cho cùng loại lỗi (hai luồng lỗi song song).
- `Loi` chỉ có chuỗi mô tả, không mã ổn định → client phải phân tích văn bản.
- Đưa `LoaiLoi.NghiepVu` thành mã HTTP trong Domain (rò rỉ HTTP vào miền).
- Truy cập `.GiaTri` mà không kiểm tra `ThanhCong`.
- Ép mọi thứ thành `Result` kể cả lỗi hạ tầng (nuốt lỗi 500 thành 4xx).
- Chỉ validate ở validator, Domain để `public set` — đường tắt qua code khác bỏ qua luật.
- Dùng `MustAsync` gọi CSDL cho mỗi rule, sinh nhiều truy vấn.
- Thông báo lỗi tiết lộ chi tiết nội bộ.

Bài tập

- Mở rộng `Loi` thành danh sách lỗi theo trường (`ICollection<LoiTruong>`); cập nhật `ValidationBehavior` và `LoiSangHttp` để trả errors như `ValidationProblem`.
- Viết extension `Ensure(Func<T,bool> dieuKien, Loi loi)` cho `Result<T>` và dùng nó trong một chuỗi `Bind`.
- Thêm rule FluentValidation `Must(BeUniqueMa)` gọi `ISanPhamRepository` — bàn về ưu/nhược so với kiểm tra trong handler.
- Đổi endpoint xuất kho để trả **body** `{ tonMoi }` thay vì 204 — cần đổi gì ở command/handler (gợi ý: `Result<int>`)?
- Viết test HTTP kiểm tra `title` (mã lỗi) trong `ProblemDetails` cho 404, 409, 422, 400.

Chương 8 — Sự kiện miền, xử lý nền và Outbox

Mục tiêu học

Sau chương này, bạn sẽ:

- Phân biệt **sự kiện miền (domain event)** và **sự kiện tích hợp (integration event)**.
- Hiểu **vấn đề ghi kép (dual write)** và giải pháp **Transactional Outbox**.
- Cài đặt outbox + **BackgroundService** xử lý nền, đảm bảo **ít nhất một lần (at-least-once)**.
- Thiết kế bên nhận **idempotent**, và kiểm thử hệ thống bất đồng bộ một cách xác định.

Code: [KhoDbContext.cs](#) (thu sự kiện → outbox), [OutboxProcessor.cs](#) , [IntegrationTests.cs](#) .

Bài toán

Khi tồn kho chạm ngưỡng cảnh báo (`SapHetHang`), hệ thống cần **báo cho bộ phận mua hàng**: gửi email, đẩy tin nhắn lên message bus, gọi webhook. Cách "ngây thơ":

```
await uow.LuuAsync(ct);           // 1) ghi CSDL
await emailSender.GuiAsync(...);  // 2) gửi email ← nếu sập/lỗi ở đây?
```

Vấn đề ghi kép (dual write)

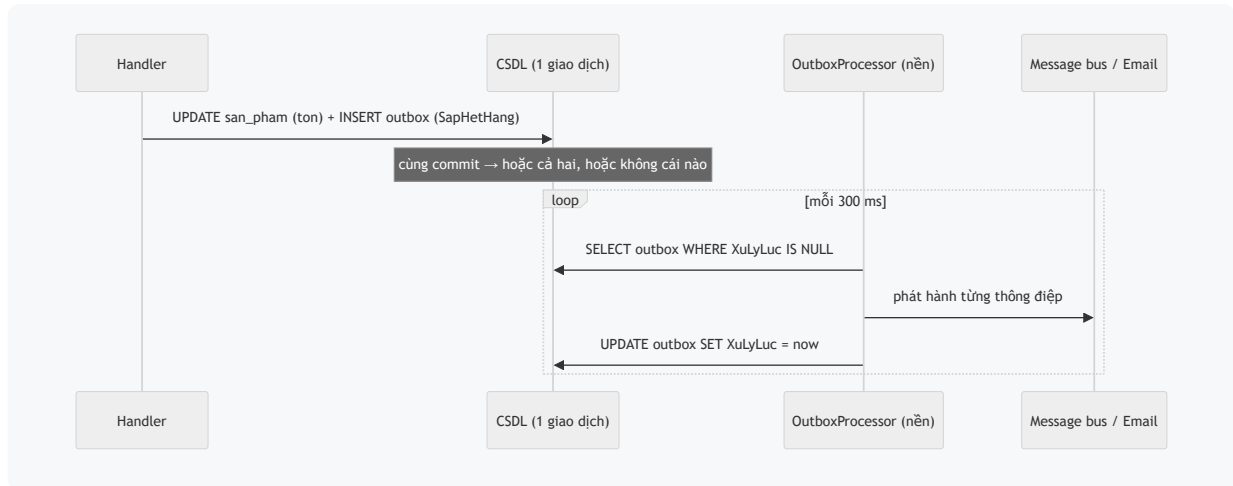
Hai hệ thống (CSDL và email/message bus) **không thể chung một giao dịch**:

Thứ tự	Sự cố	Hậu quả
Lưu CSDL rồi gửi	tiến trình sập/mạng lỗi giữa hai bước	dữ liệu đã đổi nhưng không ai được báo (mất sự kiện)
Gửi rồi lưu CSDL	lưu thất bại (xung đột)	đã báo cho người khác về thay đổi không hề xảy ra
Bọc try/catch, thử lại	lặp lại, hay sập đúng lúc	gửi trùng hoặc bỏ sót

Không có thứ tự nào an toàn. Cần một cơ chế bảo đảm: *"nếu dữ liệu đã đổi thì sự kiện chắc chắn sẽ được phát hành (ít nhất một lần)"*.

Transactional Outbox

Ý tưởng: **ghi sự kiện vào một bảng outbox trong CÙNG giao dịch CSDL** với thay đổi nghiệp vụ. Một tiến trình nền **đọc bảng outbox và phát hành** ra bên ngoài, rồi đánh dấu đã xử lý.



Nhờ **atomicity** của giao dịch: sự kiện tồn tại **khi và chỉ khi** thay đổi nghiệp vụ được commit. Sập giữa chừng không mất gì: khi khởi động lại, tiến trình nền vẫn thấy dòng chưa xử lý.

Từ sự kiện miền đến outbox

Domain chỉ **ghi nhận** sự kiện (Chương 3: `PhatSuKien`). Infrastructure thu chúng ngay trong `LuuAsync` (Unit of Work):

```

public async Task<int> LuuAsync(CancellationTokens ct)
{
    // 1) Thu sự kiện miền từ MỌI aggregate đang theo dõi -> thông điệp outbox
    var aggregates = ChangeTracker.Entries<Entity<int>>().Select(e => e.Entity).Where(e => e.SuKienMien);
    foreach (var agg in aggregates)
    {
        foreach (var sk in agg.SuKienMien)
        {
            Outbox.Add(new OutboxMessage { Loai = sk.GetType().Name, NoiDung = JsonSerializer.Serialize(sk) });
            agg.XoaSuKien();
        }
    }

    // 2) MỘT SaveChanges = dữ liệu nghiệp vụ + outbox, cùng một giao dịch
    try { return await SaveChangesAsync(ct); }
    catch (DbUpdateConcurrencyException) { throw new ConcurrencyException(); }
    ...
}

```

Handler và domain **không biết** outbox tồn tại — chỉ `PhatSuKien`. Mẫu này cũng chỉ ra vì sao tách sự kiện khỏi hành động: aggregate không cần biết *ai* nghe.

Test tích hợp `SuKienMien_DuocGhiVaoOutboxCungGiaoDich_...` kiểm chứng: sau `POST` tạo sản phẩm và xuất 3 (7 -> 4, chạm ngưỡng 5), bảng `outbox` có `SanPhamDaTao` và `SapHetHang`.

Tiến trình nền: BackgroundService

```

public class OutboxProcessor(IServiceScopeFactory scopeFactory, TimeProvider dongHo, ILogger<OutboxProcessor> logger)
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        using var timer = new PeriodicTimer(TimeSpan.FromMilliseconds(300));
    }
}

```

```

        while (await timer.WaitForNextTickAsync(stoppingToken))
            await XuLyMotLanAsync(stoppingToken);
    }

    public async Task<int> XuLyMotLanAsync(CancellationTokens ct)
    {
        await using var scope = scopeFactory.CreateAsyncScope(); // DbContext Scoped → t
        var db = scope.ServiceProvider.GetRequiredService<KhoDbContext>();
        var phatHanh = scope.ServiceProvider.GetRequiredService<IPhatHanhSuKien>();

        var cho = await db.Outbox.Where(o => o.XuLyLuc == null && o.SoLanThu < ToiDaSoLanThu)
            .OrderBy(o => o.TaoLuc).Take(ToiDaMoiLan).ToListAsync(ct);

        foreach (var m in cho)
        {
            try { await phatHanh.PhatHanhAsync(m.Loai, m.NoiDung, ct); m.XuLyLuc = dongHo.GetUtcNow();
            catch (Exception e) { m.SoLanThu++; m.LoiCuoi = e.Message; log.LogWarning(e, "Phat hanh oi
            }
            if (cho.Count > 0) await db.SaveChangesAsync(ct);
            return cho.Count;
        }
    }
}

```

Điểm cần chú ý:

- **BackgroundService** (từ `Microsoft.Extensions.Hosting`) chạy suốt vòng đời ứng dụng; `PeriodicTimer` là cách hiện đại để lập định kỳ (không trôi thời gian như `Task.Delay` trong vòng lặp). Đăng ký: `services.AddHostedService(...)`.
- **Dịch vụ Scoped trong Singleton**: `BackgroundService` là Singleton; `DbContext` là Scoped ⇒ dùng `IServiceScopeFactory.CreateAsyncScope()` cho **mỗi lượt xử lý** (Tập 2, Chương 5 — captive dependency).
- **Tách `XuLyMotLanAsync` thành phương thức công khai** để test gọi trực tiếp, xác định, không chờ timer (mẫu quan trọng để kiểm thử xử lý nền).
- **Thứ tự** theo `TaoLuc` để giữ thứ tự tương đối; **giới hạn lô (20)** và **số lần thử (5)** tránh một thông điệp hỏng chặn cả hàng đợi ("poison message"); sau khi vượt số lần thử, để yên cho người vận hành xử lý (dead-letter).
- Nhiều instance chạy đồng thời? Cần **khóa** để tránh xử lý trùng (`SELECT ... FOR UPDATE SKIP LOCKED` trên PostgreSQL/SQL Server hint `READPAST`, hoặc chỉ chạy processor ở một instance). Với at-least-once, trùng vẫn an toàn nếu bên nhận idempotent.

Bảo đảm ít nhất một lần (at-least-once)

Nếu tiến trình sập **sau khi** phát hành nhưng **trước khi** đánh dấu `XuLyLuc`, lần chạy sau **phát hành lại**. Vì vậy hệ thống phân phối là **at-least-once**, *không phải* exactly-once (gần như không thể trong hệ phân tán). Điều đó **đặt trách nhiệm** sang bên nhận: phải **idempotent** — xử lý cùng một thông điệp hai lần cho kết quả như một lần.

Cách làm:

- Mỗi thông điệp có **Id duy nhất** (`OutboxMessage.Id`); bên nhận lưu Id đã xử lý (bảng `da_xu_ly`) và bỏ qua trùng.
- Hoặc thao tác tự nhiên idempotent (đặt trạng thái = X thay vì "cộng thêm 1").
- Chương 13 đi sâu vào idempotency.

Sự kiện miền hay sự kiện tích hợp?

	Domain event	Integration event
Phạm vi	trong một bounded context/ứng dụng	giữa các hệ thống/dịch vụ
Ví dụ	SapHetHang(Ma, TonKho...)	SanPhamSapHet gửi cho dịch vụ Mua hàng
Truyền	đồng bộ trong tiến trình (handler cục bộ) hoặc outbox	qua message bus (RabbitMQ, Azure Service Bus, Kafka)
Hợp đồng	tự do thay đổi	ổn định, có phiên bản (người khác phụ thuộc)

Solution dùng một bảng outbox cho cả hai để đơn giản: **sự kiện miền được phát hành thẳng làm thông điệp**. Trong hệ thống lớn, thường **ánh xạ** sự kiện miền → integration event (DTO ổn định) trước khi phát hành, để thay đổi nội bộ không làm vỡ bên ngoài.

Xử lý sự kiện miền *trong tiến trình* (tuỳ chọn)

Đôi khi cần phản ứng **ngay trong cùng giao dịch** (ví dụ cập nhật một aggregate khác — cần nhất quán mạnh). Mẫu: trong `LuuAsync`, trước `SaveChangesAsync`, gọi `IDomainEventHandler<T>` cho từng sự kiện. Cẩn thận: handler chạy trong giao dịch, lỗi làm hỏng cả lệnh; và làm mờ ranh giới "một aggregate mỗi giao dịch". Ưu tiên **bất đồng bộ qua outbox** với nhất quán cuối cùng trừ khi nghiệp vụ đòi hỏi khác.

IPhatHanhSuKien — cổng phát hành

```
public interface IPhatHanhSuKien { Task PhatHanhAsync(string loai, string noiDungJson, CancellationTok
```

Mặc định ghi log (`LogPhatHanhSuKien`); thực tế thay bằng: gửi lên Azure Service Bus/RabbitMQ (qua **MassTransit, Wolverine, NServiceBus, Rebus**), gọi webhook, gửi email. Vì là cổng ở Application, đổi cài đặt **không đổi** domain/handler; test dùng `PhatHanhGia` gom vào danh sách.

Kiểm thử bất đồng bộ một cách xác định

Test **không** `Thread.Sleep` chờ timer. Cách làm trong solution:

1. Tắt tiến trình nền khi test: `builder.UseSetting("Outbox:Bat", "false")` .
2. Lấy `OutboxProcessor` từ DI và **gọi** `XuLyMotLanAsync` đúng lúc cần.
3. Thay `IPhatHanhSuKien` bằng bản giả ghi lại.
4. Kiểm chứng: sau **hai** lần chạy, mỗi sự kiện chỉ được phát hành **một** lần (đã đánh dấu).

```
await bxl.XuLyMotLanAsync(CancellationTokens.None);
await bxl.XuLyMotLanAsync(CancellationTokens.None);           // lần hai: không phát hành lại
Assert.Single(cuaEE, p => p.Loai == "SapHetHang");
```

Dọn dẹp và vận hành

- **Xoá/lưu trữ** dòng outbox cũ đã xử lý (job định kỳ) để bảng không phình.
- **Giám sát**: số dòng chờ, tuổi dòng cũ nhất, số dòng vượt số lần thử — cảnh báo khi tăng (Chương 11).
- **Thứ tự**: outbox đảm bảo thứ tự *cục bộ* theo aggregate chứ không phải toàn cục; thiết kế bên nhận chịu được sự kiện đến lệch thứ tự (dùng số phiên bản/ `XayRaLuc`).

- Chọn **khoảng quét**: ngắn (300 ms) = độ trễ thấp nhưng tải CSDL; dài = trễ cao. Hoặc "đánh thức" processor ngay sau commit (`Channel<T>`) và quét dự phòng chậm.

Lỗi thường gặp

- Vỡ gọi email/bus **trong handler** (dual write, mất sự kiện khi sập).
- Ghi outbox trong **giao dịch khác** với thay đổi nghiệp vụ (mất ý nghĩa của outbox).
- Bên nhận không idempotent → xử lý trùng gây hậu quả (gửi email hai lần, trừ tiền hai lần).
- Thông điệp hỏng lặp mãi (không giới hạn số lần thử) chặn hàng đợi.
- `DbContext` Scoped bị tiêm thẳng vào `BackgroundService` (captive dependency).
- `Thread.Sleep` trong test chờ tiến trình nền → test chậm và bấp bênh.
- Phát sự kiện miễn cả khi thao tác thất bại (Chương 3).
- Không dọn dẹp outbox, bảng phình vô hạn.

Bài tập

1. Thêm sự kiện `GiaThayDoi` và đảm bảo nó vào outbox khi `DoiGia` thành công; viết test.
2. Viết `IPhatHanhSuKien` gửi HTTP POST tới webhook (dùng `HttpClient` + resilience — Tập 2, Chương 19); xử lý lỗi bằng `SoLanThu`.
3. Thêm bảng `da_xu_ly` phía nhận và một consumer idempotent bỏ qua thông điệp trùng Id.
4. Viết job dọn outbox xóa dòng `XuLyLuc` cũ hơn 7 ngày (một `BackgroundService` khác hoặc `ExecuteDeleteAsync`).
5. Cố tình làm `PhatHanhAsync` ném lỗi 3 lần rồi thành công; quan sát `SoLanThu`, `LoiCuoi` và viết test.

Chương 9 — Chiến lược kiểm thử

Mục tiêu học

Sau chương này, bạn sẽ:

- Thiết kế **chiến lược kiểm thử tổng thể**: kim tự tháp, các loại test và **cái gì test ở tầng nào**.
- Viết test **Domain / Application / Architecture / Integration** như trong `kho-clean` và hiểu vì sao mỗi loại tồn tại.
- Dùng **test data builder**, fake vs mock, **CSDL thật** trong test (SQLite file, Testcontainers).
- Kiểm thử **đồng thời (concurrency)** và **bất đồng bộ** một cách xác định; đo độ phủ có ý nghĩa.

Code: [code/kho-clean/Kho.Tests/](https://github.com/kho-clean/kho-tests) — **45 test** (đã đạt) chia thành `DomainTests`, `ApplicationTests`, `ArchitectureTests`, `IntegrationTests`, `IdempotencyAuditTests` (Chương 13) (mỗi `Theory` tính nhiều ca).

Vì sao cần "chiến lược"?

Không có chiến lược, đội thường rơi vào một trong hai cực: **quá ít test** (sợ sửa code) hoặc **quá nhiều test giòn** (mỗi refactor làm đở 200 test, không ai tin test). Chiến lược trả lời: *test cái gì, ở tầng nào, bằng công cụ gì, để đạt độ tin cậy cao với chi phí bảo trì thấp?*

Kim tự tháp kiểm thử



Tầng test	Kiểm chứng	Phụ thuộc	Số lượng	Ví dụ trong solution
Domain unit	luật nghiệp vụ, bất biến, sự kiện	không	nhiều nhất	XuatKho_ChamNguongCanhBao_PhátSápHếtHangMôTlan
Application	điều phối ca sử	fake cho cổng	nhiều	ThemSanPham_SaiDinhDang_ValidationBehaviorChanTruocHandler

Tầng test	Kiểm chứng	Phụ thuộc	Số lượng	Ví dụ trong solution
	dùng, pipeline			
Architecture	quy tắc phụ thuộc, quy ước	file/reflection	ít, rẻ	Domain_KhongPhuThuocGi
Integration	HTTP + DI + EF + CSDL thật ghép đúng	môi trường thật (SQLite file)	vừa	XuatDongThoi_KhongBaoGioAm_..., outbox
E2E/UI	luồng người dùng qua giao diện	toàn hệ thống	rất ít	(ngoài phạm vi; Playwright)

Kiến trúc sạch **làm kim tự tháp khả thi**: vì nghiệp vụ nằm ở Domain/Application không phụ thuộc hạ tầng, phần lớn test chạy trong **micro-giây/mili-giây**. Ở kiến trúc "khối" (Tập 2), muốn test nghiệp vụ phải dựng CSDL và HTTP → toàn integration test, chậm, nên ít khi chạy.

1. Domain test — nhanh nhất, giá trị nhất

```
[Fact]
public void XuatKho_KhongDuHang_ThatBaiVaTonKhongDoi()
{
    var sp = Tao(ton: 2);
    sp.XoaSuKien();

    var kq = sp.XuatKho(5, Gio);

    Assert.False(kq.ThanhCong);
    Assert.Equal("SanPham.KhongDuHang", kq.Lo!.Ma);
    Assert.Equal(2, sp.TonKho);
    Assert.Empty(sp.SuKienMien); // thất bại thì KHÔNG có sự kiện
}
```

Không DI, không mock, không I/O. **Kiểm tra hành vi quan sát được** (kết quả, trạng thái, sự kiện) chứ không kiểm tra cài đặt bên trong. Thời gian truyền vào (Gio) thay vì `DateTime.Now` để test **xác định**. Dùng [Theory] cho biên (mã hợp lệ/không, giá âm, tồn đầu âm).

2. Application test — với bản giả (fake) cho cổng

```

class RepoGia : ISanPhamRepository { public List<SanPham> Ds { get; } = []; ... }
class UowGia : IUnitOfWork { public int SoLanLuu { get; private set; } public Exception? Nem { get; set; }

var s = new ServiceCollection();
s.AddSingleton<ISanPhamRepository>(_repo).AddSingleton<IUnitOfWork>(_uow).AddSingleton<IKhoDocDuLieu>(_khoDocDuLieu);
s.AddLogging(); s.AddApplication(); // pipeline THẬT + handler THẬT
_sender = s.BuildServiceProvider().CreateScope().ServiceProvider.GetRequiredService<ISender>();

```

Test đi qua **mediator và toàn bộ behavior thật** (`_sender.Send(...)`), chỉ thay hạ tầng. Nhờ vậy kiểm chứng cả **hợp tác** giữa handler–behavior: validation chặn trước handler; UoW chỉ lưu khi thành công; lỗi hạ tầng thành 409 ; truy vấn không bao giờ lưu.

Fake hay mock?

- **Fake**: cài đặt đơn giản, hoạt động thật (repository trong `List`). Test đọc như mô tả **hành vi**, bèn qua refactor.
- **Mock** (NSubstitute/Moq): thiết lập kỳ vọng lời gọi (`Received(1).LuuAsync()`). Hợp để kiểm tra **tác dụng phụ ra ngoài** (gửi email), nhưng dùng quá đà làm test **đính chặt vào cài đặt** (đổi cách gọi là đổ đủ hành vi vẫn đúng).
- Quy tắc: **fake cho thứ có trạng thái** (repository, đồng hồ), **mock/spy cho thứ chỉ "gửi đi"** (email, message bus); ưu tiên kiểm tra **kết quả** hơn kiểm tra **lời gọi**.

3. Architecture test

Đã có từ Chương 1 (đọc csproj/reflection). Thêm quy ước dễ vi phạm: aggregate không có setter công khai, handler `sealed` , tên kết thúc `Handler ...` Rẻ, chạy tức thì, ngăn xói mòn kiến trúc.

4. Integration test — môi trường thật, ít nhưng chắc

```

public class KhoFactory : WebApplicationFactory<Program>
{
    private readonly string _file = Path.Combine(Path.GetTempPath(), $"kho-clean-test-{Guid.NewGuid()}");
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseSetting("Outbox:Bat", "false"); // tắt tiến trình nền → test chỉ
        builder.ConfigureServices(s =>
        {
            s.RemoveAll<DbContextOptions<KhoDbContext>>();
            s.AddDbContext<KhoDbContext>(o => o.UseSqlite($"Data Source={_file};Default Timeout=30"));
            s.RemoveAll<IPhatHanhSuKien>(); s.AddSingleton<IPhatHanhSuKien>(PhatHanh);
        });
    }
}

```

Nguyên tắc chọn **CSDL trong integration test**:

Lựa chọn	Đánh giá
EF InMemory provider	X không có ràng buộc/giao dịch/SQL thật → "xanh giả"

Đặt tên, cấu trúc, độ tin cậy

- **Tên** mô tả *tình huống* → *kết quả*: `XuatKho_KhongDuHang_ThatBaiVaTonKhongDoi`. Khi đỏ, tên đã nói "hỏng cái gì".
- **Arrange–Act–Assert**, một hành vi mỗi test; ít điều kiện/vòng lặp.
- **Độc lập & lặp lại được**: không phụ thuộc thứ tự, giờ hệ thống, dữ liệu chung; dùng `TimeProvider` (`FakeTimeProvider` từ `Microsoft.Extensions.TimeProvider.Testing`).
- **Nhanh**: bộ Domain+Application chạy dưới 1 giây; tách integration ra (`[Trait("Loại", "Integration")]`) để chạy khi cần.
- **Xoá test flaky** ngay hoặc sửa gốc rễ; test không tin cậy còn tệ hơn không test.
- Đừng test **framework/thư viện** (EF có lưu được không?) — test **quyết định của bạn**.

Độ phủ và mutation testing

Độ phủ (coverage) đo dòng được chạy qua, **không** đo chất lượng assert. Dùng như **đèn đỏ** (vùng nghiệp vụ quan trọng phủ thấp = rủi ro) chứ không như **mục tiêu** (đuổi 100% sinh test vô nghĩa). Công cụ mạnh hơn: **mutation testing** ([Stryker.NET](#)) — tự sửa code (đổi `<` thành `<=`) và kiểm tra test có **bắt được** không; điểm mutation phản ánh sức mạnh thật của bộ test.

```
dotnet test --collect:"XPlat Code Coverage"
dotnet tool install -g dotnet-stryker && dotnet stryker # mutation testing
```

Kiểm thử hợp đồng và bảo mật

- **Contract test** (schema OpenAPI/Pact): đảm bảo client và server đồng ý hình dạng request/response — quan trọng khi nhiều đội/dịch vụ (Chương 17). Có thể snapshot OpenAPI trong CI và so sánh để **phát hiện breaking change**.
- **Test bảo mật**: `401/403` cho mọi endpoint nhạy cảm (Tập 2, Chương 22 có `KhongDangNhap_MoiEndpointTra401`), IDOR, rate limit, header.
- **Test hiệu năng/tải** (k6, NBomber): trước khi ra sản phẩm, đo độ trễ p95 và tải tối đa (Chương 11).

Quy trình gợi ý

1. Viết Domain test cho luật trước (TDD khi luật khó).
2. Handler + Application test với fake.
3. Vài integration test cho luồng chính và các lỗi ghép nối hay gặp (JSON, route, DI, migration, đồng thời).
4. Architecture test trong CI.
5. Test chạy trên **mỗi commit** (Chương 15); integration nặng (Testcontainers) chạy trên PR/đêm.
6. Khi có bug production: **viết test tái hiện trước**, rồi sửa.

Lỗi thường gặp

- Chỉ có E2E/integration test → chậm, giòn, không dám chạy thường xuyên.
- Test kiểm tra cài đặt (số lần gọi, thứ tự nội bộ) thay vì hành vi.
- Dùng EF InMemory và tin rằng CSDL "đã được test".
- `Thread.Sleep` chờ tiến trình nền.
- Test phụ thuộc thứ tự/dữ liệu chung → lúc xanh lúc đỏ.

- Mock mọi thứ, kể cả Domain.
- Đuổi độ phủ 100% bằng test không có assert.
- Không có test kiến trúc/hợp đồng: ranh giới bị phá dần mà không ai biết.

Bài tập

1. Viết `SanPhamBuilder` và chuyển ba test Domain sang dùng nó.
2. Thêm integration test: `POST` thêm sản phẩm hai lần **đồng thời** cùng mã; đúng một `201`, còn lại `409`; không có `500`.
3. Chạy `dotnet stryker` (hoặc tự "đột biến" một điều kiện trong `XuatKho`) và xem test nào bắt được.
4. Thêm test kiến trúc: không lớp nào trong Application dùng `System.Net.Http`; hoặc mọi `HttpRequestHandler` nằm trong namespace `Kho.Application.*`.
5. (Cần Docker) Thay SQLite bằng PostgreSQL trong Testcontainers cho một test; ghi lại điều gì khác biệt.

Chương 10 — Caching

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **caching** giải quyết gì và những **rủi ro** đi kèm ("hai vấn đề khó: đặt tên và vô hiệu hoá cache").
- Dùng `IMemoryCache`, `HybridCache` (.NET 9+), **Output Caching**, và biết khi nào cần cache phân tán (Redis).
- Nhận ra và chặn **cache stampede**; thiết kế **chiến lược vô hiệu hoá**.
- Áp dụng cache đúng chỗ (pipeline behavior cho query) và tránh cache dữ liệu sai đối tượng/người dùng.

Code: [code/ch10-caching/](#) — nguồn dữ liệu "chậm 300 ms" có bộ đếm lời gọi để **thấy** hiệu quả bằng số.

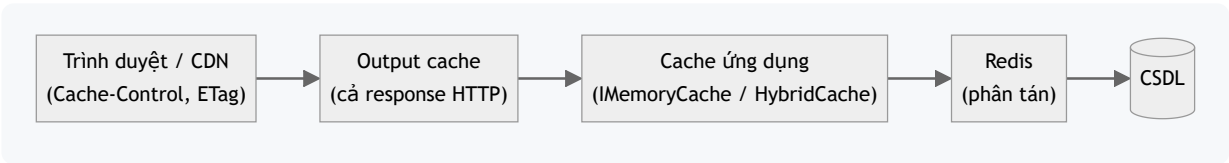
Cache là gì, để làm gì?

Cache lưu kết quả đắt (truy vấn CSDL, gọi API, tính toán) ở nơi **rẻ và gần** để lần sau dùng lại. Lợi: **giảm độ trễ**, **giảm tải** CSDL/dịch vụ ngoài, **tăng thông lượng**. Cái giá: **dữ liệu có thể cũ** (staleness), độ phức tạp vô hiệu hoá, thêm điểm lỗi, tốn bộ nhớ.

Khi nào đáng cache: dữ liệu **đọc nhiều, ghi ít**, tính toán **đắt**, chấp nhận **cũ vài giây-phút** (danh mục sản phẩm, cấu hình, báo cáo, kết quả API ngoài). Không cache: dữ liệu cần **luôn chính xác tuyệt đối** (số dư, tồn kho khi đặt hàng), dữ liệu riêng từng người dùng mà không tách khoá, thứ rẻ sẵn.

Quy tắc: đo trước khi cache. Cache che giấu vấn đề (truy vấn chậm cần chỉ mục) thay vì sửa. Cache là *tối ưu*, không phải *thiết kế*.

Các tầng cache



Tầng	Cache gì	Công cụ	Khi nào
Client/CDN	response HTTP tĩnh/công khai	Cache-Control, ETag, CDN	tài nguyên tĩnh, API công khai
Output cache	toàn bộ response endpoint	AddOutputCache	trang/báo cáo giống nhau cho nhiều người
Cache ứng dụng	đối tượng/DTO/kết quả truy vấn	IMemoryCache, HybridCache	dữ liệu dùng lại giữa nhiều endpoint
Cache phân tán	như trên, chia sẻ giữa các instance	Redis, SQL Server	nhiều instance, cần chung một bộ nhớ đệm

1. IMemoryCache — cache-aside thủ công

```
app.MapGet("/memory/{id:int}", async (int id, IMemoryCache cache, NguonDuLieuCham nguon) =>
{
    if (cache.TryGetValue($"sp:{id}", out SanPhamCache? co)) return co!;    // có trong cache
    var moi = await nguon.LayAsync(id);    // không có: đi lấy
    cache.Set($"sp:{id}", moi, TimeSpan.FromSeconds(30));
    return moi;
});
```

Mẫu **cache-aside**: kiểm tra cache → thiếu thì lấy nguồn → ghi vào cache. Đơn giản, nhưng có ba vấn đề:

1. **Chỉ trong một tiến trình** (mỗi instance một cache riêng, không nhất quán khi scale ngang).
2. **Không chống stampede** (xem dưới).
3. Phải tự quản thời hạn, khoá, kích thước (`SizeLimit`), vô hiệu hoá.

Cache stampede (thundering herd)

Khi mục hết hạn/chưa có, **nhiều request đồng thời** cùng thấy "thiếu" và **cùng chạy** truy vấn đất. Chạy code mẫu, 20 request đồng thời tới `/memory/2` (nguồn 300 ms):

```
soLanGoiNguon: 20    ← 20 lần truy vấn thật, dù chỉ cần 1
```

Với dữ liệu nóng và nguồn đất, một mục hết hạn có thể đánh sập CSDL. Cần **hợp nhất** các lần tính đồng thời (single-flight): request đầu tiên tính, các request khác **chờ kết quả đó** .

2. HybridCache — khuyến nghị cho ứng dụng mới

`Microsoft.Extensions.Caching.Hybrid` (GA từ .NET 9) kết hợp **L1 (bộ nhớ cục bộ, cực nhanh)** và **L2 (cache phân tán như Redis, tuỳ chọn)** , **chống stampede sẵn, tag** để vô hiệu hoá theo nhóm:

```
builder.Services.AddHybridCache(o => o.DefaultEntryOptions = new HybridCacheEntryOptions
{
    Expiration = TimeSpan.FromSeconds(30),    // hạn tổng thể (L2)
    LocalCacheExpiration = TimeSpan.FromSeconds(10),    // hạn ở L1 (cục bộ)
});

app.MapGet("/hybrid/{id:int}", async (int id, HybridCache cache, NguonDuLieuCham nguon, CancellationToken ct) =>
{
    await cache.GetOrCreateAsync($"sp:{id}",
        async token => await nguon.LayAsync(id),    // factory: chỉ CHẠY MỘT LẦN dù nhiều req
        tags: ["san-pham", $"sp:{id}"],
        cancellationToken: ct));
```

Kết quả thật với 20 request đồng thời:

```
IMemoryCache thủ công → soLanGoiNguon: 20
HybridCache           → soLanGoiNguon: 1    ← các request đợi chung kết quả của request đầu
```

Cách hoạt động: `GetOrCreateAsync` tra L1 → L2 → gọi factory; nếu một khoá đang được tạo, request khác **chờ cùng Task** . Kết quả được **tuần tự hoá** khi lưu vào L2 (nên kiểu dữ liệu phải serialize được; mặc định `System.Text.Json`).

Nếu **không cấu hình L2**, `HybridCache` chỉ dùng L1 nhưng vẫn có chống stampede và API thống nhất — dễ dàng **nâng lên Redis** sau này bằng `AddStackExchangeRedisCache(...)` (đăng ký `IDistributedCache`) mà không đổi code gọi.

Vô hiệu hoá theo tag

```
app.MapPost("/hybrid/{id:int}/cap-nhat", async (int id, HybridCache cache, NguồnDulieuCham nguon) =>
{
    nguon.Sua(id); // 1) ghi vào nguồn thật
    await cache.RemoveByTagAsync($"sp:{id}"); // 2) vô hiệu hoá mọi mục gắn thẻ này
    return Results.NoContent();
});
```

Chạy thử: lấy `/hybrid/3` → `phienBan: 1`; gọi cập nhật; lấy lại → `phienBan: 2` (cache đã bị xoá đúng lúc).

3. Output caching

Cache **cả response HTTP** của endpoint theo chính sách:

```
builder.Services.AddOutputCache(o => o.AddPolicy("BaoCao", p => p.Expire(TimeSpan.FromSeconds(15))).TagBuilder
app.UseOutputCache();

app.MapGet("/bao-cai", ...).CacheOutput("BaoCao");
app.MapPost("/bao-cai/lam-moi", async (IOutputCacheStore store, CancellationToken ct) => { await store
```

Kết quả thử: hai lần `GET /bao-cai` trả **cùng taolu và soLanChayThat: 1** (handler chỉ chạy một lần); sau `POST /bao-cai/lam-moi` lần kế tiếp chạy lại (`soLanChayThat: 2`). Mặc định output cache **chỉ cache GET/HEAD trả 200**, **không có Authorization /cookie** (an toàn: tránh cache dữ liệu riêng tư); có thể `VaryByQuery`, `VaryByHeader`, `VaryByValue`. Response caching (`Cache-Control`) khác: dựa vào header để **client/proxy** cache; output caching cache **phía server** và **bạn điều khiển** vô hiệu hoá.

4. Cache phân tán (Redis)

Nhiều instance ⇒ cache cục bộ mỗi nơi một khác (người dùng thấy dữ liệu khác nhau tùy gặp instance nào) và mỗi instance "làm nóng" riêng. **Redis** là bộ nhớ đệm chung:

```
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
builder.Services.AddStackExchangeRedisCache(o => o.Configuration = builder.Configuration.GetConnectionString("Redis"));
builder.Services.AddHybridCache(); // tự dùng IDistributedCache làm L2
```

Redis còn dùng làm kho phiên, khoá phân tán, hàng đợi nhẹ, rate limit phân tán (Tập 2, Chương 21). Chi phí: thêm hạ tầng và một điểm lỗi; mỗi truy cập là một lượt mạng — vì vậy L1 vẫn có giá trị. **Thiết kế cho cache hồng**: khi Redis không sẵn sàng, ứng dụng nên **vẫn chạy** (chậm hơn, chậm CSDL) chứ không sập.

Vô hiệu hoá cache — phần khó nhất

Chiến lược	Cách làm	Ưu/nhược
Hết hạn theo thời gian (TTL)	đặt Expiration ngắn	đơn giản; chấp nhận dữ liệu cũ tối đa TTL
Xoá khi ghi	sau khi cập nhật gọi RemoveByTag/Remove	dữ liệu mới ngay; phải nhớ gọi ở mọi đường ghi
Theo sự kiện	handler sự kiện miền (GiaThayDoi) xoá cache	tách rời (ghi không biết cache); xem outbox Chương 8
Khoá có phiên bản	khoá chứa version (sp:3:v7)	tự "vô hiệu" khi version đổi; cache cũ chết theo TTL

Quy tắc:

1. **Cache chỉ thứ có thể cũ**; với dữ liệu cần chính xác, đọc thẳng nguồn.
2. **Xoá sau khi ghi nguồn** (nếu xoá trước, một request xen giữa có thể nạp lại giá trị cũ vào cache).
3. Luôn đặt **TTL** như lưới an toàn dù có xoá chủ động — nếu quên xoá ở đâu đó, dữ liệu cũ tự hết.
4. Đặt tag theo **thực thể và theo nhóm** (sp:3 , san-pham) để xoá tinh hoặc rộng.

Cache và bảo mật/đúng đắn

- **Khoá phải chứa mọi yếu tố ảnh hưởng kết quả**: người dùng/tenant/ngôn ngữ/quyền. Quên TenantId ⇒ người này thấy dữ liệu người khác (lỗi hổng nghiêm trọng).
- **Không cache lỗi** (hoặc TTL cực ngắn) — lỗi tạm thời không nên "dính" 30 giây; solution đề xuất chỉ cache Result thành công.
- **Không cache dữ liệu nhạy cảm** không mã hoá ở Redis dùng chung; cân nhắc mã hoá/TTL ngắn.
- Cache đối tượng **bất biến** (DTO/record); nếu cache tham chiếu tới đối tượng có thể sửa, một chỗ sửa làm hỏng mọi người.
- **Cache penetration**: khoá không tồn tại bị gọi liên tục → lần nào cũng chạm CSDL. Cache cả kết quả "không có" trong thời gian ngắn.

Cache trong kiến trúc sạch: behavior cho query

Đặt cache thành **pipeline behavior** chỉ cho truy vấn (Chương 6) — handler không biết về cache:

```
public interface ICachedQuery { string CacheKey { get; } string[] Tags { get; } TimeSpan ThoiGianSong

public sealed class CachingBehavior<TRequest, TResponse>(HybridCache cache) : IPipelineBehavior<TRequest, TResponse>
    where TRequest : IQuery<TResponse>, ICachedQuery where TResponse : IKetQua<TResponse>
{
    public async Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next, CancellationToken cancellationToken)
    => await cache.GetOrCreateAsync(request.CacheKey, async _ => await next(),
        new HybridCacheEntryOptions { Expiration = request.ThoiGianSong }, request.Tags, cancellationToken);
}

public sealed record LaySanPhamQuery(string Ma) : IQuery<Result<SanPhamDto>>, ICachedQuery
{
    public string CacheKey => $"sp:{Ma.ToUpperInvariant()}";
    public string[] Tags => ["san-pham", CacheKey];
}
```

```
public TimeSpan ThoiGianSong => TimeSpan.FromSeconds(30);  
}
```

Và **lệnh** vô hiệu hoá: `UnitOfWorkBehavior` (hoặc handler sự kiện) gọi `cache.RemoveByTagAsync(...)` sau khi lưu thành công. Kiểu `Result` phải serialize được để lưu L2 (hoặc chỉ cache DTO, không cache `Result`).

Lỗi thường gặp

- Cache trước khi đo; cache để "che" truy vấn thiếu chỉ mục.
- Không chống stampede → sập CSDL khi mục nóng hết hạn.
- Khoá cache thiếu người dùng/tenant → rò rỉ dữ liệu.
- Xoá cache **trước** khi ghi nguồn; hoặc quên xoá ở một đường ghi.
- Không có TTL → dữ liệu cũ vĩnh viễn khi lỡ quên vô hiệu hoá.
- Cache đối tượng có thể thay đổi (entity EF đang theo dõi) — cache DTO bất biến.
- Cache lỗi; cache dữ liệu cần chính xác (tồn kho khi đặt hàng).
- Redis là điểm lỗi duy nhất — ứng dụng sập khi Redis sập.
- Tuần tự hoá kiểu không hỗ trợ khi dùng L2 (chỉ lộ ra khi bật Redis).

Bài tập

1. Thêm `Cache-Control: public, max-age=60` cho `/khong-cache/{id}` và quan sát bằng `curl -i`; so sánh với output cache phía server.
2. Cache "không tìm thấy" 5 giây trong endpoint hybrid để chống penetration; đo số lần gọi nguồn khi bị tấn công bằng id ngẫu nhiên.
3. Viết `CachingBehavior` ở trên vào `kho-clean` cho `LaySanPhamQuery` và test: lần gọi thứ hai không chạm repository; sau `NhapKhoCommand` cache bị xoá.
4. (Cần Docker) Chạy Redis, cấu hình L2 cho `HybridCache`, chạy hai instance ứng dụng trên hai cổng và xác nhận cập nhật ở instance A làm mất cache ở instance B.
5. Đo (`Stopwatch`) độ trễ p50/p95 của `/khong-cache`, `/memory`, `/hybrid` với 200 request và vẽ nhận xét.

Chương 11 — Quan sát và hiệu năng

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **observability** (khả năng quan sát) và ba trụ cột: **logs, metrics, traces**.
- Cấu hình **OpenTelemetry** cho [ASP.NET Core](#): tracing, metrics, logs, và **truyền ngữ cảnh** giữa các dịch vụ.
- Tự tạo **span** và **metric** nghiệp vụ; phân biệt **liveness** và **readiness** health check.
- Có phương pháp **đo → tìm nút thắt → tối ưu** hiệu năng, kèm danh sách kiểm tra cho EF Core và [ASP.NET Core](#).

Code: [code/ch11-quan-sat/](#) — luồng đặt hàng nhiều bước, in trace/metric/log ra console.

Từ "giám sát" đến "quan sát"

Giám sát (monitoring) trả lời các câu hỏi *đã biết trước* ("CPU bao nhiêu? có lỗi không?"). **Quan sát (observability)** cho phép hỏi *câu hỏi chưa lường trước* ("vì sao đơn hàng của khách X chậm lúc 14:03?") bằng cách **phát dữ liệu đủ giàu từ bên trong**. Ba trụ cột:

Trụ cột	Là gì	Trả lời	Ví dụ
Logs	sự kiện rời rạc có thời điểm	<i>chuyện gì đã xảy ra?</i>	"Dat hang LT001 thất bại: vượt hạn mức"
Metrics	số đo tổng hợp theo thời gian (rẻ, đếm/đo)	<i>hệ thống đang thế nào? xu hướng?</i>	số đơn/giây, p95 độ trễ, tỉ lệ lỗi
Traces	đường đi một request xuyên qua các thành phần/dịch vụ	<i>thời gian đi đâu? ai gọi ai?</i>	request → KiemTraTon → ThanhToan → dịch vụ ngoài

Ba thứ **bổ sung nhau**: metric báo *có vấn đề*, trace chỉ *ở đâu*, log giải thích *vì sao*. Chìa khoá: chúng được **liên kết bằng TraceId**.

OpenTelemetry

OpenTelemetry (OTel) là chuẩn mở (CNCF) cho việc **tạo và xuất** telemetry, độc lập nhà cung cấp: code của bạn phát dữ liệu theo một chuẩn (OTLP); đích là **Jaeger, Grafana Tempo/Loki/Prometheus, Azure Monitor/Application Insights, Datadog, New Relic, Elastic...** Đích đích không đổi code ứng dụng. Trong .NET, OTel dựa trên API có sẵn: **ActivitySource / Activity** (trace), **Meter** (metrics), **ILogger** (logs).

```
builder.Services.AddOpenTelemetry()
    .ConfigureResource(r => r.AddService("kho-dat-hang", serviceVersion: "1.0.0")) // "ai phát"
    .WithTracing(t => t
        .AddAspNetCoreInstrumentation() // tự động: span cho mỗi request đến
        .AddHttpClientInstrumentation() // tự động: span cho request đi + gắn header
        .AddSource(DatHangTelemetry.TenNguon) // nguồn span TỰ VIẾT
        .AddConsoleExporter() // demo. Thực tế: .AddOtlpExporter()
    ).WithMetrics(m => m
        .AddAspNetCoreInstrumentation()
        .AddMeter(DatHangTelemetry.TenNguon)
```

Quy ước: tên span mô tả **thao tác** (không chứa giá trị thay đổi như id — dùng `SetTag`); tag theo **semantic conventions** của OTEL khi có (`http.request.method`, `db.system`). **ĐỪNG ghi dữ liệu nhạy cảm** vào tag/log (mật

khẩu, token, thông tin cá nhân). Đăng ký `AddSource("Kho.DatHang")` — nếu quên, span tự viết không được xuất.

Metrics

```
private readonly Meter _meter = new("Kho.DatHang");
DonHang = _meter.CreateCounter<long>("don_hang_dat", unit: "{don}", description: "So don hang da xu ly");
ThoiGian = _meter.CreateHistogram<double>("thoi_gian_dat_hang", unit: "ms");

tele.DonHang.Add(1, new KeyValuePair<string, object?>("ket_qua", ketQua));
tele.ThoiGian.Record(dong.Elapsed.TotalMilliseconds, new KeyValuePair<string, object?>("ket_qua", ketQua));
```

Công cụ	Đo gì
Counter<T>	đại lượng chỉ tăng (số đơn, số lỗi)
UpDownCounter<T>	tăng/giảm (số kết nối đang mở)
Histogram<T>	phân phối (độ trễ) → tính p50/p95/p99
ObservableGauge<T>	giá trị tức thời đọc khi cần (dung lượng hàng đợi, bộ nhớ)

Nhãn (tag) cho phép cắt lát (`ket_qua=that_bai`) nhưng **độ đa dạng (cardinality) phải thấp**: tuyệt đối không gắn `UserId` / `OrderId` làm nhãn metric — hàng triệu tổ hợp làm sập hệ thống lưu trữ. Dùng trace/log cho dữ liệu chi tiết từng đơn.

[ASP.NET](#) Core sẵn có metric hạ tầng (`http.server.request.duration` , số kết nối, `kestrel.*`) và runtime (`dotnet.gc.*` , `dotnet.thread_pool.*`). Bốn "tín hiệu vàng" (Google SRE) nên theo dõi cho mỗi dịch vụ: **độ trễ (latency)**, **lưu lượng (traffic)**, **lỗi (errors)**, **độ bão hoà (saturation)**.

Logs kết nối với trace

Khi `ILogger` đi qua OTel, mỗi bản ghi log **tự mang `TraceId` và `SpanId`** của span hiện tại. Trong công cụ (Grafana/Kibana/App Insights) bạn nhảy từ một dòng log lỗi tới **toàn bộ trace** chứa nó và ngược lại. Kết hợp với **structured logging** (tham số đặt tên — Tập 1, Chương 39) để lọc theo `{Ma}` , `{KetQua}` . Trả `traceId` trong `ProblemDetails` (Tập 2, Chương 9) để người dùng báo lại và bạn tra ngay.

Chính sách: mức log theo môi trường; log **sự kiện nghiệp vụ quan trọng** và **lỗi**; lấy mẫu (sampling) trace khi lưu lượng lớn (ví dụ 5–10%, nhưng giữ **100% trace lỗi/chậm** — *tail-based sampling* ở Collector).

Health checks: sống hay sẵn sàng?

```
builder.Services.AddHealthChecks()
    .AddCheck("tien-trinh", () => HealthCheckResult.Healthy(), tags: ["live"])
    .AddCheck("csdl", () => HealthCheckResult.Healthy("Ket noi CSDL on"), tags: ["ready"])
    .AddCheck("hang-doi", () => HealthCheckResult.Degraded("Hang doi dang day"), tags: ["ready"]);

app.MapHealthChecks("/health/live", new() { Predicate = c => c.Tags.Contains("live") });
app.MapHealthChecks("/health/ready", new() { Predicate = c => c.Tags.Contains("ready") });
```

	Liveness (/health/live)	Readiness (/health/ready)
Câu hỏi	tiến trình còn sống (không treo)?	sẵn sàng nhận tải (phụ thuộc ổn)?
Kiểm tra	tối thiểu, không chậm phụ thuộc	CSDL, cache, dịch vụ cần thiết
Nếu fail	orchestrator khởi động lại container	load balancer ngừng gửi lưu lượng (không restart)

Sai lầm kinh điển: gán kiểm tra CSDL vào liveness → CSDL chậm ⇒ orchestrator **restart hàng loạt** dịch vụ khoẻ mạnh, làm sự cố tệ hơn. Kết quả có 3 mức: `Healthy` (200), `Degraded` (200 — chạy được nhưng cảnh báo), `Unhealthy` (503). Thư viện `AspNetCore.HealthChecks.*` có sẵn kiểm tra SQL Server, PostgreSQL, Redis, RabbitMQ...

Hiệu năng: quy trình

"Premature optimization is the root of all evil" — nhưng đo trước thì không.

1. **Xác định mục tiêu** (SLO): "p95 < 300 ms ở 200 req/s".
2. **Đo ở điều kiện gần thực tế** (dữ liệu và tải thật): công cụ tải **k6**, **NBomber**, `wrk`, `bombardier`.
3. **Tìm nút thắt** bằng trace/metric: thời gian đi vào **CSDL?** **mạng ngoài?** **CPU?** **khoá?** **GC?** Đừng đoán.
4. **Sửa một thứ**, đo lại, so sánh. Lặp lại.
5. Dừng khi đạt mục tiêu.

Công cụ chẩn đoán .NET: `dotnet-counters` (metric trực tiếp), `dotnet-trace` (profile CPU), `dotnet-dump` / `dotnet-gcdump` (bộ nhớ), **BenchmarkDotNet** (vi mô: so sánh hai cài đặt một hàm), profiler của Visual Studio/Rider/PerfView.

Danh sách kiểm tra hiệu năng (**ASP.NET** Core + EF Core)

Truy cập dữ liệu (thường là nút thắt số 1):

- Số câu SQL mỗi request là hằng số nhỏ? (không **N+1** — Tập 2, Chương 13)
- Truy vấn đọc dùng **projection** + `AsNoTracking`; chỉ nạp cột cần; **phân trang** có giới hạn.
- Có **chỉ mục** phù hợp; đọc `EXPLAIN` / kế hoạch thực thi; tránh hàm trên cột trong `WHERE` (làm mất chỉ mục).
- **Cập nhật hàng loạt** bằng `ExecuteUpdate/Delete`; **batch** `SaveChanges` (EF tự gom); tránh `SaveChanges` trong vòng lặp.
- Khoá/giao dịch **ngắn**; tránh giữ giao dịch mở khi gọi mạng.
- **Compiled queries** cho truy vấn nóng; pooling `AddDbContextPool`.

Ứng dụng:

- **Async đến tận cùng**; không chặn luồng (`.Result`); không tạo `Task.Run` thừa (Tập 1, Chương 33).
- **Cache** dữ liệu đọc nhiều (Chương 10); nén response (`UseResponseCompression` , Brotli).
- `HttpClient` qua factory; giới hạn **kích thước response**, phân trang API; `System.Text.Json` **source generator** giảm chi phí.
- Tránh cấp phát thừa trong đường nóng (`Span<T>` , `ArrayPool<T>` , `StringBuilder`); nhưng **đo trước** khi làm phức tạp code.
- **Thread pool starvation**: nhiều request chặn luồng chờ I/O → dùng async; theo dõi `dotnet.thread_pool.queue.length`.
- **Native AOT/ReadyToRun** cho giảm thời gian khởi động (serverless, container mở rộng nhanh).

Hạ tầng: HTTP/2/3, reverse proxy nén và giữ kết nối, CDN cho tài nguyên tĩnh, đặt ứng dụng gần CSDL (độ trễ mạng), kích thước pool kết nối CSDL phù hợp số instance.

Cảnh báo và runbook

Telemetry vô nghĩa nếu không ai nhìn. **Cảnh báo (alert)** nên bắt **triệu chứng ảnh hưởng người dùng** (tỉ lệ lỗi 5xx > 2% trong 5 phút; p95 > mục tiêu), không phải mọi biến động CPU. Mỗi cảnh báo có **runbook**: kiểm tra gì, ai xử lý, cách khắc phục. Sau sự cố làm **post-mortem không đổ lỗi**, sửa gốc rễ và thêm telemetry/cảnh báo còn thiếu.

Lỗi thường gặp

- Quên `AddSource / AddMeter` cho nguồn tự viết → span/metric không xuất hiện.
- `StartActivity` trả `null` (chưa có listener) mà gọi `.SetTag` không dùng `?.`
- Nhãn metric có cardinality cao (`UserId` , đường dẫn với id).
- Log/tag chứa dữ liệu nhạy cảm; log ở mức `Debug` trong production.
- Gắn kiểm tra phụ thuộc vào **liveness** → restart dây chuyền.
- Ghi telemetry đồng bộ chặn request; không cấu hình sampling khi lưu lượng lớn.
- Tối ưu theo cảm tính, không đo; hoặc đo trên máy dev với dữ liệu nhỏ.
- Cảnh báo quá nhiều ("mù cảnh báo") hoặc không có runbook.

Bài tập

1. Thêm `UpDownCounter` `"so_don_dang_xu_ly"` (tăng khi vào, giảm khi ra) vào luồng đặt hàng và quan sát khi bắn tải song song.
2. Cấu hình `AddOtlpExporter()` và chạy **.NET Aspire dashboard** hoặc Jaeger (Docker) để xem trace dạng đồ thị.
3. Viết `DelegatingHandler` thêm header `X-Correlation-Id = Activity.Current.TraceId` cho `HttpClient` (và giải thích vì sao `traceparent` thường đã đủ).
4. Dùng k6 bắn 200 req/s vào `/dat-hang` và đọc histogram `thoi_gian_dat_hang` ; tính p95.
5. Thêm health check "ready" thật kiểm tra kết nối SQLite của `kho-clean` (dùng `CanConnectAsync`) và một check `Degraded` khi outbox chờ > 100 dòng.

Chương 12 — OpenID Connect và OAuth 2.0

Mục tiêu học

Sau chương này, bạn sẽ:

- Phân biệt **xác thực (authentication)** với **ủy quyền (authorization)** và biết **OAuth 2.0** và **OpenID Connect (OIDC)** giải quyết phần nào.
- Nắm các vai trò (client, resource owner, authorization server, resource server) và **luồng Authorization Code + PKCE** — luồng chuẩn hiện nay.
- Biến API [ASP.NET Core](#) thành **resource server** kiểm tra JWT do IdP cấp: issuer, audience, chữ ký, hạn dùng; phân quyền theo **scope** và **role**.
- Biết chọn: tự xây đăng nhập hay dùng **IdP** (Keycloak, Microsoft Entra ID, Auth0, Duende IdentityServer...).

Code: [code/ch12-oidc/](#) — resource server chạy được cả khi không có IdP (chế độ demo) và với IdP thật qua cấu hình.

Lưu ý trung thực: ví dụ chạy với "IdP giả" tự ký token để bạn thử được ngay. Phần tích hợp với IdP thật (Keycloak/Entra/Auth0) chỉ khác ở **cấu hình Authority / Audience** — chương này chỉ ra cách làm, nhưng máy tác giả không có IdP thật để chạy thử end-to-end; hãy kiểm tra theo tài liệu của IdP bạn dùng.

Vì sao không tự viết đăng nhập?

Ở Tập 2 (Chương 20) bạn tự làm đăng nhập bằng cookie và JWT. Với một ứng dụng đơn, điều đó ổn. Nhưng khi có **nhiều ứng dụng/dịch vụ**, mỗi nơi tự lưu mật khẩu là thảm họa: nhiều nơi rò rỉ, khó thêm MFA, đăng nhập một lần (SSO) bất khả thi. Giải pháp: **tập trung xác thực vào một dịch vụ chuyên trách (Identity Provider — IdP)**; các ứng dụng chỉ **tin token** do IdP cấp. Đó là điều OAuth 2.0/OIDC chuẩn hoá.

OAuth 2.0 và OIDC: hai lớp, hai câu hỏi

	Trả lời	Cho ra
OAuth 2.0	"Ứng dụng này được phép làm gì thay mặt người dùng?" (ủy quyền, ủy thác quyền truy cập)	access token (kèm scope)
OpenID Connect (OIDC, lớp trên OAuth 2.0)	"Người dùng là ai? " (xác thực)	ID token (JWT chứa sub, name, email...) + endpoint userinfo

Sai lầm kinh điển: dùng **access token** để chứng minh "người này đã đăng nhập" ở phía client. Access token dành cho **API** (audience là API); danh tính cho ứng dụng nằm ở **ID token**.

Các vai trò

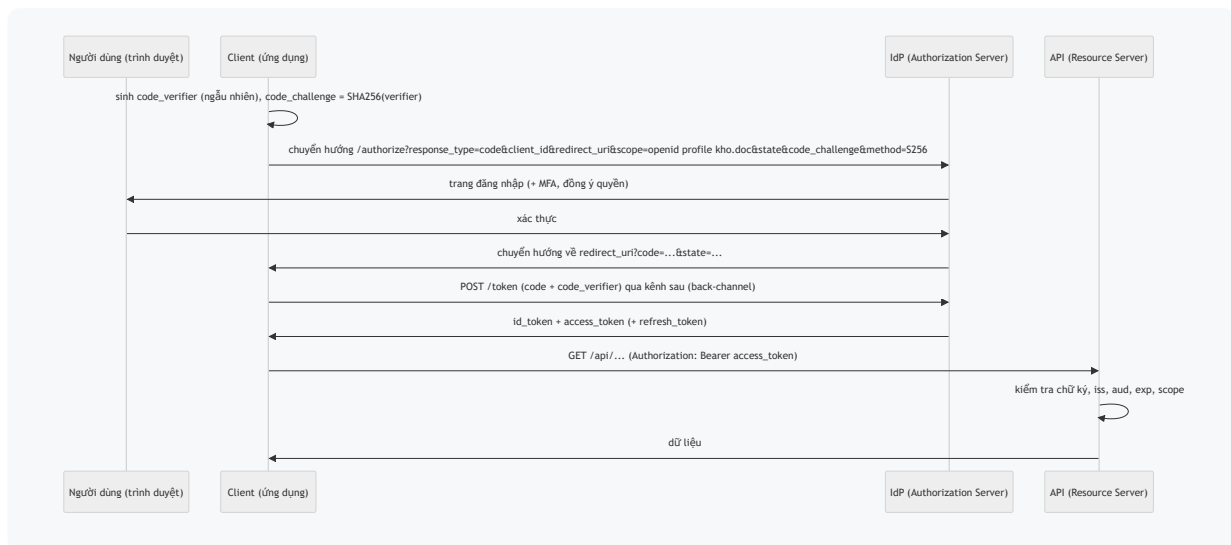
- **Resource Owner:** người dùng.
- **Client:** ứng dụng muốn truy cập (SPA, Blazor, mobile, web MVC, hay một dịch vụ khác).
- **Authorization Server / IdP:** xác thực người dùng, cấp token (Keycloak, Entra ID...).
- **Resource Server:** API bảo vệ tài nguyên, **kiểm tra access token** (chính là API của bạn).

Loại client

- **Confidential** (giữ được bí mật: ứng dụng web server-side) — có `client_secret`.
- **Public** (không giữ được bí mật: SPA, mobile, desktop) — **không** có secret; bắt buộc **PKCE**.

Luồng Authorization Code + PKCE

Đây là luồng khuyến nghị cho mọi client tương tác (OAuth 2.1 gộp PKCE vào bắt buộc). *Implicit* và *Resource Owner Password* đã **bị loại bỏ** — đừng dùng.



Vì sao an toàn?

- **code** ngắn hạn, một lần, đi qua trình duyệt (có thể bị đánh cắp), nhưng **vô dụng nếu thiếu code_verifier** — bí mật chỉ client giữ và gửi qua kênh trực tiếp tới IdP. Đó là **PKCE** (Proof Key for Code Exchange): chặn tấn công *chặn authorization code*.
- **state** ngẫu nhiên chống CSRF trên bước chuyển hướng; **nonce** (OIDC) gắn ID token với phiên, chống replay.
- **redirect_uri** phải khớp **chính xác** danh sách đăng ký trước — chống chuyển hướng token về kẻ tấn công.

Tính PKCE chỉ là băm — code mẫu có endpoint `/pkce` cho đúng vector kiểm thử của RFC 7636: `verifier = dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWF0EjXk` → `challenge = E9MeIhoa20wvFrEMTJguCHaoeK1t8URWbuGJSstw-cM` (đã chạy và khớp).

```
string verifier = Base64Url(RandomNumberGenerator.GetBytes(32));
string challenge = Base64Url(SHA256.HashData(Encoding.ASCII.GetBytes(verifier)));
static string Base64Url(byte[] b) => Convert.ToBase64String(b).TrimEnd('=').Replace('+', '-').Replace('/', '_');
```

Trong thực tế bạn **không tự cài** luồng này: middleware `AddOpenIdConnect` (web server-side) hoặc thư viện của SPA (`oidc-client-ts`, MSAL) làm hộ. Điều bạn cần hiểu là *nó làm gì để cấu hình và gỡ lỗi đúng*.

Các loại token

	Access token	ID token	Refresh token
Dành cho	API (resource server)	client (biết người dùng là ai)	IdP (đổi lấy token mới)
Thời hạn	ngắn (5–60 phút)	ngắn	dài (ngày–tuần), có thể xoay vòng
Định dạng	thường JWT (hoặc opaque)	luôn JWT	opaque
Lưu ở	bộ nhớ client	—	nơi an toàn nhất có thể (BFF/cookie httpOnly)

Mẫu **BFF (Backend-for-Frontend)**: với SPA, đặt một backend [ASP.NET Core](#) giữ token và trao cho trình duyệt **cookie httpOnly**, thay vì để token trong JavaScript (`localStorage` bị đánh cắp qua XSS). Blazor Server và MVC vốn đã hoạt động như vậy.

API là Resource Server

Nhiệm vụ của API rất hẹp: **với mỗi request, quyết định tin token hay không**, rồi dùng claim để phân quyền.

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.Authority = "https://login.example.com/realms/kho"; // IdP: tự tải discovery (.well-known,
        o.Audience = "kho-api"; // token phải DÀNH CHO API này
        o.MapInboundClaims = false; // giữ tên claim gốc: "sub", "scope"
    });
```

Authority làm phần khó: tải **JWKS** (bộ khoá công khai) từ IdP, **cache và tự cập nhật khi IdP xoay khoá**. Với mỗi token, thư viện kiểm tra:

- 1. **Chữ ký** đúng khoá của IdP (không thể giả mạo nếu không có khoá riêng).
- 2. **iss** (issuer) là IdP bạn tin.
- 3. **aud** (audience) là API của bạn — chặn dùng token cấp cho API khác.
- 4. **exp / nbf** : còn hạn (có `clockSkew` để bù lệch đồng hồ; mặc định 5 phút, nên giảm).

Chạy code mẫu ở chế độ demo (IdP giả ký bằng khoá đối xứng nhưng vẫn kiểm tra đủ issuer, audience, hạn), kết quả thật:

Tình huống	Kết quả
Không có token	401
Token scope=kho.doc gọi GET	200
Token kho.doc gọi POST (cần kho.ghi)	403
Token kho.doc kho.ghi gọi POST	201
DELETE không có role quan-tri	403
DELETE với role quan-tri	204
Token hết hạn	401
Token bị sửa chữ ký	401

401 = chưa xác thực được (không/sai token); **403** = biết bạn là ai nhưng không đủ quyền. Đừng nhầm hai mã này.

Scope và role

- **Scope:** quyền **ứng dụng client** được người dùng/IdP uỷ quyền ("ứng dụng này được đọc kho"). Nằm trong claim `scope` (chuỗi cách nhau dấu cách).
- **Role/nhóm: vai trò của người dùng** trong tổ chức ("quản trị"). Nằm trong claim `roles / groups`, cấu hình ở IdP.

Quyền thực sự = **cả hai**: client phải có scope và người dùng phải có vai trò. Trong code:

```
builder.Services.AddAuthorizationBuilder()
    .AddPolicy("DocKho", p => p.RequireAuthenticatedUser().RequireAssertion(c => CoScope(c.User, "kho")
    .AddPolicy("QuanTri", p => p.RequireRole("quan-tri"));

app.MapGet("/api/san-pham", ...).RequireAuthorization("DocKho");
```

Với dữ liệu theo từng người (ví dụ "chỉ xem đơn của mình") dùng `sub` làm khoá chủ sở hữu và kiểm tra **resource-based authorization** (Tập 2, Chương 20). Scope/role không thay thế được kiểm tra "đối tượng này của ai".

Ứng dụng web đăng nhập bằng OIDC

Với MVC/Razor Pages/Blazor Server, dùng cookie cho phiên cục bộ và OIDC cho đăng nhập:

```
builder.Services.AddAuthentication(o =>
{
    o.DefaultScheme = CookieAuthenticationDefaults.AuthenticationScheme;
    o.DefaultChallengeScheme = OpenIdConnectDefaults.AuthenticationScheme; // chưa đăng nhập -> challenge
})
.AddCookie()
.AddOpenIdConnect(o =>
{
    o.Authority = builder.Configuration["Oidc:Authority"];
    o.ClientId = builder.Configuration["Oidc:ClientId"];
    o.ClientSecret = builder.Configuration["Oidc:ClientSecret"]; // từ user-secrets / b...
    o.ResponseType = "code"; // Authorization Code flow
    o.SaveTokens = true; // lưu token vào cookie
    o.Scope.Add("kho.doc");
    o.GetClaimsFromUserInfoEndpoint = true;
});
```

Web app rồi gọi API bằng `HttpClient` gắn access token (`GetTokenAsync("access_token")`), hoặc dùng thư viện quản lý làm mới token (`Duende.AccessTokenManagement` , `Microsoft.Identity.Web`).

Giữa các dịch vụ: client credentials

Dịch vụ gọi dịch vụ **không có người dùng**: dùng luồng **Client Credentials** — dịch vụ đổi `client_id` + `client_secret` (hoặc chứng chỉ/managed identity) lấy access token với scope máy-với-máy. Trên đám mây ưu tiên **managed identity** (Azure) / **workload identity** (Kubernetes) để không phải quản lý secret.

Chọn giải pháp

Lựa chọn	Khi nào
Dịch vụ đám mây (Microsoft Entra ID / External ID, Auth0, Okta, Cognito...)	ít vận hành, cần MFA/SSO/nhà cung cấp xã hội, sẵn sàng trả phí
Keycloak (mã nguồn mở, tự chạy)	tự chủ dữ liệu, muốn miễn phí bản quyền, có năng lực vận hành
Duende IdentityServer / OpenIddict (framework .NET)	cần tùy biến sâu, đội .NET mạnh (Duide có giấy phép thương mại)
ASP.NET Core Identity riêng	ứng dụng đơn, không cần liên kết nhiều hệ thống

Nguyên tắc: **đừng tự phát minh giao thức xác thực**, và đừng tự lưu/băm mật khẩu nếu có thể uỷ cho IdP.

Lỗi thường gặp

- Dùng luồng **Implicit/Password**; SPA lưu token trong `localStorage`.
- Không kiểm tra **audience** → token của API khác dùng được.
- Không đặt `clockSkew` hợp lý; token sống hàng ngày; không xoay refresh token.
- Quên PKCE cho public client; `redirect_uri` dùng ký tự đại diện.
- `MapInboundClaims` mặc định đổi tên claim (`sub` → URI dài) khiến `FindFirst("sub")` trả `null` — đặt `false` hoặc dùng đúng tên.
- Chấp nhận token chỉ dựa vào **chữ ký** mà bỏ qua `iss` / `aud`; tắt `ValidateIssuer` "cho nhanh".
- Nhầm **401 với 403**; phân quyền chỉ bằng role mà bỏ qua kiểm tra quyền sở hữu.
- Commit `client_secret`; log ra token (token là mật khẩu tạm thời).

Bài tập

- Chạy `ch12-oidc`, lấy token `kho.doc` rồi giải mã phần payload (base64url) để xem `iss`, `aud`, `exp`, `scope`. Dùng `jwt.io` chỉ với **token demo**, không bao giờ dán token thật.
- Thêm policy `"DonCuaToi"`: endpoint `GET /api/don/{maNguoiDung}` chỉ cho phép khi `sub` khớp (hoặc role `quan-tri`), viết bằng `IAuthorizationHandler`.
- Chạy **Keycloak** bằng Docker (`quay.io/keycloak/keycloak`), tạo realm/client/scope `kho.doc`, đặt `oidc:Authority` và lấy token thật từ endpoint `/token`; gọi API. (Cần Docker — chương này chưa chạy được ở máy tác giả.)
- Thêm test tích hợp cho `kho-clean` dùng token demo (giữ `WebApplicationFactory` cấp token cục bộ) để kiểm tra 401/403/200.
- Vẽ luồng Client Credentials của một dịch vụ báo cáo gọi API kho bằng sơ đồ tuần tự.

Chương 13 — Idempotency, audit log và bảo vệ dữ liệu

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **idempotency** (tính lặp an toàn) và vì sao mạng không đáng tin buộc API ghi phải chịu được **gửi lặp**.
- Cài **Idempotency-Key** đúng cách: chiếm khoá bằng **một lệnh INSERT nguyên tử**, lưu và **phát lại** kết quả, xử lý đồng thời và nội dung khác.
- Ghi **nhật ký kiểm toán (audit log)**: ai, làm gì, lúc nào, giá trị trước/sau — **cùng giao dịch** với thay đổi nghiệp vụ.
- Áp dụng nguyên tắc **bảo vệ dữ liệu**: dữ liệu cá nhân (PII), mã hoá, quản lý bí mật, giữ/xoá dữ liệu.

Code: [code/kho-clean/](#) — Kho.Infrastructure/Idempotency/ , Kho.Web/IdempotencyMiddleware.cs , bảng `nhat_ky_kiem_toan` và lớp `IdempotencyAuditTests` (5 test mới; toàn solution **45 test đều đạt**).

Vấn đề: "đã trừ kho chưa?"

Khách bấm "Xuất kho 3 cái". Request tới server, server trừ tồn, nhưng **phản hồi bị mất** (rớt mạng, timeout, proxy). Client không biết thành công hay không, và thử lại. Nếu API không được thiết kế cho việc này, tồn bị trừ **hai lần**. Retry còn đến từ tầng hạ tầng (Polly ở Tập 2 Chương 19, message broker giao "ít nhất một lần"), người dùng bấm đúp, trình duyệt gửi lại.

Idempotent nghĩa là: thực hiện một thao tác **nhiều lần cho cùng kết quả như một lần**.

Phương thức HTTP	Idempotent theo đặc tả?	Ghi chú
GET, HEAD, OPTIONS	có (an toàn)	chỉ đọc
PUT, DELETE	có	"đặt thành X", "xoá X" — lặp lại vẫn ra cùng trạng thái
POST	không	"tạo mới", "xuất 3 cái" — mỗi lần một hiệu ứng
PATCH	tuỳ	tăng tương đối (+3) không idempotent

Thiết kế ưu tiên nhất là làm thao tác **idempotent tự nhiên**: `PUT /san-pham/LT001/ton {"ton": 7}` (đặt tuyệt đối) thay vì `POST .../xuat {"soLuong": 3}` (tương đối). Nhưng nhiều nghiệp vụ (thanh toán, xuất kho, đặt hàng) vốn là "tương đối" — với chúng, dùng **khóa idempotency**.

Idempotency-Key

Quy ước (Stripe phổ biến, có bản nháp IETF): client sinh một **khóa duy nhất cho mỗi ý định** (thường GUID) và gửi ở header. Server nhớ khóa → kết quả:

```
POST /api/san-pham/LT001/xuat
Idempotency-Key: 7d2c1a9e-7b0e-4e0e-9a51-2b0f6c1d0a11
Content-Type: application/json

{"soLuong": 3}
```

- Lần đầu: xử lý, lưu kết quả, trả `204` .

- Gửi lại **cùng khoá, cùng nội dung: không xử lý lại**, trả **y hệt** kết quả cũ (kèm `Idempotency-Replayed: true`).
- **Cùng khoá, nội dung khác:** lỗi lập trình phía client → 422.
- Cùng khoá, bản đầu **đang chạy:** 409 (client thử lại sau).

Client sinh khoá **một lần cho một hành động** rồi dùng lại nó cho mọi lần thử lại của hành động đó; khoá mới cho hành động mới.

Cài đặt: để CSDL phân xử

Sai lầm phổ biến: "đọc xem khoá có chưa; chưa có thì xử lý rồi ghi" — hai request đến cùng lúc **cùng thấy "chưa có"** và cùng xử lý (race condition, giống Tập 2 Chương 22). Cách đúng: **chiếm khoá bằng một lệnh nguyên tử** và để **ràng buộc duy nhất của CSDL** chọn người thắng.

```
// bảng idempotency: khoá chính = Khoa
int them = await db.Database.ExecuteSqlInterpolatedAsync(
    $"INSERT OR IGNORE INTO idempotency (Khoa, DauVan, MaTrangThai, TaoLucMs) VALUES ({khoa}, {dauVan}, 0, 0);";
if (them == 1) return ChiemDuoc; // mình là người đầu tiên
// ngược lại: đã có -> đọc bản ghi để quyết định
var cu = await db.Idempotency.AsNoTracking().Where(x => x.Khoa == khoa).Select(...).FirstAsync(ct);
if (cu.DauVan != dauVan) return KhacNoiDung; // cùng khoá, yêu cầu khác
if (cu.MaTrangThai == 0) return DangXuLy; // bản kia chưa xong
return DaXong(cu.MaTrangThai, cu.NoiDung); // phát lại
```

(`INSERT OR IGNORE` là cú pháp SQLite; PostgreSQL dùng `INSERT ... ON CONFLICT DO NOTHING`, SQL Server dùng bắt lỗi vi phạm khoá duy nhất. Nguyên tắc y hệt.)

Dấu vân tay `DauVan` = SHA-256 của `phương thức + đường dẫn + nội dung` để phát hiện "cùng khoá, khác nội dung".

Middleware bao quanh endpoint:

```
var kq = await store.ChiemAsync(khoa, dauVan, ct);
switch (kq.TrangThai) { /* KhacNoiDung -> 422; DangXuLy -> 409; DaXongTraLai -> ghi lại status + body

// Chiếm được -> xử lý thật, "bắt" response vào bộ đệm để lưu lại
ctx.Response.Body = dem;
await next(ctx);
if (ctx.Response.StatusCode >= 500) await store.NhaAsync(khoa, ...); // lỗi tạm thời: thả khoá để c
else await store.LuuKetQuaAsync(khoa, ctx.Response.StatusCode, body, contentType, ...);
```

Điểm tinh tế cần nhớ:

- **Lỗi 5xx** không được "đóng đinh" kết quả: thả khoá (xóa) để lần thử lại được xử lý. Lỗi nghiệp vụ 4xx (ví dụ hết hàng 422) *có thể* lưu — cùng khoá cho cùng đáp án.
- Khoá có **hạn** (ví dụ 24–48 giờ) và job dọn dẹp bảng — không để bảng phình vô hạn.
- **Phạm vi khoá theo người dùng/tenant** (ghép `sub` vào khoá) để người này không "va" khoá người kia hoặc dò kết quả người khác.
- Thực sự an toàn khi tác dụng phụ và bản ghi khoá **cùng một giao dịch**. Ví dụ này lưu kết quả sau khi handler commit; nếu process chết giữa hai bước, khoá vẫn ở trạng thái "đang xử lý" (`MaTrangThai = 0`) dù

việc đã xong — hệ thống thanh toán nghiêm túc đặt bản ghi khoá vào **cùng transaction** với nghiệp vụ hoặc có bước đối soát/hết hạn khoá đang chạy dở. Hãy nêu rõ giới hạn này khi chọn thiết kế.

- Trên cụm nhiều instance phải dùng **CSDL chung** (như trên) — bộ nhớ cục bộ không đủ.

Kết quả kiểm thử (đã chạy)

Test	Điều được chứng minh
GuiLaiCungKhoa_ChiXuLyMotLan...	gửi hai lần cùng khoá → tồn chỉ trừ một lần (10 → 7), lần hai có Idempotency-Replayed
KhongCoKhoa_MoiLanDeuXuLy	không header → hành vi cũ (10 → 4)
CungKhoaNhungNoiDungKhac_422	cùng khoá, số lượng khác → 422
HaiRequestCungKhoaDongThoi...	8 request song song cùng khoá → chỉ một xử lý, tồn 10 → 8

Consumer idempotent (thông điệp "ít nhất một lần")

Outbox (Chương 8) và broker giao thông điệp **ít nhất một lần** — nghĩa là **có thể trùng**. Bên nhận phải xử lý lặp an toàn: lưu `MessageId` đã xử lý (bảng *inbox*) trong **cùng giao dịch** với tác dụng phụ, hoặc thiết kế thao tác tự nhiên idempotent (upsert theo khoá). Cùng nguyên lý với `Idempotency-Key` : **khóa duy nhất + ràng buộc CSDL**.

Nhật ký kiểm toán (audit log)

Audit log trả lời "ai đã làm gì, khi nào, trước/sau ra sao" — phục vụ điều tra sự cố, tuân thủ (tài chính, y tế), giải quyết tranh chấp. Khác **log ứng dụng** (chẩn đoán, có thể xoay vòng/xoá): audit là **bản ghi nghiệp vụ, chỉ thêm, giữ lâu**.

Ở `kho-clean` , `KhoDbContext.LuuAsync` ghi nhật ký **trong cùng một giao dịch** với thay đổi nghiệp vụ:

```
foreach (var e in ChangeTracker.Entries<SanPham>()).Where(e => e.State is EntityState.Added or EntityState.Modified)
{
    var doi = e.State == EntityState.Modified ? e.Properties.Where(p => p.IsModified).ToList() : [];
    NhatKy.Add(new NhatKyKiemToan
    {
        Luc = luc, Nguai = NguaiThucHien,
        HanhDong = e.State == EntityState.Added ? "Them" : "Sua",
        DoiTuong = $"SanPham:{e.Entity.Ma.GiaTri}",
        Truoc = doi.Count == 0 ? null : JsonSerializer.Serialize(doi.ToDictionary(p => p.Metadata.Name, v => v.Value)),
        Sau = doi.Count == 0 ? null : JsonSerializer.Serialize(doi.ToDictionary(p => p.Metadata.Name, v => v.Value))
    });
}
```

Vì cùng giao dịch: **nghiệp vụ đổi thành công** ⇔ **có dòng nhật ký** — không bao giờ lệch. Test

`NhatKy_GhiAiLamGi_KemGiaTriTruocSau` xác nhận: an tạo sản phẩm, bình xuất 4 → nhật ký có `Nguai=bình` , `Truoc={"TonKho":"10"} , Sau={"TonKho":"6"} .`

"Ai" đến từ lớp ngoài (`NguaiThucHien` do web đặt từ `User / X-Nguoi` ; thực tế lấy từ claim `sub` của JWT — Chương 12), **không** để Domain biết HTTP.

Nguyên tắc thiết kế audit:

- **Chỉ thêm**, quyền ghi bảng bị hạn chế; không cho ứng dụng `UPDATE/DELETE` (dùng người dùng CSDL khác hoặc bảng *append-only*/immutable storage khi yêu cầu pháp lý).
- Ghi **cả hành động bị từ chối** với thao tác nhạy cảm (đăng nhập thất bại, truy cập trái phép).
- **Đừng ghi bí mật** (mật khẩu, số thẻ) vào giá trị trước/sau; che/loại các cột nhạy cảm.
- Kèm `TraceId` (Chương 11) để nối audit với trace.
- Với hệ thống lớn: **event sourcing** hoặc **temporal tables** (SQL Server) / **trigger** là phương án khác; hoặc phát sự kiện miền qua outbox sang kho audit riêng.
- Tách **audit (nghệp vụ)** khỏi **log kỹ thuật**; đặt chính sách lưu giữ riêng.

Bảo vệ dữ liệu

Phân loại và giảm thiểu

- Xác định dữ liệu **cá nhân/nhạy cảm (PII)**: tên, email, số điện thoại, địa chỉ, ID cá nhân, dữ liệu tài chính/sức khoẻ.
- **Thu thập tối thiểu**: không giữ những gì không cần. Dữ liệu không có thì không thể rò rỉ.
- Tuân thủ quy định áp dụng (Việt Nam: **Nghị định 13/2023/NĐ-CP** về bảo vệ dữ liệu cá nhân; nếu phục vụ người dùng EU: GDPR): cần **mục đích, sự đồng ý, quyền truy cập/xoá**, thông báo khi rò rỉ. Hãy tham vấn pháp chế — sách này không phải tư vấn pháp lý.

Mã hoá

- **Đang truyền**: HTTPS/TLS mọi nơi (kể cả giữa dịch vụ nội bộ và tới CSDL).
- **Khi lưu**: mã hoá đĩa/CSDL do nhà cung cấp; với cột rất nhạy cảm dùng **mã hoá cấp ứng dụng** ([ASP.NET Core Data Protection](#) `IDataProtector.Protect/Unprotect` , hoặc AES-GCM với khoá lưu ở **Key Vault**). Khoá **tách khỏi** dữ liệu và **xoay vòng** được.
- **Mật khẩu**: không bao giờ mã hoá hai chiều — **băm chậm có muối** (`PasswordHasher<T>` của Identity dùng PBKDF2; hoặc Argon2/bcrypt) — Tập 2, Chương 20.
- Đừng tự phát minh thuật toán; dùng API chuẩn (`System.Security.Cryptography`).

Bí mật (secrets)

- **Không bao giờ** commit vào Git: chuỗi kết nối, khoá API, `client_secret` .
- Phát triển: **dotnet user-secrets** . Triển khai: **biến môi trường/Key Vault** (Azure Key Vault, AWS Secrets Manager, HashiCorp Vault)/secret của Kubernetes; dùng **managed identity** để loại bỏ secret khi có thể.
- Quét secret tự động trong CI (ví dụ `gitLeaks` , GitHub secret scanning); xoay vòng ngay khi nghi ngờ lộ.

Log và dữ liệu cá nhân

Không log toàn bộ request/response body, token, mật khẩu, số thẻ. Che (`****1234`) hoặc băm. Phần nhật ký kiểm toán cũng thuộc dữ liệu cần bảo vệ và có thời hạn lưu.

Lưu giữ và xoá

Đặt **thời hạn lưu** cho từng loại dữ liệu và job xoá/ẩn danh hoá. Nếu người dùng yêu cầu xoá: xoá (hoặc ẩn danh) dữ liệu cá nhân nhưng giữ được bản ghi kiểm toán/tài chính bắt buộc theo luật (tách định danh khỏi giao dịch: dùng mã giả danh).

Lỗi thường gặp

- "Kiểm tra rồi ghi" khoá idempotency thay vì **chiếm nguyên tử** → hai request cùng thắng.
- Lưu kết quả lỗi 5xx bằng khoá → client không thể thử lại thành công.
- Không kiểm tra **nội dung** kèm khoá → dùng lại khoá cho yêu cầu khác trả nhầm kết quả.
- Idempotency chỉ ở bộ nhớ tiến trình (không chạy được khi có nhiều instance/restart).
- Audit ghi **ngoài** giao dịch nghiệp vụ → lệch khi một bên lỗi.
- Cho ứng dụng quyền sửa/xoá audit; ghi bí mật vào audit/log.
- Mã hoá mà khoá nằm cạnh dữ liệu (cùng repo/cùng bảng).
- Log token/PII; commit `appsettings` chứa mật khẩu.

Bài tập

1. Thêm `Idempotency-Key` cho endpoint `POST /api/san-pham` (tạo mới) và viết test: hai request cùng khoá tạo đúng một sản phẩm (và không dựa vào ràng buộc trùng mã).
2. Thêm hạn 24 giờ cho khoá và một `BackgroundService` dọn các bản ghi `TaoLucMs` quá hạn; viết test dùng `FakeTimeProvider`.
3. Ghi thêm `TraceId` vào `NhatKyKiemToan` (lấy từ `Activity.Current`).
4. Mở rộng audit cho hành động **bị từ chối** (xuất thiếu hàng 422) — ghi ở đâu là hợp lý (behavior trong pipeline)? Cài thử.
5. Viết `ProtectedString` value converter dùng `IDataProtector` để mã hoá cột `Ghi chú` của sản phẩm; kiểm tra dữ liệu trong SQLite là chuỗi mã hoá.
6. Cấu hình `dotnet user-secrets` cho chuỗi kết nối và đọc ở `Program.cs`; giải thích vì sao không dùng `appsettings.json` cho bí mật.

Chương 14 — Docker và docker-compose

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **container** là gì, khác máy ảo ra sao, và vì sao "chạy được trên máy tôi" không còn là vấn đề.
- Viết **Dockerfile nhiều giai đoạn (multi-stage)** cho ứng dụng [ASP.NET Core](#): nhỏ, cache tốt, chạy không bằng root.
- Cấu hình ứng dụng qua **biến môi trường**, dùng **volume** cho dữ liệu, dùng **docker-compose** để chạy nhiều dịch vụ.
- Biết các thực hành an toàn và các lỗi thường gặp.

Code: [code/kho-clean/Dockerfile](#), [docker-compose.yml](#), [.dockerignore](#).

Lưu ý trung thực về kiểm chứng: khi viết sách, Docker daemon trên máy tác giả không chạy nên **chưa build được image thật**. Những gì đã kiểm chứng: lệnh `dotnet publish -c Release` dùng đúng trong Dockerfile chạy thành công và bản publish khởi động ở môi trường Production (tự migrate CSDL SQLite, /health trả 200, API trả đúng lỗi 400). Nội dung Dockerfile/compose tuân theo tài liệu chính thức của Microsoft/Docker; nếu build ở máy bạn gặp khác biệt (phiên bản image, tên biến), hãy xem `docker build` cho thông báo cụ thể.

Container là gì?

Container đóng gói ứng dụng **cùng mọi thứ nó cần** (runtime, thư viện, cấu hình mặc định) thành một **image** bất biến; chạy image ra **container**. Khác **máy ảo (VM)**: VM ảo hoá cả phần cứng và chạy hệ điều hành đầy đủ (nặng, khởi động phút); container **dùng chung nhân (kernel)** của máy chủ, chỉ cô lập tiến trình/mạng/hệ tệp bằng namespace và cgroup (nhẹ, khởi động giây).

Khái niệm	Ý nghĩa
Image	khuôn bất biến, gồm nhiều lớp (layer) chỉ đọc
Container	một thể hiện đang chạy của image (thêm lớp ghi mỏng, mất khi xoá container)
Registry	kho lưu image (Docker Hub, GitHub Container Registry, Azure Container Registry, ECR)
Volume	thư mục lưu bền ngoài vòng đời container
Tag	nhãn phiên bản (kho-web:1.4.2); latest chỉ là một tag quy ước, không phải "mới nhất" thật

Lợi ích cho .NET: môi trường **giống nhau** ở dev/CI/production, triển khai bằng **thay image** (không cài đặt trên máy chủ), **mở rộng** bằng cách chạy nhiều bản, **quay lui** bằng cách chạy lại tag cũ. Hầu hết nền tảng đám mây (Chương 16) đều chạy container.

Dockerfile nhiều giai đoạn

```
# ===== Giai đoạn 1: BUILD (có SDK, nặng ~1 GB) =====  
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
```

```

WORKDIR /src

# Chép RIÊNG các file csproj trước -> lớp "restore" được cache; chỉ khi đổi package mới restore lại
COPY Kho.Domain/Kho.Domain.csproj Kho.Domain/
COPY Kho.Application/Kho.Application.csproj Kho.Application/
COPY Kho.Infrastructure/Kho.Infrastructure.csproj Kho.Infrastructure/
COPY Kho.Web/Kho.Web.csproj Kho.Web/
RUN dotnet restore Kho.Web/Kho.Web.csproj

COPY . .
RUN dotnet publish Kho.Web/Kho.Web.csproj -c Release -o /app/publish --no-restore /p:UseAppHost=false

# ===== Giai đoạn 2: RUNTIME (chỉ runtime ASP.NET, nhỏ) =====
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
WORKDIR /app
COPY --from=build /app/publish .

RUN mkdir -p /data && chown $APP_UID /data
ENV ConnectionStrings__Kho="Data Source=/data/kho.db" ASPNETCORE_HTTP_PORTS=8080
VOLUME /data

USER $APP_UID
EXPOSE 8080
ENTRYPOINT ["dotnet", "Kho.Web.dll"]

```

Giải thích từng quyết định:

1. **Multi-stage:** SDK (compiler, công cụ) chỉ ở giai đoạn build; image cuối chỉ có **runtime** (`aspnet`), nhỏ hơn nhiều và **ít bề mặt tấn công** (không có compiler cho kẻ xâm nhập).
2. **Cache theo lớp:** Docker cache từng lệnh; nếu đầu vào không đổi, dùng lại. Chép `*.csproj` và `restore` **trước** khi chép mã nguồn: sửa mã (thường xuyên) không làm mất lớp restore (chậm, tải package). Đây là mẹo quan trọng nhất về tốc độ build.
3. `--no-restore` , `UseAppHost=false` : publish nhanh hơn, không cần file thực thi native vì dùng `dotnet` `Kho.Web.dll` .
4. **Không chạy bằng root:** image .NET 8+ có sẵn người dùng `app` (UID trong biến `$APP_UID`); `USER $APP_UID` để nếu ứng dụng bị chiếm quyền, kẻ tấn công không có root trong container. Cấp quyền ghi thư mục dữ liệu cho người dùng đó.
5. **Cổng 8080:** image .NET 8+ mặc định lắng nghe **8080** (không phải 80) vì cổng dưới 1024 cần đặc quyền. `ASPNETCORE_HTTP_PORTS` cấu hình cổng. TLS thường **kết thúc ở reverse proxy/ingress** phía trước, không phải trong container.
6. **ENTRYPOINT dạng mảng (exec form):** tiến trình `dotnet` là PID 1 nhận tín hiệu `SIGTERM` trực tiếp → tắt êm (graceful shutdown, hoàn tất request đang chạy, dừng `BackgroundService` như outbox).

.dockerignore

```

**/bin/
**/obj/
**/*.db
.git

```

Không đưa `bin/obj` (ghi đè kết quả build trong container), cơ sở dữ liệu cục bộ, `.git` vào **ngữ cảnh build** — nhỏ hơn, nhanh hơn, và không rò rỉ dữ liệu/bí mật vào image.

Build và chạy

```
docker build -t kho-web:dev tap3/code/kho-clean
docker run --rm -p 8080:8080 -v kho-data:/data kho-web:dev
curl http://localhost:8080/health
```

Ngoài Dockerfile, .NET SDK còn có thể tạo image **không cần Dockerfile**: `dotnet publish -t:PublishContainer` (gói `Microsoft.NET.Build.Containers`, cấu hình trong csproj). Tiện cho CI; Dockerfile linh hoạt hơn khi cần tùy biến.

Cấu hình bằng biến môi trường

Nguyên tắc **12-factor**: cấu hình theo môi trường **nằm ngoài image**. [ASP.NET](#) Core map biến môi trường tới cấu hình: dấu `:` thành `_` (hai gạch dưới):

Biến	Tương ứng appsettings
ConnectionStrings__Kho	ConnectionStrings:Kho
Outbox__Bat	Outbox:Bat
Logging__LogLevel__Default	Logging:LogLevel:Default
ASPNETCORE_ENVIRONMENT	môi trường (Development/Production)

Một image cho mọi môi trường: build một lần, khác biệt chỉ nằm ở biến môi trường/secret khi chạy. **Không nhúng bí mật vào image** (`ENV PASSWORD=...` nằm mãi trong lịch sử layer); truyền lúc chạy qua secret của nền tảng (Chương 16).

Dữ liệu: container không lưu trạng thái

Container là **tạm thời**: xóa đi là mất lớp ghi. Dữ liệu phải ở **volume** (như `kho-data:/data` cho SQLite) hoặc — chuẩn hơn ở production — **dịch vụ CSDL ngoài** (PostgreSQL/SQL Server, thường là dịch vụ quản lý). SQLite trong volume phù hợp demo và ứng dụng nhỏ **một instance**; khi chạy nhiều bản (mở rộng ngang) bắt buộc chuyển sang CSDL máy chủ.

docker-compose

Compose khai báo cả hệ thống nhiều container trong một file YAML:

```
services:
  kho-web:
    build: .
    ports: ["8080:8080"]
    environment:
      ASPNETCORE_ENVIRONMENT: Production
      ConnectionStrings__Kho: "Data Source=/data/kho.db"
    volumes: [kho-data:/data]
    restart: unless-stopped
```

```
volumes:
  kho-data:
```

`docker compose up --build` dựng và chạy; `docker compose logs -f kho-web`; `docker compose down` (thêm `-v` để xoá cả volume — cẩn thận, mất dữ liệu).

Thêm PostgreSQL (mẫu tham khảo)

Khi chuyển sang PostgreSQL: đổi provider EF (`Npgsql.EntityFrameworkCore.PostgreSQL`), tạo lại migration, và compose có thêm dịch vụ CSDL:

```
services:
  kho-web:
    build: .
    ports: ["8080:8080"]
    environment:
      ConnectionStrings__Kho: "Host=csdl;Database=kho;Username=kho;Password=${KHO_DB_PASSWORD}"
    depends_on:
      csdl:
        condition: service_healthy      # chờ CSDL SẴN SÀNG, không chỉ "đã khởi động"
  csdl:
    image: postgres:17
    environment:
      POSTGRES_DB: kho
      POSTGRES_USER: kho
      POSTGRES_PASSWORD: ${KHO_DB_PASSWORD}
    volumes: [pg-data:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U kho -d kho"]
      interval: 5s
      retries: 10
volumes:
  pg-data:
```

- Các dịch vụ trong cùng compose gọi nhau **bằng tên dịch vụ** (`Host=csdl`) qua mạng riêng.
- Mật khẩu lấy từ biến `${KHO_DB_PASSWORD}` (file `.env` **không commit**, hoặc `docker compose --env-file`).
- Ứng dụng nên **chịu được CSDL chưa sẵn sàng** lúc khởi động (retry), đừng dựa hoàn toàn vào `depends_on` . Tự `MigrateAsync` khi khởi động chỉ hợp với một instance; khi nhiều bản, chạy migration bằng **job riêng** (hoặc `dotnet ef migrations bundle`) trước khi triển khai.

(Đây là mẫu theo tài liệu, **chưa chạy ở máy tác giả**; provider PostgreSQL cũng cần đổi vài chỗ SQL đặc thù SQLite như `INSERT OR IGNORE` ở Chương 13 → `ON CONFLICT DO NOTHING` .)

Health check trong container

`/health` của ứng dụng (Chương 11) là nền tảng để orchestrator biết sống/sẵn sàng. Image `aspnet` mặc định **không có curl** (và bản *chiseled* không có cả shell), nên khai báo `HEALTHCHECK` trong Dockerfile khó; trên Kubernetes/Container Apps dùng **probe HTTP** do nền tảng gọi (không cần công cụ trong container), và trên compose có thể kiểm tra từ dịch vụ khác hoặc dùng image có `curl` .

Thực hành tốt và an toàn

- **Ghim phiên bản:** `dotnet/aspnet:10.0` (hoặc ghim tới patch/digest cho tái lập); tránh `latest`.
- **Image nhỏ/an toàn hơn:** biến thể `-noble-chiseled` (Ubuntu chiseled: không shell, không package manager, chạy sẵn non-root) và `-alpine`; đánh đổi: khó debug (không có shell), musl libc.
- **Quét lỗ hổng** image (Trivy, Docker Scout, Defender) trong CI; cập nhật base image thường xuyên (và bảo mật).
- **Không chạy root**, không mount `docker.sock`, không dùng `--privileged`.
- **Hệ tệp chỉ đọc** (`read_only: true` + volume ghi cho thư mục cần thiết) nếu có thể.
- **Giới hạn tài nguyên** (`mem_limit`, `cpus`); .NET tự nhận giới hạn bộ nhớ container để đặt heap GC.
- **Log ra stdout/stderr** (mặc định của [ASP.NET](#) Core console logger); nền tảng thu gom — đừng ghi file log trong container.
- **Một tiến trình chính mỗi container**; ứng dụng phải **tắt êm** khi nhận SIGTERM.
- Đừng nhét **secret** hay `appsettings.Production.json` chứa bí mật vào image.

Lỗi thường gặp

- Chép toàn bộ mã trước `restore` → mỗi lần sửa một dòng lại tải lại toàn bộ package.
- Thiếu `.dockerignore` → ngửi cảnh build lớn, `bin/obj` cũ làm hỏng build.
- Ứng dụng nghe `localhost` trong container → không truy cập được từ ngoài (dùng `ASPNETCORE_HTTP_PORTS / URLS=http://+:8080`).
- Quên volume → **mất dữ liệu SQLite** khi tạo lại container; hoặc lỗi quyền ghi vì chạy non-root mà thư mục thuộc root.
- Dùng `latest` → build không tái lập được.
- Nhúng secret vào `ENV / ARG / image`; commit `.env`.
- Giả định `depends_on` đảm bảo CSDL sẵn sàng.
- Cổng 80 trong container non-root → lỗi quyền (dùng 8080).
- Chạy migration tự động ở mỗi bản trong cụm → đua nhau migrate.

Bài tập

1. Build image của `kho-clean`, chạy, tạo vài sản phẩm bằng `curl`, **xoá container**, chạy lại cùng volume và xác nhận dữ liệu còn. Chạy lại **không** volume và xác nhận mất.
2. So sánh kích thước image khi dùng `sdk` làm image cuối với multi-stage; ghi lại số liệu bằng `docker images`.
3. Đo tác dụng cache: sửa một dòng mã rồi `docker build` lại; xem lớp `dotnet restore` có `CACHED` không. Sau đó đổi thứ tự (chép hết trước `restore`) và so sánh thời gian.
4. Tạo image không cần Dockerfile bằng `dotnet publish -t:PublishContainer` (kèm `ContainerRepository`, `ContainerFamily` trong `csproj`).
5. Chuyển `kho-clean` sang PostgreSQL theo mẫu compose ở trên, sửa `INSERT OR IGNORE`, tạo lại migration và chạy `docker compose up`.
6. Thêm dịch vụ **OpenTelemetry Collector** hoặc **Jaeger** vào compose và cấu hình `OTEL_EXPORTER_OTLP_ENDPOINT` cho ứng dụng (kết nối Chương 11).

Chương 15 — CI/CD với GitHub Actions

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **CI (tích hợp liên tục)**, **CD (phân phối/triển khai liên tục)** và vì sao chúng giảm rủi ro thay vì tăng công sức.
- Viết workflow **GitHub Actions** build, test, quét và dựng image cho ứng dụng .NET.
- Thiết kế **pipeline** theo giai đoạn: build → test → đóng gói → triển khai staging → duyệt → production; biết **chiến lược triển khai** (rolling, blue-green, canary) và **rollback**.
- Quản lý **secret**, quyền tối thiểu, và tránh các lỗi phổ biến.

Code: workflow thật của repo này ở [.github/workflows/kho-clean.yml](#) .

Lưu ý trung thực: workflow chỉ chạy được trên GitHub, nên tác giả **chưa chạy thật**. Đã kiểm chứng cục bộ đúng các lệnh mà pipeline gọi: `dotnet build -c Release -warnaserror` (0 cảnh báo) và `dotnet test` (45 test đều đạt). Cú pháp YAML tuân theo tài liệu GitHub Actions; lần chạy đầu bạn có thể phải chỉnh nhỏ (đường dẫn, phiên bản action).

CI/CD là gì và để làm gì

- **CI (Continuous Integration)**: mỗi lần push/PR, máy chủ **tự động build và chạy test** trên môi trường sạch. Lỗi được phát hiện trong vài phút sau khi gây ra, khi còn nhớ mình vừa sửa gì — thay vì phát hiện lúc phát hành. "Trên máy tôi chạy được" chấm dứt vì CI là **nguồn sự thật**.
- **Continuous Delivery**: mọi thay đổi qua CI đều **sẵn sàng phát hành** (artifact/image đã tạo, đã kiểm thử); việc bấm "triển khai" có thể là quyết định của con người.
- **Continuous Deployment**: tự động triển khai lên production khi mọi kiểm tra đạt, không cần bước duyệt tay.

Nguyên tắc: **thay đổi nhỏ, thường xuyên, tự động** ít rủi ro hơn phát hành lớn, hiếm, thủ công. Pipeline tốt cho đội **niềm tin** để triển khai nhiều lần mỗi ngày.

Các khái niệm của GitHub Actions

Khái niệm	Ý nghĩa
Workflow	file YAML trong <code>.github/workflows/</code>
Event (on)	điều kiện kích hoạt: <code>push</code> , <code>pull_request</code> , <code>schedule</code> , <code>workflow_dispatch</code> (chạy tay), <code>release</code>
Job	nhóm bước chạy trên một runner ; các job chạy song song trừ khi có <code>needs</code>
Step	lệnh (<code>run</code>) hoặc action (<code>uses</code>) có sẵn
Runner	máy chạy job (<code>ubuntu-latest</code> , <code>windows-latest</code> , hoặc tự dựng — <i>self-hosted</i>)
Artifact	tệp giữ lại từ job (kết quả test, gói build) để job khác/người dùng tải
Secret / Variable	cấu hình mật / không mật, đặt ở repo/môi trường

Khái niệm	Ý nghĩa
Environment	"staging", "production" — có secret riêng và quy tắc duyệt

Workflow CI cho kho-clean

```
name: kho-clean

on:
  push:
    branches: [main]
    paths: ["tap3/code/kho-clean/**", ".github/workflows/kho-clean.yml"]
  pull_request:
    paths: ["tap3/code/kho-clean/**", ".github/workflows/kho-clean.yml"]

permissions:
  contents: read # quyền TỐI THIỂU cho GITHUB_TOKEN

concurrency:
  group: kho-clean-${{ github.ref }}
  cancel-in-progress: true # commit mới → huỷ lần chạy cũ cùng nhánh

defaults:
  run:
    working-directory: tap3/code/kho-clean

jobs:
  build-test:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 10.0.x
      - run: dotnet restore Kho.slnx
      - run: dotnet build Kho.slnx -c Release --no-restore -warnaserror
      - run: dotnet test Kho.slnx -c Release --no-build --logger "trx;LogFileName=keto-qua.trx" --collect:"**/*TestResults.trx"
      - if: always()
        uses: actions/upload-artifact@v4
        with:
          name: keto-qua-test
          path: tap3/code/kho-clean/**/TestResults/**

  docker:
    needs: build-test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/build-push-action@v6
        with:
          context: tap3/code/kho-clean
          push: false
```

```
tags: kho-web:ci
cache-from: type=gha
cache-to: type=gha,mode=max
```

Từng chỗ đáng chú ý:

- **paths** : chỉ chạy khi thư mục liên quan đổi — monorepo (repo này chứa nhiều dự án) khỏi tốn phút CI vô ích.
- **permissions: contents: read** : mặc định token có thể có quyền ghi rộng; **luôn thu hẹp** rồi chỉ cấp thêm cho job cần (ví dụ `packages: write` để đẩy image). Nguyên tắc **đặc quyền tối thiểu**.
- **concurrency + cancel-in-progress** : không lãng phí runner cho commit đã bị đè.
- **timeout-minutes** : job treo không ăn hết phút miễn phí.
- **-warnaserror** : cảnh báo (nullable, deprecated) là **nợ kỹ thuật ngầm**; cho nó làm hỏng build khi dự án còn sạch. Đã xác nhận solution hiện tại build với 0 cảnh báo.
- **--no-build / --no-restore** : không làm lại việc đã làm ở bước trước.
- **if: always()** khi tải artifact: kết quả test cần có **cả khi test đỏ**.
- **needs: build-test** : chỉ dựng image khi test đạt.
- **Ghim phiên bản action** (`@v4`); với an toàn cao hơn ghim theo **commit SHA** (tránh tác giả action đẩy mã độc vào tag).

Cache NuGet

Restore mỗi lần tải lại package. `actions/setup-dotnet` hỗ trợ `cache: true` nhưng yêu cầu **file khoá** `packages.lock.json` (bật `<RestorePackagesWithLockFile>true</RestorePackagesWithLockFile>` trong `csproj/Directory.Build.props`). Khoá phụ thuộc còn giúp **build tái lập được** và phát hiện thay đổi package ngoài ý muốn. Cách khác: `actions/cache` thư mục `~/.nuget/packages` với khoá là hash của các `*.csproj`.

Ma trận (matrix)

Kiểm thử thư viện trên nhiều hệ điều hành/phiên bản .NET:

```
strategy:
  fail-fast: false
  matrix:
    os: [ubuntu-latest, windows-latest]
    dotnet: [10.0.x]
  runs-on: ${ matrix.os }
```

Ứng dụng chạy trong container Linux thì thường chỉ cần `ubuntu-latest`.

Từ CI tới CD: đẩy image và triển khai

Ví dụ phát hành: khi có tag `v*`, dựng và đẩy image lên **GitHub Container Registry (GHCR)**, rồi triển khai.

```
name: phat-hanh
on:
  push:
    tags: ["v*"]

permissions:
  contents: read
```

```

packages: write                                # cần để đẩy vào GHCR

jobs:
  image:
    runs-on: ubuntu-latest
    outputs:
      digest: ${ steps.build.outputs.digest }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }      # token tự cấp, KHÔNG phải mật khẩu cá nhân
      - id: build
        uses: docker/build-push-action@v6
        with:
          context: tap3/code/kho-clean
          push: true
          tags: ghcr.io/${ github.repository }/kho-web:${ github.ref_name }

  staging:
    needs: image
    runs-on: ubuntu-latest
    environment: staging                        # secret + quy tắc riêng của môi trường
    steps:
      - run: echo "Triển khai ${ github.ref_name } lên staging (lệnh cụ thể tùy nền tảng – Chương 10)"

  production:
    needs: staging
    runs-on: ubuntu-latest
    environment: production                    # cấu hình "Required reviewers": CHỜ NGƯỜI
    steps:
      - run: echo "Triển khai lên production"

```

Environment production với "Required reviewers" chèn cổng duyệt tay giữa staging và production — Continuous *Delivery* có kiểm soát. Bỏ cổng đó là Continuous *Deployment*.

Chiến lược triển khai

Chiến lược	Cách làm	Ưu	Nhược
Recreate	tắt bản cũ, bật bản mới	đơn giản	có thời gian ngừng
Rolling	thay dần từng instance	không ngừng, ít tài nguyên thêm	hai phiên bản chạy song song (phải tương thích ngược)
Blue-Green	dựng môi trường mới (green) đầy đủ, chuyển lưu lượng một lần	rollback tức thì (chuyển lại)	tốn gấp đôi tài nguyên
Canary	chuyển 1–5% lưu lượng sang bản mới, quan sát metric, tăng dần	phát hiện lỗi sớm, ảnh hưởng nhỏ	cần quan sát tốt (Chương 11) và định tuyến lưu lượng

Mọi chiến lược "không ngừng" đòi hỏi **thay đổi CSDL tương thích ngược** — mẫu **expand/contract** (mở rộng → triển khai → thu hẹp):

1. *Expand*: thêm cột mới (cho phép null), mã mới ghi cả hai cột, mã cũ vẫn chạy được.
2. Di chuyển dữ liệu; chuyển hẳn sang cột mới.
3. *Contract*: ở lần phát hành sau mới xoá cột cũ.

Đổi tên/xoá cột trong **một** bước là cách chắc chắn nhất để làm hồng rolling deployment.

Rollback: giữ image tag cũ, triển khai lại là quay lui code; **migration** thì khó lùi — thiết kế migration tương thích (expand/contract), sao lưu trước thay đổi lớn, và kiểm tra khôi phục.

Feature flag tách "triển khai mã" khỏi "bật tính năng": mã lên production ở trạng thái tắt, bật dần cho nhóm người dùng, tắt ngay nếu có sự cố (`Microsoft.FeatureManagement`).

Một pipeline tốt gồm những gì

1. **Build + phân tích tĩnh** (cảnh báo là lỗi, `dotnet format --verify-no-changes`, analyzer).
2. **Test nhanh trước** (đơn vị) rồi test chậm (tích hợp); báo cáo độ phủ (đừng coi độ phủ là mục tiêu tự thân).
3. **Quét bảo mật**: `dotnet list package --vulnerable --include-transitive`, quét secret (`gitleaks`), quét image (Trivy), CodeQL, Dependabot cập nhật phụ thuộc.
4. **Dựng artifact một lần** rồi **thăng cấp cùng artifact** qua các môi trường (build once, deploy many) — không build lại cho production.
5. **Triển khai tự động lên staging**, chạy **smoke test** (gọi `/health/ready`, vài endpoint chính) sau triển khai.
6. **Cổng duyệt** trước production; theo dõi metric sau triển khai (Chương 11) và **tự rollback** nếu tỉ lệ lỗi tăng.
7. **Nhanh**: mục tiêu CI dưới ~10 phút; chậm sẽ bị né tránh.

Migration trong pipeline

Chạy `dotnet ef migrations bundle` (tạo tệp thực thi tự chứa) như **bước riêng, một lần**, trước khi khởi động các instance mới; **không** để mọi instance cùng tự `MigrateAsync` khi cụm chạy nhiều bản.

Secret và bảo mật pipeline

- Đặt secret ở **Settings** → **Secrets** (theo repo/environment), truy cập bằng `${{ secrets.TEN }}`; log tự che giá trị nhưng **đừng in/ echo** chúng và đừng truyền qua tham số dòng lệnh dễ lộ.
- **PR từ fork** không nhận secret (mặc định) — đúng vì mã lạ không được đọc secret; cẩn trọng với `pull_request_target`.
- Ưu tiên **OIDC liên kết (workload identity federation)** với đám mây (Azure/AWS/GCP): job xin token ngắn hạn từ GitHub, **không cần lưu khoá dài hạn** trong secret (`permissions: id-token: write`).
- **Tin cậy có chọn lọc** vào action bên thứ ba; ghim SHA; bật Dependabot cho chính các action.
- Runner tự dựng (self-hosted) cho repo **công khai** rất nguy hiểm.

Các nền tảng khác

Khái niệm y hệt ở **Azure DevOps Pipelines**, **GitLab CI**, **Jenkins**, **TeamCity**. Học một nền tảng, chuyển sang nền khác chủ yếu là học cú pháp.

Lỗi thường gặp

- Chỉ chạy CI trên nhánh chính — lỗi tới quá muộn; nên chạy trên **pull request** và chặn merge khi đỏ (branch protection).
- Cấp `permissions` rộng; dùng PAT cá nhân thay vì `GITHUB_TOKEN`.
- Build lại cho từng môi trường thay vì thăng cấp cùng artifact.
- Test chập chờn (flaky) bị "bấm chạy lại" cho qua — làm mất niềm tin; sửa hoặc cách ly.
- Tag `latest` cho image triển khai; không có cách xác định "đang chạy phiên bản nào".
- Migration phá vỡ tương thích ngược trong rolling deployment.
- Không có smoke test/giám sát sau triển khai; không diễn tập rollback.
- Secret in ra log, lưu trong image, hoặc commit vào `.github`.
- Pipeline quá chậm, không cache.

Bài tập

1. Đẩy repo lên GitHub, bật workflow `kho-clean.yml`, tạo PR có test đỏ và xem PR bị chặn khi bật *branch protection*.
2. Bật **khóa phụ thuộc** (`packages.lock.json`) và `cache: true` cho `setup-dotnet`; so sánh thời gian restore.
3. Viết workflow `phat-hanh` đẩy image lên GHCR khi gắn tag `v1.0.0`; thêm environment `production` với người duyệt.
4. Thêm bước quét `dotnet list package --vulnerable --include-transitive` và làm hỏng pipeline nếu có lỗi hỏng.
5. Thêm **Dependabot** (`.github/dependabot.yml`) cho NuGet, GitHub Actions và Docker, lịch hàng tuần.
6. Vẽ kế hoạch **expand/contract** để đổi tên cột `Nhom` → `DanhMuc` mà không ngừng dịch vụ.

Chương 16 — Triển khai lên máy chủ và đám mây

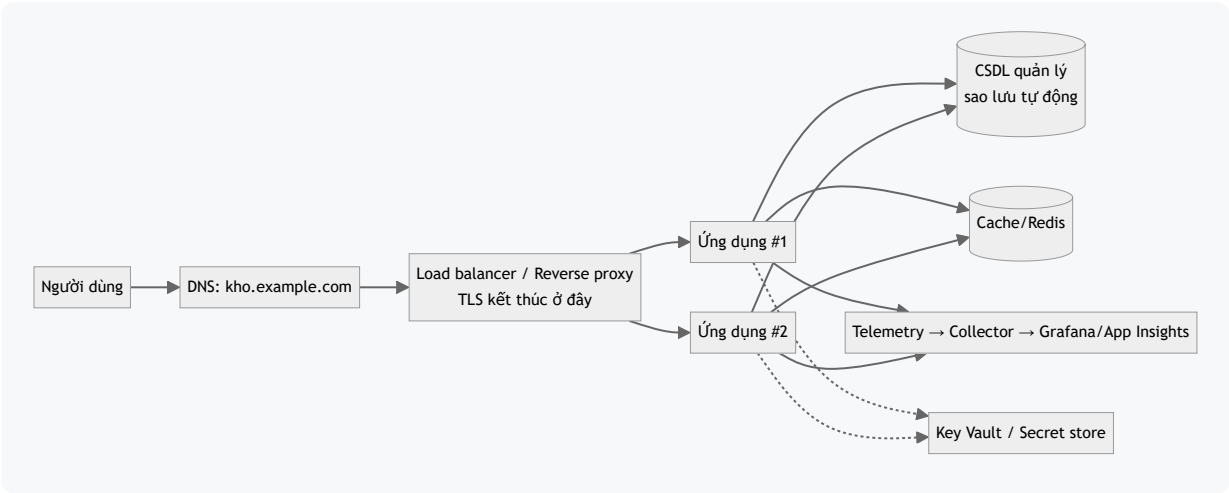
Mục tiêu học

Sau chương này, bạn sẽ:

- Có **bản đồ các lựa chọn hosting** cho ứng dụng [ASP.NET Core](#): máy chủ ảo (VPS), PaaS, container serverless, Kubernetes — và tiêu chí chọn.
- Triển khai kiểu **truyền thống**: Linux + Kestrel + reverse proxy + systemd + TLS.
- Cấu hình ứng dụng đứng **sau proxy/load balancer** đúng cách (forwarded headers, health probe, tắt êm).
- Nắm các thành phần **sản xuất** cần có: CSDL quản lý, secret, sao lưu, tên miền và TLS, giám sát, hạ tầng dưới dạng mã (IaC), chi phí.

Lưu ý trung thực: chương này là **hướng dẫn và mẫu cấu hình**; tác giả không triển khai lên đám mây thật trong lúc viết sách nên các file mẫu (nginx, systemd, Kubernetes) chưa được chạy thử. Chúng theo tài liệu chính thức; hãy kiểm tra trên môi trường của bạn, bắt đầu ở quy mô nhỏ. Tên dịch vụ và giá thay đổi — đối chiếu tài liệu mới nhất của nhà cung cấp.

Bức tranh: ứng dụng của bạn cần những gì để "lên mạng"



Đây là hình mẫu chung cho hầu hết hệ thống web: **DNS → TLS/proxy → nhiều bản ứng dụng không trạng thái → CSDL riêng → cache, telemetry, secret**. "Không trạng thái" là điều kiện để nhân bản: phiên đăng nhập, cache, tệp tải lên phải ở **kho ngoài** (CSDL, Redis, blob storage), không ở bộ nhớ/đĩa của một instance.

Các lựa chọn hosting

Lựa chọn	Ví dụ	Bạn quản lý	Hợp khi
VPS / máy ảo	DigitalOcean, Vultr, Linode, EC2, Azure VM, VPS Việt Nam	HĐH, cập nhật, proxy, TLS, giám sát, sao lưu	học, dự án nhỏ, kiểm soát tối đa, chi phí cố định thấp
PaaS (Platform-as-a-Service)	Azure App Service, AWS Elastic Beanstalk/App Runner, Google App	ứng dụng và cấu hình	đội nhỏ, muốn ít vận hành, mở rộng đơn giản

Lựa chọn	Ví dụ	Bạn quản lý	Hợp khi
	Engine, Render, Railway, Fly.io		
Container serverless	Azure Container Apps, Google Cloud Run, AWS ECS Fargate	image + cấu hình mở rộng	chạy container mà không quản máy chủ; mở rộng về 0 , trả theo dùng
Kubernetes	AKS, EKS, GKE, tự dựng (k3s)	rất nhiều (nhưng cực linh hoạt)	nhiều dịch vụ, đội có năng lực vận hành nền tảng
Serverless hàm	Azure Functions, AWS Lambda	mã hàm	tác vụ theo sự kiện, tải thất thường

Lời khuyên thực dụng: **bắt đầu đơn giản nhất phù hợp** — thường là PaaS hoặc container serverless (một image, HTTP, giám sát sẵn). Kubernetes mạnh nhưng có **chi phí vận hành lớn**; chọn khi thực sự cần (nhiều dịch vụ, yêu cầu tùy biến mạng/lập lịch), đừng chọn vì "thời thượng". Tránh khoá chặt: nếu ứng dụng đã là container + cấu hình bằng biến môi trường (Chương 14), chuyển giữa các nền tảng khá dễ.

Cách 1: Linux + Kestrel + reverse proxy (truyền thống)

Kestrel là máy chủ web nhanh, sẵn có trong [ASP.NET](#) Core, nhưng **đặt sau reverse proxy** (nginx, Caddy, YARP, hoặc load balancer đám mây) để: kết thúc TLS, nén, giới hạn tốc độ/kích thước, phục vụ tệp tĩnh, đệm client chậm, cân bằng tải.

1. Publish (hoặc chạy container):

```
dotnet publish Kho.Web -c Release -o /srv/kho -p:UseAppHost=false
```

2. systemd giữ tiến trình sống, khởi động cùng máy, tự khởi động lại khi chết:

```
# /etc/systemd/system/kho.service
[Unit]
Description=Kho web
After=network.target

[Service]
WorkingDirectory=/srv/kho
ExecStart=/usr/bin/dotnet /srv/kho/Kho.Web.dll
Restart=always
RestartSec=5
KillSignal=SIGINT # tắt êm; TimeoutStopSec đủ để hoàn tất request
TimeoutStopSec=30
User=kho # người dùng riêng, KHÔNG root
Environment=ASPNETCORE_ENVIRONMENT=Production
Environment=ASPNETCORE_URLS=http://127.0.0.1:5000 # chỉ nghe nội bộ, proxy đứng trước
EnvironmentFile=/etc/kho/kho.env # chứa ConnectionStrings... (quyền 600, chủ kho)
NoNewPrivileges=true
ProtectSystem=strict
ReadWritePaths=/var/lib/kho

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable --now kho
journalctl -u kho -f          # log (stdout của ứng dụng)
```

3. Reverse proxy — Caddy (TLS Let's Encrypt tự động) là cách ngắn nhất:

```
kho.example.com {
    reverse_proxy 127.0.0.1:5000
    encode zstd gzip
}
```

Hoặc **nginx**:

```
server {
    listen 443 ssl http2;
    server_name kho.example.com;
    ssl_certificate      /etc/letsencrypt/live/kho.example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/kho.example.com/privkey.pem;
    client_max_body_size 10m;

    location / {
        proxy_pass          http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header    Host                $host;
        proxy_set_header    X-Forwarded-For     $proxy_add_x_forwarded_for;
        proxy_set_header    X-Forwarded-Proto   $scheme;
        proxy_set_header    Upgrade             $http_upgrade;    # WebSocket/SignalR/Blazor Server
        proxy_set_header    Connection          $connection_upgrade;
    }
}
```

Ngoài ra: **tường lửa** chỉ mở 80/443 (và SSH có khoá, tắt đăng nhập bằng mật khẩu), **cập nhật bảo mật tự động**, **fail2ban**, sao lưu cơ sở dữ liệu ra nơi khác và **thủ khôi phục**.

Đứng sau proxy: các cấu hình bắt buộc

Khi có proxy/load balancer, ứng dụng thấy kết nối từ **proxy** (HTTP, IP của proxy), không phải từ người dùng. Hậu quả nếu bỏ qua: Request.IsHttps sai (cookie Secure, chuyển hướng HTTPS, OIDC redirect_uri sai scheme), IP client sai (giới hạn tốc độ, audit, log). Sửa bằng **Forwarded Headers**:

```
builder.Services.Configure<ForwardedHeadersOptions>(o =>
{
    o.ForwardedHeaders = ForwardedHeaders.XForwardedFor | ForwardedHeaders.XForwardedProto;
    // Mặc định CHỈ tin proxy ở loopback. Trong container/đám mây, proxy có IP khác:
    // khai báo mạng tin cậy CỤ THỂ, đừng tin tất cả.
    o.KnownIPNetworks.Add(System.Net.IPNetwork.Parse("10.0.0.0/8"));
});

var app = builder.Build();
app.UseForwardedHeaders();           // ĐẶT ĐẦU TIÊN trong pipeline (trước HTTPS redirect, auth, rate ...)
```

Chỉ tin `X-Forwarded-*` từ **proxy của bạn**: nếu tin từ bất kỳ nguồn, kẻ tấn công tự gửi header giả `X-Forwarded-For: 1.2.3.4` để **giả IP** và qua mặt giới hạn tốc độ/danh sách IP. (Nhiều nền tảng đám mây/Kubernetes đã hỗ trợ đặt biến `ASPNETCORE_FORWARDEDHEADERS_ENABLED=true` — bật cả hai header, chỉ dùng khi chắc chắn ứng dụng chỉ nhận lưu lượng qua proxy tin cậy.)

Các điểm khác:

- **UseHttpsRedirection** thường bỏ ở phía ứng dụng khi TLS kết thúc ở proxy (proxy làm chuyển hướng) — hoặc bật cùng forwarded headers đúng.
- **HSTS** (`UseHsts`) trong production.
- **Health probe**: proxy/orchestrator gọi `/health/live` và `/health/ready` (Chương 11).
- **Tắt êm**: nhận SIGTERM → ngừng nhận request mới, hoàn tất request đang chạy, dừng `BackgroundService` (outbox); đặt `HostOptions.ShutdownTimeout` phù hợp thời gian nền tảng chờ.
- **Data Protection keys** (cookie xác thực, antiforgery): khi nhiều bản, phải **chia sẻ chung** (lưu vào CSDL/Redis/blob + bảo vệ bằng khoá ở Key Vault); nếu không, cookie tạo ở bản A không giải mã được ở bản B → người dùng bị "đăng xuất ngẫu nhiên".
- **Đồng hồ và múi giờ**: máy chủ dùng UTC; lưu `DateTimeOffset` UTC, hiển thị múi giờ người dùng.

Cách 2: PaaS và container serverless

Với container (Chương 14) bạn thường chỉ cần: **đẩy image lên registry** → **tạo dịch vụ trỏ tới image** → **đặt biến môi trường/secret** → **mở cổng 8080** → **cấu hình probe và mở rộng**. Ví dụ với Azure Container Apps:

```
az containerapp up --name kho-web --resource-group rg-kho \
  --image ghcr.io/owner/kho-web:1.0.0 --target-port 8080 --ingress external \
  --env-vars ASPNETCORE_ENVIRONMENT=Production ConnectionStrings__Kho=secretref:kho-conn
az containerapp update --name kho-web --resource-group rg-kho --min-replicas 1 --max-replicas 5 \
  --scale-rule-name http --scale-rule-type http --scale-rule-http-concurrency 50
```

(Cú pháp dùng để minh hoạ; tham số chính xác xem `az containerapp --help`.) Google Cloud Run, AWS App Runner/ECS Fargate tương tự: **image + cổng + biến môi trường + giới hạn CPU/RAM + số bản tối thiểu/tối đa**. Ưu điểm: TLS, DNS tùy chọn, mở rộng, health probe và log/metric **có sẵn**. Lưu ý **khởi động lạnh** khi mở rộng về 0 (đặt `min-replicas` ≥ 1 cho dịch vụ nhạy độ trễ; **ReadyToRun/Native AOT** giảm thời gian khởi động).

Cách 3: Kubernetes (cái nhìn nhập môn)

Kubernetes (K8s) chạy và quản **hàng loạt container** theo trạng thái mong muốn bạn khai báo: tự khởi động lại, mở rộng, cập nhật cuốn chiếu, cân bằng tải. Mẫu tối thiểu:

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: kho-web }
spec:
  replicas: 3
  selector: { matchLabels: { app: kho-web } }
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxUnavailable: 0, maxSurge: 1 } # không giảm năng lực khi cập nhật
  template:
    metadata: { labels: { app: kho-web } }
    spec:
```

```

containers:
  - name: web
    image: ghcr.io/owner/kho-web:1.0.0 # ghim tag/digest, không dùng latest
    ports: [{ containerPort: 8080 }]
    env:
      - { name: ASPNETCORE_ENVIRONMENT, value: Production }
      - name: ConnectionStrings__Kho
        valueFrom: { secretKeyRef: { name: kho-secret, key: conn } }
    resources:
      requests: { cpu: 100m, memory: 128Mi }
      limits: { memory: 256Mi }
    startupProbe: { httpGet: { path: /health/live, port: 8080 }, failureThreshold: 30, periodSeconds: 10 }
    livenessProbe: { httpGet: { path: /health/live, port: 8080 }, periodSeconds: 10 }
    readinessProbe: { httpGet: { path: /health/ready, port: 8080 }, periodSeconds: 5 }
    securityContext: { runAsNonRoot: true, allowPrivilegeEscalation: false, readOnlyRootFilesystem: true }
---
apiVersion: v1
kind: Service
metadata: { name: kho-web }
spec:
  selector: { app: kho-web }
  ports: [{ port: 80, targetPort: 8080 }]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: kho-web }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: kho-web }
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 70 } }

```

Đây chính là nơi **liveness vs readiness** (Chương 11) thể hiện: liveness fail → K8s **khởi động lại** container; readiness fail → K8s **rút khỏi Service** (không nhận lưu lượng) mà không restart. **Ingress / Gateway** cung cấp định tuyến HTTP và TLS ngoài cụm. **Helm/Kustomize** đóng gói/tham số hoá manifest; **GitOps** (Argo CD, Flux) đồng bộ trạng thái cụm từ Git.

Thành phần sản xuất khác

Cơ sở dữ liệu

- Dùng **dịch vụ CSDL quản lý** (Azure SQL/PostgreSQL Flexible Server, AWS RDS, Cloud SQL): sao lưu tự động, vá lỗi, sao chép/HA. **Đừng** tự chạy CSDL production trong container không có kế hoạch sao lưu.
- Sao lưu ≠ khôi phục được**: định kỳ **thử khôi phục** và đo thời gian (RTO) và lượng dữ liệu tối đa mất (RPO).
- Kết nối tới CSDL trong mạng riêng, không mở ra Internet; người dùng CSDL của ứng dụng **quyền tối thiểu** (không phải admin).
- Migration: Chương 15 (bundle, expand/contract).

Secret và cấu hình

Key Vault / Secrets Manager, đọc bằng **managed identity**; ứng dụng dùng

`builder.Configuration.AddAzureKeyVault(...)` hoặc nền tảng tiêm biến môi trường. Xoay vòng định kỳ; không commit (Chương 13).

Tên miền, TLS, CDN

DNS trỏ tên miền vào load balancer; **chứng chỉ TLS tự động** (Let's Encrypt/nhà cung cấp quản lý) và tự gia hạn; **CDN/WAF** (Cloudflare, Azure Front Door) đứng trước để cache tài nguyên tĩnh, chống DDoS và các tấn công web phổ biến.

Giám sát và cảnh báo

Bật thu thập **log + metric + trace** (Chương 11) tới nền tảng quan sát; đặt cảnh báo trên triệu chứng người dùng; có runbook và người trực.

Hạ tầng dưới dạng mã (IaC)

Đừng bấm chuột dựng hạ tầng: khai báo bằng **Bicep/ARM** (Azure), **Terraform/OpenTofu** (đa đám mây), **Pulumi** (dùng C#), **AWS CDK**. Lợi ích: tái lập môi trường (dev/staging/prod giống nhau), review qua PR, lịch sử thay đổi, khôi phục sau thảm họa.

```
// Pulumi (C#): hạ tầng cũng là mã C#
var app = new Pulumi.AzureNative.App.ContainerApp("kho-web", new() { /* ... */ });
```

Chi phí

Đặt **ngân sách và cảnh báo chi phí** ngay từ đầu; gắn thẻ (tag) tài nguyên; tắt môi trường dev ngoài giờ; chọn kích cỡ theo **số đo thật**, không đoán. Chi phí bằng thông ra (egress) và log/telemetry thường bất ngờ nhất.

Danh sách kiểm tra "sẵn sàng production"

- [] HTTPS mọi nơi; HSTS; forwarded headers cấu hình đúng.
- [] Không có secret trong mã/image; xoay vòng được.
- [] Health probe live/ready; tắt êm; ít nhất 2 bản nếu cần sẵn sàng cao.
- [] CSDL quản lý, sao lưu, **đã thử khôi phục**; migration có kế hoạch.
- [] Log/metric/trace về một nơi; cảnh báo và runbook.
- [] Giới hạn tốc độ, kích thước request, timeout (Tập 2, Chương 9 và 19).
- [] Data Protection keys chia sẻ khi nhiều bản; không lưu trạng thái cục bộ.
- [] CI/CD tự động, có rollback đã diễn tập.
- [] Cập nhật bảo mật base image/phụ thuộc thường xuyên; quét lỗ hổng.
- [] Đã kiểm thử tải ở quy mô dự kiến (Chương 11).
- [] Ngân sách và cảnh báo chi phí.

Lỗi thường gặp

- Bỏ forwarded headers → redirect vòng lặp, cookie `Secure` không gửi, IP client toàn là IP proxy; hoặc tin mọi `X-Forwarded-For`.
- Cho Kestrel nghe trực tiếp ra Internet không có proxy/TLS đúng.
- Lưu phiên/tệp tải lên/cache trên đĩa hoặc RAM của một instance rồi mở rộng ngang.

- Chạy nhiều bản với **Data Protection keys riêng** → đăng xuất ngẫu nhiên, lỗi antiforgery.
- Không có sao lưu, hoặc sao lưu nhưng chưa từng khôi phục.
- Mở cổng CSDL/Redis ra Internet công cộng.
- Không giới hạn tài nguyên; container OOM-kill mà không có cảnh báo.
- Triển khai thủ công bằng `scp` và sửa tay trên máy chủ ("máy chủ tuyết"): không tái lập được.
- Không đặt ngân sách → hoá đơn bất ngờ.

Bài tập

1. Thuê VPS nhỏ (hoặc dùng máy ảo cục bộ), triển khai `kho-clean` bằng `systemd` + Caddy có TLS thật; kiểm tra `journalctl` và thử `systemctl kill -s SIGTERM` để xem tắt êm.
2. Thêm `UseForwardedHeaders` cho `kho-clean`; dùng `curl -H "X-Forwarded-Proto: https"` để chứng minh `Request.Scheme` đổi khi gọi từ proxy tin cậy nhưng **không** đổi khi từ nguồn khác.
3. Triển khai image lên một dịch vụ container serverless (Cloud Run/Container Apps free tier), đặt secret chuỗi kết nối, thử mở rộng 1→3 bản.
4. Với `minikube` / `kind`, áp dụng các manifest mẫu; bắn tải và quan sát HPA; làm `readiness` fail và xem Pod bị rút khỏi Service.
5. Viết Bicep/Terraform tối thiểu cho một Container App + Log Analytics; đưa vào repo và chạy `plan` / `what-if`.
6. Lập kế hoạch **RTO/RPO** cho hệ thống kho: nếu CSDL hỏng lúc 10:00, khôi phục thế nào, mất tối đa bao nhiêu dữ liệu?

Chương 17 — Microservices và giao tiếp giữa dịch vụ

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **microservices** là gì, cái giá của nó, và **khi nào KHÔNG nên dùng** (đa số dự án nên bắt đầu bằng **modular monolith**).
- Xác định **ranh giới dịch vụ** theo bounded context; hiểu nguyên tắc **mỗi dịch vụ sở hữu dữ liệu của mình**.
- Chọn cách giao tiếp: **đồng bộ** (HTTP/gRPC) hay **bất đồng bộ** (thông điệp/sự kiện qua broker); nắm **saga** thay cho giao dịch phân tán.
- Biết các thành phần nền: **API gateway (YARP)**, service discovery, resilience, quan sát phân tán, **.NET Aspire**.

Lưu ý trung thực: chương này thiên về **tư duy thiết kế** và mẫu mã ngắn theo tài liệu chính thức; dự án của sách (`kho-clean`) cố ý là **một monolith có mô-đun rõ ràng**, và các đoạn YARP/gRPC/MassTransit dưới đây **chưa được chạy** ở máy tác giả. Mục đích là giúp bạn đưa ra quyết định sáng suốt, không phải bán microservices.

Monolith, modular monolith, microservices

	Monolith "spaghetti"	Modular monolith	Microservices
Triển khai	một khối	một khối	nhiều dịch vụ độc lập
Ranh giới	không rõ	rõ ràng trong mã (mô-đun/assembly, chỉ giao tiếp qua hợp đồng)	rõ ràng, ép bởi mạng
Dữ liệu	một CSDL, ai cũng dùng	một CSDL, mỗi mô-đun một schema/bảng riêng	mỗi dịch vụ một CSDL
Độ phức tạp vận hành	thấp	thấp	cao
Đội	nhỏ	nhỏ-vừa	nhiều đội độc lập

Cấu trúc ở Chương 1–2 (Clean Architecture + ranh giới rõ) chính là bước đầu của **modular monolith**: nếu ranh giới giữa các mô-đun đã sạch, việc **tách thành dịch vụ khi thực sự cần** sau này chỉ là chuyện di chuyển, không phải viết lại. Ngược lại, chia nhỏ quá sớm khi ranh giới chưa hiểu rõ tạo ra **"distributed monolith"** — tệ nhất của cả hai thế giới: chậm, dễ hỏng, khó thay đổi như monolith nhưng vận hành phức tạp như microservices.

Microservices mang lại gì — và giá phải trả

Lợi ích: đội triển khai độc lập, mở rộng từng phần theo tải, chọn công nghệ riêng cho từng dịch vụ, cô lập lỗi (một dịch vụ chết không nhất thiết kéo cả hệ thống).

Cái giá (đừng xem nhẹ):

- **Mạng không đáng tin:** mọi lời gọi có thể chậm/lỗi/trùng (Chương 13 idempotency; Tập 2 Chương 19 resilience). Cái từng là lời gọi hàm nay là lời gọi mạng.
- **Nhất quán dữ liệu:** không còn giao dịch ACID xuyên dịch vụ → saga, nhất quán cuối cùng.
- **Vận hành:** mỗi dịch vụ cần CI/CD, giám sát, cấu hình, bảo mật, phiên bản riêng. Cần đội nền tảng.
- **Gỡ lỗi:** một request đi qua chục dịch vụ → bắt buộc có **trace phân tán** (Chương 11).
- **Kiểm thử:** test tích hợp/hợp đồng phức tạp hơn nhiều.
- **Thay đổi xuyên dịch vụ** đắt: sửa hợp đồng cần phối hợp nhiều đội.

Quy tắc kinh nghiệm: *đừng dùng microservices trừ khi bạn có vấn đề mà nó giải quyết* — nhiều đội cần triển khai độc lập, các phần có nhu cầu mở rộng rất khác nhau, hoặc ranh giới miền đã ổn định và được hiểu rõ. Đội nhỏ/sản phẩm mới: **modular monolith trước**.

Xác định ranh giới: bounded context

Dùng **bounded context** của DDD (Chương 3): mỗi ngữ cảnh có mô hình và ngôn ngữ riêng. Ví dụ hệ thống thương mại: **Danh mục** (sản phẩm, giá), **Kho** (tồn, nhập/xuất), **Đặt hàng**, **Thanh toán**, **Giao hàng**, **Khách hàng**. Chữ "Sản phẩm" trong Kho (mã, tồn, vị trí) khác "Sản phẩm" trong Danh mục (mô tả, ảnh, giá). Đó là dấu hiệu ranh giới.

Tiêu chí ranh giới tốt:

- **Gắn kết cao** bên trong, **liên kết lỏng** giữa các dịch vụ.
- Một thay đổi nghiệp vụ thường chỉ chạm **một** dịch vụ.
- Sở hữu **dữ liệu của chính mình**; dịch vụ khác **không truy cập thẳng CSDL** của nó — chỉ qua API/sự kiện.
- Khớp cấu trúc đội (định luật Conway).

Dấu hiệu chia sai: hai dịch vụ luôn phải đổi cùng nhau; một hành động người dùng cần 8 lời gọi đồng bộ nối đuôi; dịch vụ dùng chung một bảng.

Giao tiếp giữa các dịch vụ

Đồng bộ: yêu cầu/đáp ứng

HTTP/REST (JSON) — quen thuộc, dễ gỡ lỗi, tương thích rộng. **gRPC** — nhị phân (Protocol Buffers) trên HTTP/2: nhanh, hợp đồng chặt (.proto), sinh mã client/server, hỗ trợ streaming; tốt cho **gọi nội bộ giữa dịch vụ**, khó dùng trực tiếp từ trình duyệt.

```
// kho.proto – hợp đồng là nguồn sự thật, sinh mã C# cho cả hai phía
service KhoService {
    rpc LayTon (LayTonRequest) returns (LayTonReply);
}
message LayTonRequest { string ma = 1; }
message LayTonReply { int32 ton = 1; }
```

Cách gọi đồng bộ tạo **ghép nối tạm thời (temporal coupling)**: dịch vụ A chỉ chạy khi B đang chạy. Chuỗi A→B→C nhân độ trễ và **nhân xác suất lỗi** (ba dịch vụ 99,9% → còn ≈ 99,7%). Bắt buộc có: **timeout, retry có backoff + jitter, circuit breaker, bulkhead** (Microsoft.Extensions.Http.Resilience — Tập 2, Chương 19) và **idempotency** cho lời gọi ghi.

Bất đồng bộ: thông điệp và sự kiện

Dịch vụ giao tiếp qua **message broker** (RabbitMQ, Azure Service Bus, Kafka, AWS SQS/SNS). Người gửi không chờ; người nhận xử lý khi sẵn sàng.

Loại	Ý nghĩa	Ví dụ
Lệnh (command)	"hãy làm X" — gửi tới một người nhận	TruTonKho
Sự kiện (event)	"X đã xảy ra" — nhều người có thể quan tâm; người gửi không biết ai nghe	DonHangDaDat

Sự kiện giảm ghép nối nhất: dịch vụ Đặt hàng phát DonHangDaDat ; Kho, Thanh toán, Thông báo tự đăng ký lắng nghe — thêm người nghe mới **không sửa** dịch vụ Đặt hàng. Đây là hướng đi của **sự kiện tích hợp (integration event)**, phát qua **Transactional Outbox** (Chương 8) để không mất và không phát khi giao dịch thất bại.

Kèm theo là các sự thật cần chấp nhận:

- Giao **ít nhất một lần** → người nhận phải **idempotent** (inbox/khoá duy nhất — Chương 13).
- **Thứ tự** không đảm bảo toàn cục (Kafka đảm bảo theo partition/khoá; Service Bus theo session).
- **Nhất quán cuối cùng**: dữ liệu ở dịch vụ khác cập nhật sau một khoảng trễ → giao diện/nghiệp vụ phải chịu được.
- **Thư chết (dead-letter queue)** cho thông điệp xử lý mãi không được; cảnh báo và quy trình xử lý.
- Phiên bản hoá hợp đồng sự kiện (thêm trường tương thích, không đổi nghĩa trường cũ).

Thư viện .NET: **MassTransit**, **Wolverine**, **NServiceBus** (thương mại), hoặc SDK gốc của broker (`Azure.Messaging.ServiceBus` , `RabbitMQ.Client`). Ví dụ MassTransit (theo tài liệu):

```
public record DonHangDaDat(Guid DonHangId, string MaSanPham, int SoLuong);

public class DonHangDaDatConsumer(IKhoService kho) : IConsumer<DonHangDaDat>
{
    public async Task Consume(ConsumeContext<DonHangDaDat> ctx)
        => await kho.TruTonAsync(ctx.Message.MaSanPham, ctx.Message.SoLuong, ctx.MessageId!.Value);
}

services.AddMassTransit(x =>
{
    x.AddConsumer<DonHangDaDatConsumer>();
    x.UsingRabbitMq((ctx, cfg) => { cfg.Host("rabbitmq"); cfg.ConfigureEndpoints(ctx); });
});
```

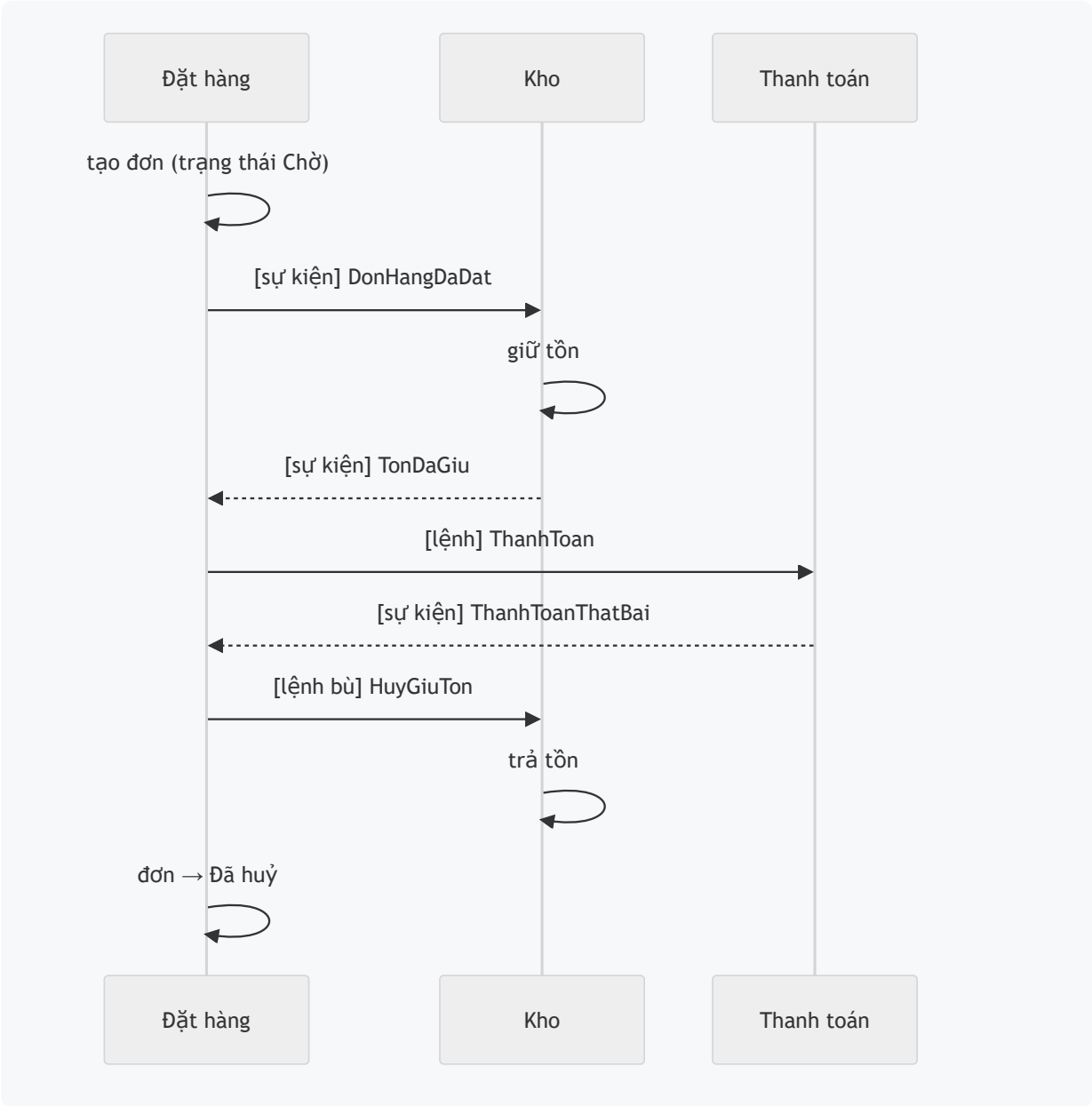
Chọn đồng bộ hay bất đồng bộ? *Truy vấn cần câu trả lời ngay* (xem giá hiện tại) → đồng bộ. *Thông báo một việc đã xảy ra / cần làm mà người dùng không chờ* → bất đồng bộ. Thường kết hợp cả hai; ưu tiên bất đồng bộ để giảm ghép nối cho luồng nghiệp vụ chính.

Dữ liệu và giao dịch phân tán

Mỗi dịch vụ một **CSDL** nghĩa là **không còn JOIN** xuyên dịch vụ và **không còn một giao dịch cho toàn nghiệp vụ**. Hệ quả và cách xử lý:

- **Đọc dữ liệu của dịch vụ khác**: gọi API; hoặc **sao chép cục bộ** bản chiếu chỉ đọc (read model) cập nhật bằng sự kiện (ví dụ Đặt hàng giữ bản sao tên/giá sản phẩm tại thời điểm đặt).
- **Nghiệp vụ nhiều bước** → **Saga**: chuỗi giao dịch cục bộ, mỗi bước có **hành động bù trừ (compensation)** nếu bước sau thất bại.

Saga: đặt hàng



Hai kiểu điều phối:

- **Choreography** (tự phối hợp): mỗi dịch vụ nghe sự kiện và tự phản ứng; không có trung tâm. Đơn giản với ít bước, khó theo dõi khi nhiều bước.
- **Orchestration** (nhạc trưởng): một **state machine/saga orchestrator** ra lệnh từng bước và quản lý bù trừ (MassTransit *state machine*, Wolverine, Temporal, Durable Functions). Dễ hiểu luồng hơn, có điểm tập trung.

Hành động bù trừ **không phải "hoàn tác"**: gửi email rồi không thu hồi được; bù trừ là hành động nghiệp vụ mới (gửi email đính chính). Thiết kế trạng thái trung gian (Chờ , DangGiuTon) hiển thị được cho người dùng.

API Gateway và BFF

Client không nên gọi trực tiếp hàng chục dịch vụ. **API Gateway** là điểm vào duy nhất: định tuyến, xác thực, giới hạn tốc độ, gộp/chuyển đổi, TLS. Trong .NET dùng **YARP** (Yet Another Reverse Proxy, của Microsoft):

```
builder.Services.AddReverseProxy().LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));
app.MapReverseProxy();
```

```
"ReverseProxy": {
  "Routes": {
    "kho": { "ClusterId": "kho", "Match": { "Path": "/api/kho/{**catch-all}" }, "Transforms": [
    "don": { "ClusterId": "don", "Match": { "Path": "/api/don/{**catch-all}" }, "Transforms": [
  ],
  "Clusters": {
    "kho": { "Destinations": { "d1": { "Address": "http://kho-web:8080/" } } },
    "don": { "Destinations": { "d1": { "Address": "http://don-web:8080/" } } }
  }
}
```

BFF (Backend-for-Frontend): một gateway/backend riêng cho **từng loại client** (web, mobile) gộp dữ liệu phù hợp màn hình đó (giảm số lời gọi của client) và giữ token an toàn (Chương 12). Đừng nhồi nghiệp vụ vào gateway — nó chỉ là "ống dẫn thông minh".

Các mối quan tâm xuyên suốt

- **Service discovery**: dịch vụ tìm nhau bằng **tên** (DNS của compose/Kubernetes; `Microsoft.Extensions.ServiceDiscovery`), không hard-code địa chỉ IP.
- **Cấu hình và secret tập trung** (Chương 16).
- **Xác thực giữa dịch vụ**: token dịch vụ (client credentials — Chương 12), mTLS trong service mesh; **không tin** mạng nội bộ chỉ vì "bên trong" (zero trust).
- **Quan sát phân tán**: propagation `traceparent`, correlation, metric theo dịch vụ, dashboard, cảnh báo (Chương 11). Không có nó, microservices là "hộp đen".
- **Hợp đồng và kiểm thử**: **consumer-driven contract test** (Pact) — bên tiêu thụ khai báo kỳ vọng, bên cung cấp kiểm tra được; test tích hợp trong CI với các thành phần thật (Testcontainers).
- **Phiên bản hoá API**: `Asp.Versioning`; tương thích ngược; loại bỏ dần (deprecation).
- **Chống lỗi dây chuyền**: timeout ngắn, circuit breaker, bulkhead, *load shedding*, hạ cấp có kiểm soát (degrade).

.NET Aspire

.NET Aspire là bộ công cụ/khung của Microsoft cho ứng dụng **phân tán**, **"cloud-native"**: khai báo cả hệ thống (dịch vụ, CSDL, cache, broker) trong **một dự án AppHost bằng C#**, tự cấu hình **service discovery**, **health check**, **OpenTelemetry** (dashboard xem log/trace/metric cục bộ), khởi chạy container phụ thuộc, rồi xuất manifest để triển khai (Container Apps, Kubernetes). Phù hợp để phát triển và chạy thử nhiều dịch vụ trên máy dev mà không phải viết compose bằng tay:

```
var builder = DistributedApplication.CreateBuilder(args);
var csdl = builder.AddPostgres("csdl").AddDatabase("kho");
var kho = builder.AddProject<Projects.Kho_Web>("kho-web").WithReference(csdl);
builder.AddProject<Projects.Don_Web>("don-web").WithReference(kho);
builder.Build().Run();
```

(Mã minh hoạ theo tài liệu; Aspire tiến hoá nhanh, xem tài liệu phiên bản hiện hành.)

Con đường thực tế: từ monolith tới dịch vụ

1. Bắt đầu **modular monolith** với ranh giới sạch (mô-đun giao tiếp qua giao diện/sự kiện nội bộ; **kiểm tra bằng architecture test** như Chương 1 và 9).
2. Đo, tìm phần **thực sự** cần độc lập (mở rộng khác biệt, đội riêng, nhịp phát hành riêng).
3. **Strangler Fig**: dựng dịch vụ mới bên cạnh, dùng gateway chuyển dần lưu lượng từng chức năng, gỡ phần cũ khi xong.
4. Tách **dữ liệu** trước rồi mới tách **mã** (khó nhất là dữ liệu).
5. Đầu tư **nền tảng** (CI/CD, quan sát, cấu hình) **trước** khi có nhiều dịch vụ.

Lỗi thường gặp

- **Distributed monolith**: dịch vụ gọi nhau đồng bộ theo chuỗi, chung CSDL, phải triển khai cùng nhau.
- Chia theo **tầng kỹ thuật** (dịch vụ "Controller", "Repository") thay vì theo **ngành vụ**.
- Chia quá nhỏ ("nano-service") khi đội chỉ có vài người.
- Dùng chung CSDL/bảng giữa các dịch vụ; truy cập thẳng bảng của dịch vụ khác.
- Không xử lý trùng/thất bại khi dùng thông điệp; không có dead-letter; mất/không phát sự kiện vì thiếu outbox.
- Cố duy trì giao dịch ACID phân tán (2PC) xuyên dịch vụ.
- Không có trace phân tán/giám sát; gỡ lỗi bằng cách "SSH vào từng máy".
- Coi microservices là mục tiêu thay vì phương tiện; theo thời thượng khi chưa có nhu cầu.
- Thiếu phiên bản hoá hợp đồng → sửa một dịch vụ phá vỡ các dịch vụ khác.

Bài tập

1. Vẽ bounded context và bản đồ ngữ cảnh (context map) cho hệ thống bán hàng gồm Danh mục, Kho, Đặt hàng, Thanh toán, Giao hàng; ghi rõ **sự kiện** giữa các ngữ cảnh.
2. Trong `kho-clean`, xác định các "điểm nối" nếu sau này tách Kho thành dịch vụ: giao diện nào đã sẵn (`ISanPhamRepository`, sự kiện outbox `TonKhoThayDoi`)? Cái gì còn thiếu?
3. Thiết kế saga đặt hàng: liệt kê trạng thái, sự kiện, lệnh và **hành động bù trừ** cho từng lỗi có thể (hết hàng, thanh toán thất bại, giao hàng lỗi). So sánh choreography với orchestration.
4. Tính xác suất khả dụng của chuỗi 4 dịch vụ mỗi cái 99,9%; giải thích vì sao gọi bất đồng bộ cải thiện được.
5. Viết cấu hình YARP cho hai dịch vụ giả (chạy hai bản `kho-clean` ở hai cổng) và xác minh định tuyến theo tiền tố đường dẫn.
6. (Nâng cao) Thêm **MassTransit + RabbitMQ** (Docker) vào `kho-clean`: `OutboxProcessor` phát `TonKhoThayDoi` lên RabbitMQ, một consumer console in ra và xử lý idempotent.

Chương 18 — Dự án tổng hợp Tập 3: Hệ thống đặt hàng

Mục tiêu học

Chương này gom mọi thứ của Tập 3 vào **một hệ thống**: mở rộng `kho-clean` bằng mô-đun **Đơn hàng** (đặt hàng nhiều dòng, huỷ đơn), rồi rà soát nó qua từng chương: kiến trúc, miền, CQRS, Result, outbox, idempotency, audit, kiểm thử, quan sát, đóng gói và triển khai.

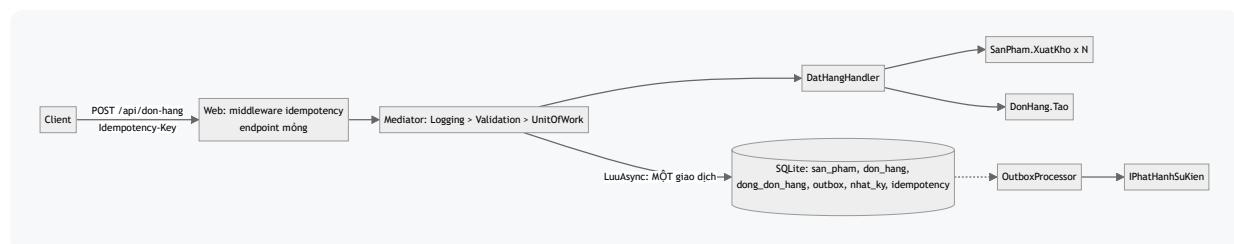
Code: [code/kho-clean/](#) — thư mục `Kho.Domain/DonHangs`, `Kho.Application/DonHangs`, `Kho.Web/Endpoints/DonHangEndpoints.cs`, `Kho.Tests/DonHangTests.cs`.

Trạng thái kiểm chứng (đọc kỹ): mã Đơn hàng **biên dịch được**, migration đã sinh, và các test **domain** của nó chạy được. Nhưng các test **tích hợp** (đặt hàng, huỷ đơn, đặt đồng thời, idempotency, outbox) **chưa chạy được ở máy tác giả**: Windows chặn nạp `Kho.Infrastructure.dll` bằng chính sách Application Control (lỗi `0x800711C7`), dù trước đó 45 test cũ đều đạt trên cùng thiết lập. Vì vậy các khẳng định về hành vi end-to-end dưới đây là **thiết kế và kỳ vọng của test**, chưa phải kết quả đã quan sát. Hãy chạy `dotnet test` ở máy bạn; nếu có test đỏ, coi đó là một phần của bài tập gỡ lỗi.

Yêu cầu

- Đặt hàng gồm **nhiều dòng** (mã sản phẩm + số lượng). Mỗi dòng trừ kho.
- Tất cả hoặc không gì cả**: một dòng thiếu hàng thì không dòng nào bị trừ.
- Đơn lưu **bản chụp đơn giá** lúc đặt (giá đổi sau này không đổi đơn cũ).
- Huỷ đơn** trả lại tồn, và không huỷ được hai lần.
- Nhiều người đặt cùng lúc món cuối cùng: **tồn không bao giờ âm**.
- Gửi lại request (mạng rớt) với `Idempotency-Key` **không tạo đơn thứ hai**.
- Sự kiện `DonHangDaDat` / `DonHangDaHuy` đi qua **outbox** để hệ thống khác (thanh toán, thông báo) tiêu thụ.

Thiết kế



Hai aggregate, một use case. `SanPham` và `DonHang` là hai aggregate riêng (Chương 3): `DonHang` chỉ giữ **mã** sản phẩm và bản chụp giá, không giữ đối tượng `SanPham`. Use case đặt hàng (tầng Application) **điều phối** cả hai trong **một giao dịch** — chấp nhận được khi cùng một CSDL. Nếu sau này `Kho` và `Đơn hàng` thành hai dịch vụ, chỗ này biến thành **saga** (Chương 17): giữ tồn → tạo đơn → bù trừ khi lỗi.

Miền: DonHang

```
public static Result<DonHang> Tao(string ma, string? kháchHang, IEnumerable<DongDon> dong, DateTimeOffset
```

```

// kiểm tra: có khách, 1..50 dòng, không trùng sản phẩm, số lượng > 0 → Result.Fail(Loi.DuLieuKh
var don = new DonHang { Ma = ma, KhachHang = khachHang.Trim(), TrangThai = TrangThaiDon.DaDat, Da
don._dong.AddRange(ds);
don.TongTien = ds.Aggregate(Tien.Khong, (tong, d) => tong + d.ThanhTien);
don.PhatSuKien(new DonHangDaDat(ma, don.KhachHang, don.TongTien.SoTien, ds.Count, bayGio));
return don;
}

public Result Huy(string? lyDo, DateTimeOffset bayGio)
{
    if (TrangThai == TrangThaiDon.DaHuy) return Result.Fail(Loi.NghiepVu("DonHang.DaHuy", ...)); //
    TrangThai = TrangThaiDon.DaHuy;
    PhatSuKien(new DonHangDaHuy(Ma, ..., bayGio));
    return Result.Ok();
}

```

Bất biến do chính aggregate giữ; trạng thái đổi qua phương thức nghiệp vụ; `DongDon` là thực thể con (owned) sống và chết cùng đơn.

Ứng dụng: đặt hàng tất-cả-hoặc-không

```

foreach (var d in cmd.Dong)
{
    var sp = await sanPhams.LayTheoMaAsync(d.MaSanPham, ct);
    if (sp is null) return Result<string>.Fail(Loi.KhongTimThay(...));
    var xuat = sp.XuatKho(d.SoLuong, bayGio);
    if (!xuat.ThanhCong) return Result<string>.Fail(xuat.Loi!); // trả lỗi NGAY
    dong.Add(new DongDon(sp.Ma.GiaTri, d.SoLuong, sp.DonGia)); // bản chụp giá
}
var don = DonHang.Tao(ma, cmd.KhachHang, dong, bayGio);
donHangs.Them(don.GiaTri);
return ma;

```

Tính nguyên tử đến từ **hai thứ phối hợp** (Chương 6 và 7): handler trả `Result` thất bại → `UnitOfWorkBehavior` **không gọi** `LuuAsync` nên các thay đổi trong bộ nhớ bị bỏ; còn khi thành công, mọi thay đổi (tồn nhiều sản phẩm, đơn, dòng đơn, outbox, audit) được ghi trong **một giao dịch**. Cạnh tranh đồng thời do **concurrency token** `TonKho` xử lý: người thua nhận `DbUpdateConcurrencyException` → `ConcurrencyException` → 409 (Chương 8).

Hủy đơn: đổi trạng thái rồi `NhapKho` lại từng dòng, cùng giao dịch — một **hành động bù trừ** nội bộ.

Hạ tầng: ánh xạ

`DonHang` ↔ bảng `don_hang`, `DongDon` ↔ `dong_don_hang` qua `OwnsMany` (khóa ngoài + cascade); `Tien` chuyển `double` cho SQLite (đã gặp ở Chương 4); trạng thái lưu chuỗi; `Ma` có chỉ mục duy nhất. `KhoDbContext.LuuAsync` vẫn **là chỗ duy nhất** gom sự kiện miền vào outbox — mô-đun mới không cần viết thêm dòng nào cho outbox, vì nó dựa trên `Entity<int>` chung. Đó là lợi ích của việc thiết kế tổng quát ở Chương 8.

Web

```

POST /api/don-hang
Idempotency-Key: dat-hang-7d2c

```

```
X-Nguoi: an
{ "khachHang": "An", "dong": [ { "maSanPham": "LT001", "soLuong": 2 }, { "maSanPham": "CH002", "soLuong": 1 } ] }
→ 201 { "ma": "DH-3F9A0C51B2E7" }
```

Endpoint mở: dừng lệnh, gửi qua mediator, dịch `Result` → HTTP (400/404/409/422) bằng `ToHttp()` có sẵn.
Middleware idempotency (Chương 13) bảo vệ nó **mà không sửa một dòng nghiệp vụ**.

Bản đồ: mỗi chương của Tập 3 hiện diện ở đâu

Chương	Trong hệ thống
1–2 Phân lớp, Clean Architecture	4 dự án, quy tắc phụ thuộc, architecture test
3 Domain modeling	SanPham, DonHang, value object Tien/MaSanPham, sự kiện miền
4 Repository/Specification	ISanPhamRepository, IDonHangRepository, read model SanPhamDoc
5–6 CQRS, Pipeline	DatHangCommand, LayDonHangQuery, Logging>Validation>UnitOfWork
7 Result/Validation	Result<string>, FluentValidation cho lệnh đặt hàng, ToHttp()
8 Sự kiện & Outbox	DonHangDaDat → bảng outbox → OutboxProcessor
9 Kiểm thử	domain test, integration test với SQLite tệp tạm
10–11 Cache, quan sát	(mở rộng) cache đọc đơn/sản phẩm; span cho DatHang
12 OIDC	thay X-Nguoi bằng claim sub của JWT
13 Idempotency/Audit	Idempotency-Key, nhát_ky_kiem_toan
14–16 Docker, CI/CD, cloud	Dockerfile, docker-compose.yml, .github/workflows/kho-clean.yml
17 Microservices	ranh giới Kho/Đơn hàng sẵn sàng tách khi cần

Các test được thiết kế (cần chạy ở máy bạn)

Test	Ý định
Tao_TinhTongTien_VaPhatSuKienDonHangDaDat (domain)	tổng tiền và sự kiện đúng
Tao_KhongDong_HoacTrungSanPham... (domain)	các bất biến
Huy_HaiLan_LanHaiLoiNghiepVu (domain)	không huỷ hai lần
DatHang_TruKhoTatCaCacDong...	trừ kho mọi dòng, đọc lại đơn
DatHang_MotDongThieuHang_422_VaKhongDongNaoBiTru	tính nguyên tử
HuyDon_TraLaiTon_VaKhongHuyDuocLanHai	bù trừ tồn
DatHangDongThoi_ChiDatDuocDungSoLuongCon...	12 request tranh 5 món → đúng 5 thành công, tồn 0
DatHang_CoIdempotencyKey...VaSuKienDiQuaOutbox	một đơn, một sự kiện phát hành

Bài tập (phần chính của chương)

1. **Chạy và sửa.** Chạy `dotnet test` ở máy bạn. Test nào đỏ? Đọc lỗi, tìm nguyên nhân (gợi ý các điểm nhạy cảm: ánh xạ `OwnsMany` với field `_dong`, chuyển `decimal` sang `double`, thứ tự ràng buộc khoá ngoài) và

sửa. Ghi lại nguyên nhân.

2. **Nhập thanh toán.** Thêm trạng thái `DaThanhToan` và lệnh `ThanhToanDonHang` (quy tắc: chỉ đơn `DaDat` mới thanh toán; không huỷ được đơn đã thanh toán, hoặc huỷ kèm hoàn tiền). Viết domain test trước.
3. **Đọc hiệu quả.** Thêm read model `DonHangDoc` + truy vấn phân trang danh sách đơn của một khách (không nạp aggregate), kèm test.
4. **Quan sát.** Thêm `ActivitySource / Meter` như Chương 11: đếm `don_hang_dat` theo kết quả, span `DatHang` với tag số dòng.
5. **Bảo mật.** Áp Chương 12: yêu cầu JWT; đơn thuộc `sub` của người đặt; chỉ chủ đơn hoặc `quan-tri` mới huỷ/xem.
6. **Tách dịch vụ (thiết kế).** Vẽ saga khi Đơn hàng tách khỏi Kho: sự kiện, lệnh, bù trừ, và ba điều bạn phải đổi trong code hiện tại.
7. **Triển khai.** Build image, chạy compose, đưa lên một dịch vụ container, và bật pipeline CI của Chương 15.

Lỗi thường gặp

- Để handler `SaveChanges` riêng → mất tính nguyên tử và bỏ qua behavior.
- Giữ tham chiếu đối tượng giữa hai aggregate, kéo theo nạp/khoá lan rộng.
- Quên bản chụp giá: đổi giá sản phẩm làm đổi lịch sử đơn.
- Tin vào chuỗi các thử nghiệm "chắc là đúng" khi chưa chạy — như chương này đã nói thẳng, kết quả chưa quan sát không được gọi là đã kiểm chứng.

Phụ lục A — Lỗi và bẫy kiến trúc thường gặp

Bảng tra nhanh các sai lầm hay gặp khi áp dụng nội dung Tập 3, kèm dấu hiệu nhận biết và hướng xử lý. Số trong ngoặc là chương liên quan.

Kiến trúc và phân lớp (1–2)

Bẫy	Dấu hiệu	Cách xử lý
Domain phụ thuộc EF Core/ASP.NET	<code>using Microsoft.EntityFrameworkCore</code> trong Domain	Chiều phụ thuộc hướng vào trong; ánh xạ ở Infrastructure; thêm architecture test
Controller/endpoint chứa nghiệp vụ	<code>if/else</code> quy tắc trong endpoint	Đẩy vào aggregate/handler; endpoint chỉ dịch HTTP
Application trả <code>IQueryable</code>	truy vấn EF rò lên trên	Cổng dạng phương thức async trả DTO
Chia lớp theo kỹ thuật cho dự án nhỏ	ba tầng chỉ chuyển tiếp lời gọi	Đơn giản hoá; kiến trúc phải tương xứng độ phức tạp
Interface cho mọi thứ	mỗi lớp có <code>IFoo</code> một cài đặt	Chỉ tạo cổng ở ranh giới thật (I/O, ngoài hệ thống)

Miền và dữ liệu (3–4)

Bẫy	Dấu hiệu	Xử lý
Anemic model	entity chỉ có getter/setter, logic ở service	Chuyển hành vi vào entity; setter private
Aggregate quá lớn	nạp cả cây, khoá xung đột liên tục	Aggregate nhỏ; tham chiếu bằng id
Giữ đối tượng giữa aggregate	<code>don.SanPham.TonKho--</code>	Tham chiếu bằng mã; điều phối ở use case
Value object có setter	"Tien" bị sửa giữa chừng	Bất biến, so sánh theo giá trị
Repository lộ mọi truy vấn	40 phương thức <code>LayTheo...</code>	Repository theo aggregate; đọc qua read model
Specification thay cho việc đơn giản	pattern cho truy vấn dùng một lần	Chỉ dùng khi tái sử dụng/kết hợp điều kiện
SQLite: <code>decimal</code> , <code>DateTimeOffset</code> , <code>Contains</code>	lỗi dịch LINQ / sai thứ tự	Đổi sang <code>double</code> /binary converter, <code>EF.Functions.Like</code> — hoặc dùng CSDL máy chủ

CQRS, pipeline, Result (5–7)

Bẫy	Xử lý
Bọc CQRS + mediator cho CRUD đơn giản	Chỉ dùng khi lợi ích rõ (pipeline dùng chung, tách đọc/ghi)
Truy vấn đi qua <code>UnitOfWork</code> behavior	Chỉ ràng buộc behavior này vào <code>ICommand</code>
Sai thứ tự behavior	Logging > Validation > <code>UnitOfWork</code> ; có test kiểm thứ tự

Bẫy	Xử lý
Ném exception cho lỗi nghiệp vụ dự kiến	Dùng Result; exception cho điều bất thường
Lăn lỗi nghiệp vụ với lỗi hạ tầng	Hạ tầng ném exception cụ thể, Application dịch
Validate lặp ở nhiều tầng	Định dạng ở validator; bất biến ở domain — mỗi cái một vai
Bỏ qua Result (không kiểm ThanhCong)	Analyzer/test; API Match; không truy cập .GiaTri bừa
Ánh xạ lỗi → HTTP rải rác	Một điểm ToHttp() duy nhất

Sự kiện và outbox (8)

- Phát sự kiện **trước** khi commit → phát ma khi giao dịch thất bại. Dùng outbox cùng giao dịch.
- Consumer không idempotent; xử lý trùng gây tác dụng phụ kép.
- Outbox không dọn dẹp → bảng phình; không giới hạn số lần thử/không dead-letter.
- Nhiều instance cùng lấy một dòng outbox → cần khoá hàng/ SKIP LOCKED hoặc cập nhật có điều kiện.
- Sự kiện chứa cả entity thay vì dữ liệu tối thiểu, phiên bản hoá kém.

Kiểm thử (9)

- Mock quá đà → test chỉ kiểm cách gọi, không kiểm hành vi. Ưu tiên fake/CSDL thật cho tích hợp.
- :memory: chia sẻ kết nối làm hỏng test đồng thời → dùng tệp tạm.
- Test phụ thuộc thứ tự/dữ liệu chung; dùng dữ liệu riêng mỗi test (mã duy nhất).
- Sleep để chờ thay vì điều khiển thời gian (TimeProvider / FakeTimeProvider).
- Đuổi theo % độ phủ, bỏ qua test hành vi quan trọng; test chấp chớn bị bỏ qua.

Caching (10)

- Cache không có chính sách vô hiệu hoá; TTL vô hạn.
- **Stampede**: nhiều request cùng nạp một khoá hết hạn → dùng HybridCache /khoá.
- Cache dữ liệu theo **người dùng** trong cache dùng chung (rò rỉ dữ liệu); output cache cho response có cookie/ Authorization .
- Cache trong bộ nhớ khi chạy nhiều instance mà cần nhất quán.
- Cache kết quả rỗng/lỗi; khoá không phân biệt tham số.

Quan sát (11)

- Quên AddSource / AddMeter cho nguồn tự viết.
- Nhãn metric có cardinality cao (UserId).
- Log dữ liệu nhạy cảm; log ở mức Debug trên production.
- Kiểm tra phụ thuộc trong **liveness** → restart dây chuyền.
- Tối ưu không đo; cảnh báo không có runbook.

Bảo mật và độ tin cậy (12–13)

- Implicit/Password flow; token trong localStorage ; không PKCE; redirect_uri ký tự đại diện.
- Không kiểm tra aud / iss ; nhầm 401 với 403; chỉ dùng role, bỏ kiểm tra quyền sở hữu.
- Idempotency kiểu "kiểm tra rồi ghi" (race); lưu kết quả 5xx; không đối chiếu nội dung với khoá.

- Audit ngoài giao dịch; cho phép sửa/xoá audit; ghi bí mật vào log/audit.
- Secret trong Git/image; khoá mã hoá cạnh dữ liệu.

Đóng gói và triển khai (14–17)

- Docker: chép mã trước `restore` ; thiếu `.dockerignore` ; chạy root; dùng `latest` ; quên volume; nghe `localhost` trong container.
- CI/CD: `permissions` rộng; build lại cho từng môi trường; migration phá tương thích ngược; thiếu smoke test/rollback.
- Cloud: quên forwarded headers; tin mọi `X-Forwarded-*` ; trạng thái cục bộ khi nhân bản; Data Protection keys không chia sẻ; chưa từng thử khôi phục sao lưu.
- Microservices: **distributed monolith**, chung CSDL, chia theo tầng kỹ thuật, không có trace phân tán, cố dùng giao dịch phân tán.

Nguyên tắc chung khi gặp lỗi lạ

1. Tái hiện tối thiểu (một test đỏ).
2. Đọc **thông báo lỗi và inner exception** đầy đủ; đọc SQL EF sinh ra (`LogTo /log` mức `Information` của `Microsoft.EntityFrameworkCore.Database.Command`).
3. Dùng `traceId` xuyên log/trace (Chương 11).
4. Thay đổi **một** thứ mỗi lần; viết test khoá lỗi trước khi sửa.
5. Nghi ngờ môi trường (chính sách hệ điều hành, phiên bản SDK, cổng bận) trước khi nghi ngờ trình biên dịch — và nếu môi trường chặn thì báo cho người có quyền, đừng lách.

Phụ lục B — Tài liệu tham khảo

Danh sách nguồn nên đọc tiếp. Địa chỉ và phiên bản có thể thay đổi; hãy đối chiếu bản mới nhất.

Tài liệu chính thức

- **.NET / [ASP.NET](#) Core / EF Core:** [learn.microsoft.com/dotnet](#) , [learn.microsoft.com/aspnet/core](#) , [learn.microsoft.com/ef/core](#) .
- **Kiến trúc:** *.NET Microservices: Architecture for Containerized .NET Applications* và *Architecting Modern Web Applications with [ASP.NET](#) Core and Azure* (eBook miễn phí của Microsoft).
- **OpenTelemetry:** [opentelemetry.io](#) (đặc tả, tài liệu .NET, Collector).
- **OAuth 2.0 / OIDC:** RFC 6749 (OAuth 2.0), RFC 7636 (PKCE), RFC 9700 (best current practice), OpenID Connect Core 1.0, RFC 7519 (JWT), RFC 9110 (ngữ nghĩa HTTP).
- **W3C Trace Context:** [w3.org/TR/trace-context](#) .
- **Docker:** [docs.docker.com](#) (Dockerfile best practices, Compose). **Kubernetes:** [kubernetes.io/docs](#) .
- **GitHub Actions:** [docs.github.com/actions](#) .
- **YARP:** [microsoft.github.io/reverse-proxy](#) . **.NET Aspire:** tài liệu trên [learn.microsoft.com](#).
- **Nghị định 13/2023/NĐ-CP** về bảo vệ dữ liệu cá nhân (Việt Nam); GDPR nếu phục vụ EU.

Sách

Sách	Tác giả	Chủ đề
<i>Clean Architecture</i>	Robert C. Martin	nguyên tắc phụ thuộc, ranh giới
<i>Domain-Driven Design</i>	Eric Evans	miền, bounded context, aggregate
<i>Implementing Domain-Driven Design</i>	Vaughn Vernon	DDD thực hành
<i>Patterns of Enterprise Application Architecture</i>	Martin Fowler	Repository, Unit of Work...
<i>Designing Data-Intensive Applications</i>	Martin Kleppmann	nhất quán, giao dịch, hệ phân tán
<i>Building Microservices</i>	Sam Newman	microservices, ranh giới, dữ liệu
<i>Release It!</i>	Michael Nygard	ổn định, circuit breaker, bulkhead
<i>Site Reliability Engineering</i>	Google	SLO, cảnh báo, vận hành
<i>Accelerate</i>	Forsgren, Humble, Kim	CI/CD, hiệu quả phát hành
<i>Unit Testing: Principles, Practices, and Patterns</i>	Vladimir Khorikov	kiểm thử có giá trị
<i>Dependency Injection Principles, Practices, and Patterns</i>	Mark Seemann, Steven van Deursen	DI

Thư viện và công cụ nhắc tới trong Tập 3

Nhu cầu	Công cụ
Validation	FluentValidation
Mediator	MediatR (thương mại từ 2025 — xem giấy phép), Wolverine, hoặc tự viết như Chương 5
Kiểm thử	xUnit, NSubstitute, WebApplicationFactory, Testcontainers, Bogus, Verify
Kiểm tra kiến trúc	NetArchTest, ArchUnitNET
Cache	Microsoft.Extensions.Caching.Hybrid, Redis (StackExchange.Redis)
Quan sát	OpenTelemetry, Grafana (Tempo/Loki/Prometheus), Jaeger, Application Insights
Định danh	Keycloak, Microsoft Entra ID, Auth0, Duende IdentityServer, OpenIddict
Messaging	MassTransit, Wolverine, NServiceBus, RabbitMQ, Azure Service Bus, Kafka
Gateway	YARP
Hạ tầng dưới dạng mã	Bicep, Terraform/OpenTofu, Pulumi
Tải	k6, NBomber; vi mô: BenchmarkDotNet
Chẩn đoán	dotnet-counters, dotnet-trace, dotnet-dump

Blog và cộng đồng

- Blog .NET của Microsoft (`devblogs.microsoft.com/dotnet`), bản ghi chú phát hành .NET và [ASP.NET](#) Core.
- Blog của Milan Jovanović, Andrew Lock, Steve Smith (Ardalis), Nick Chapsas, Jimmy Bogard (tác giả AutoMapper/MediatR).
- Martin Fowler (`martinfowler.com`): mô tả các pattern kinh điển.
- Cộng đồng: `r/dotnet` , Stack Overflow, GitHub Discussions của dotnet, các nhóm .NET Việt Nam.

Ba tập của bộ sách

- **Tập 1** — C# và .NET nền tảng: ngôn ngữ, thư viện, bất đồng bộ, kiểm thử, ứng dụng dòng lệnh.
- **Tập 2** — [ASP.NET](#) Core: HTTP, Minimal API, MVC/Razor Pages/Blazor, EF Core, xác thực, bảo mật, dự án tổng hợp.
- **Tập 3** — Kiến trúc và vận hành: Clean Architecture, DDD, CQRS, outbox, kiểm thử, cache, quan sát, OIDC, Docker, CI/CD, cloud, microservices.

Phụ lục C — Cheat sheet kiến trúc và triển khai

Quy tắc phụ thuộc (Clean Architecture)

Web → Application → Domain

└─→ Infrastructure → Application (cài đặt các "cổng")

Domain: không phụ thuộc gì. Application: chỉ Domain. Infrastructure/Web: nối mọi thứ ở composition rồi

Chọn mẫu thiết kế

Bạn cần...	Dùng
Bảo vệ quy tắc nghiệp vụ	aggregate + phương thức nghiệp vụ, setter private
Giá trị không có danh tính	value object bất biến (record)
Lỗi nghiệp vụ dự kiến	Result<T> (không exception)
Việc dùng chung cho mọi use case	pipeline behavior
Tách đọc/ghi	CQRS: lệnh qua aggregate, truy vấn qua read model/DTO
Phát sự kiện đáng tin	outbox trong cùng giao dịch
Gửi lại an toàn	Idempotency-Key + khoá duy nhất (INSERT nguyên tử)
Ai làm gì	audit log cùng giao dịch, chỉ thêm
Nghệp vụ nhiều dịch vụ	saga (bù trừ), không 2PC

Pipeline mặc định

Logging > Validation > UnitOfWork > Handler — lệnh thành công thì `LuuAsync` một lần; thất bại thì không lưu gì.

Mã trạng thái theo loại lỗi

LoaiLoi	HTTP
DuLieuKhongHopLe	400
KhongTimThay	404
XungDot (trùng, đồng thời)	409
NghiepVu	422
(không lường trước)	500 ProblemDetails, không lộ chi tiết
Chưa xác thực / không đủ quyền	401 / 403

Cache

Công cụ	Dùng cho
IMemoryCache	cache cục bộ đơn giản, một instance
HybridCache	L1+L2, chống stampede , tag để vô hiệu hoá
Output cache	cả response HTTP, chính sách + tag
Redis	cache phân tán/nhiều instance

Luôn có: TTL, chiến lược vô hiệu hoá, khoá gồm mọi tham số ảnh hưởng kết quả, không cache dữ liệu theo người dùng ở nơi dùng chung.

Quan sát

- **Logs** (chuyện gì), **Metrics** (xu hướng), **Traces** (đi đâu) — nối bằng `TraceId`.
- Metric nhãn **cardinality thấp**; không `UserId / OrderId`.
- Bốn tín hiệu vàng: latency, traffic, errors, saturation.
- **Liveness** (còn sống? — không kiểm tra phụ thuộc) $\neq$ **Readiness** (nhận tải được? — kiểm tra phụ thuộc).

OAuth 2.0 / OIDC

- Web/SPA/mobile: **Authorization Code + PKCE**. Không Implicit/Password.
- **ID token** = ai là người dùng (cho client); **access token** = quyền gọi API (cho resource server).
- API kiểm: chữ ký, `iss`, `aud`, `exp`, `scope` /role. `MapInboundClaims = false`.
- Máy-với-máy: **Client Credentials** (ưu tiên managed identity).

Docker

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build      # copy *.csproj → restore → copy code → publish
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final  # copy publish; USER $APP_UID; cổng 8080
ENTRYPOINT ["dotnet", "App.dll"]
```

Biến môi trường: `A:B` → `A__B`. Dữ liệu ở volume/CSDL ngoài. Không secret trong image. Không `latest`.

Lệnh hay dùng

```
dotnet new webapi -n Ten && dotnet run
dotnet build -c Release -warnaserror
dotnet test --logger trx
dotnet publish -c Release -o out -p:UseAppHost=false

dotnet ef migrations add Ten -p Infrastructure -s Web -o Persistence/Migrations
dotnet ef database update -p Infrastructure -s Web
dotnet ef migrations bundle -p Infrastructure -s Web          # tệp chạy migration cho CI/CD

dotnet user-secrets init && dotnet user-secrets set "Khoa" "giatri"
dotnet list package --vulnerable --include-transitive
dotnet-counters monitor -p <PID>
```

```
docker build -t app:dev . && docker run --rm -p 8080:8080 -v data:/data app:dev
docker compose up --build / logs -f / down
```

Triển khai và phát hành

- **CI**: restore → build (-warnaserror) → test → quét → dựng artifact **một lần**.
- **CD**: thăng cấp cùng artifact; staging → smoke test → duyệt → production.
- Chiến lược: recreate, **rolling**, **blue-green**, **canary**; DB theo **expand/contract**.
- Migration chạy như bước riêng (bundle), không để mọi instance tự migrate.
- Nhiều instance: Data Protection keys chia sẻ; không trạng thái cục bộ; forwarded headers **chỉ tin proxy của bạn**.

Danh sách kiểm tra ngăn cho một thay đổi

- [] Quy tắc nghiệp vụ nằm ở Domain, có test domain?
- [] Use case là lệnh/truy vấn rõ ràng; lỗi dự kiến trả `Result` ?
- [] Ghi dữ liệu có thể bị gửi lặp → idempotent?
- [] Thay đổi trạng thái quan trọng có audit; sự kiện đi qua outbox?
- [] Có log/metric/trace đủ để điều tra khi lỗi?
- [] Migration tương thích ngược; có kế hoạch rollback?
- [] Không secret/PII trong mã, log, image?
- [] CI xanh; đã chạy thật (không chỉ "biên dịch được")?

